# M08 — Prácticas integrales: publicación mediante API Gateway

**Autor del curso: JAIME GALLO**

## Cómo utilizar el manual

Continúa `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`: 34 proyectos, 22 proyectos de pruebas, 403 pruebas y 42 documentos. Cada punto crea una copia independiente de la etapa anterior. Los 96 pasos conservan los ejercicios de análisis y añaden progresivamente un gateway ejecutable.

**Requisitos.** Visual Studio con desarrollo ASP.NET, SDK .NET 8 (compatible con el global.json heredado), SQL Server Express o LocalDB, Python 3 y PowerShell 7. Para ejecutar las pruebas con contenedores se requiere Docker Desktop; para el recorrido de eventos se requiere RabbitMQ. Conserva los proyectos hexagonales, la observabilidad, Polly y outbox/inbox ya construidos.

**SQL Server para las pruebas.** El helper `tests/Compartido/SqlServerPruebas.cs` lee `ACERIA_TEST_SQLSERVER` y crea bases únicas. Con LocalDB: `$env:ACERIA_TEST_SQLSERVER = "Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True"`. La cuenta de SQL Server debe poder crear y eliminar las bases de prueba. Sin la variable, el helper utiliza Testcontainers.MsSql 3.10.0 y SQL Server 2022 Express mediante Docker. Una conexión inválida hace fallar las pruebas: no se sustituyen por memoria ni por SQLite.

**RabbitMQ.** Desde la carpeta de etapa, `docker compose up -d` inicia el broker didáctico heredado. Para una instancia existente configura `$env:ACERIA_TEST_RABBITMQ = "amqp://curso:curso-local@localhost:5672/"`. Las credenciales son exclusivamente de laboratorio. Los workers de las aplicaciones están desactivados por defecto y se activan expresamente en el recorrido de eventos. Las pruebas heredadas pueden necesitar el broker aunque no lo utilice el proxy.

**Convención de edición.** Cada bloque muestra una ruta relativa a la etapa, una operación y el archivo completo. Crear exige que no exista; reemplazar exige que exista. Los cambios se hacen en la copia actual, conservando intactas las etapas anteriores. Las notas de lectura destacan decisiones técnicas, contratos y comprobaciones específicas; no se repiten explicaciones de llaves, importaciones o líneas vacías. Las citas numeradas que se conservan corresponden a las líneas físicas de su bloque de código. Los cuatro retos se ejecutan en copias aparte, sin acumularlos en los puntos obligatorios.

**Alcance de los catálogos.** Sus flags describen políticas y decisiones; no autentican usuarios, no limitan tráfico, no comprueban salud ni prueban disponibilidad. Clientes.Api sigue siendo un ejemplo conceptual. El sistema ejecutable publica Producción e Inventario mediante YARP y sus pruebas comprueban ese comportamiento concreto. La protección productiva de red, identidad, TLS, autorización y cuotas requiere configuración adicional; este módulo no declara que ya exista.

# Preparar Docker Desktop en Windows para consumir RabbitMQ

El alumno trabaja en **Windows con PowerShell, Visual Studio, .NET 8 y SQL Server Express LocalDB**. En este módulo Docker Desktop solo aloja **RabbitMQ**: Producción, Inventario y (en M08) YARP se ejecutan directamente con `dotnet run`. El despliegue conjunto de todas las aplicaciones en contenedores queda fuera de esta práctica.

1. Instala y abre Docker Desktop en Windows con **WSL 2** y contenedores **Linux**. Antes de arrancar el broker ejecuta `docker version`, `docker compose version` y `docker info --format '{{.OSType}}'`; la última comprobación debe mostrar `linux`. Si Docker no responde, inicia Docker Desktop y repite la comprobación.
2. Abre PowerShell en la **carpeta de la etapa concreta** que estés trabajando, donde debe existir `compose.yml` (se crea en M06 y se hereda en M07/M08). Comprueba `Test-Path .\compose.yml`; debe devolver `True`. Ejecuta `docker compose -f .\compose.yml config --quiet`; cualquier error indica que no puedes continuar.
3. Ejecuta `docker compose -f .\compose.yml up -d rabbitmq` y comprueba el estado con `docker compose -f .\compose.yml ps`. Espera al arranque real con `docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping`; si devuelve error, espera unos segundos y repite, sin dar la práctica por superada.
4. Comprueba los puertos locales `5672` (AMQP) y `15672` (administración): `Test-NetConnection localhost -Port 5672` y `Test-NetConnection localhost -Port 15672` deben devolver `TcpTestSucceeded=True`. Abre `http://localhost:15672/` con `curso` / `curso-local` **solo en este laboratorio**.
5. En **cada terminal PowerShell que ejecute un servicio**, configura `$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'` y, cuando el ejercicio indique mensajería, `$env:Comunicacion__WorkersActivos = 'true'`. Los procesos no heredan las variables de otras ventanas. Conserva LocalDB como motor SQL, salvo que el paso exija explícitamente otra instalación.
6. Si ya tienes otro broker en `5672`, no levantes dos Compose con los mismos puertos: utiliza el que está activo si coincide con la configuración del ejercicio o detén primero el anterior. Para detener sin borrar mensajes: `docker compose -f .\compose.yml stop rabbitmq`; para recuperar: `docker compose -f .\compose.yml start rabbitmq` y repite el diagnóstico. No uses `down -v` cuando debas conservar el volumen y las proyecciones.
7. Si aparecen errores, consulta `docker compose -f .\compose.yml logs --tail 80 rabbitmq`, revisa los puertos ocupados y la salud del broker. Un contenedor arrancado no demuestra que el flujo de outbox/inbox funcione: ejecuta después las peticiones, comprobaciones y pruebas E2E del punto.

**Criterio para continuar:** broker disponible, conexión AMQP verificada y servicios Windows listos para consumirlo. La validación definitiva de mensajería se realiza en los procedimientos funcionales del módulo, no solo mediante `docker compose ps`.

# Inventario esperado y comprobación

Los valores siguientes son criterios que deben verificarse ejecutando la suite. Los informes del módulo registran los resultados observados y sus hashes.

| Punto | Catálogo nuevo | Funcionales nuevas | Pruebas acumuladas | Proyectos | Proyectos de tests | Documentos |
|---|---|---|---|---|---|---|
| M07 7.4 | — | — | 403 | 34 | 22 | 42 |
| 8.1 | 12 | 0 | 415 | 35 | 23 | 44 |
| 8.2 | 10 | 26 | 451 | 37 | 24 | 46 |

| Punto | Catálogo nuevo | Funcionales nuevas | Pruebas acumuladas | Proyectos | Proyectos de tests | Documentos |
|---|---|---|---|---|---|---|
| 8.3 | 10 | 15 | 476 | 38 | 25 | 48 |
| 8.4 | 10 | 5 | 491 | 39 | 26 | 50 |

# Punto 8.1 — Introducción al patrón API Gateway

## Definir qué publica el gateway y qué responsabilidades conserva cada servicio

Un API Gateway ofrece un punto de entrada común, pero no sustituye los casos de uso ni las reglas de negocio de Producción e Inventario. Antes de instalar YARP vamos a identificar las rutas que necesitan los clientes, los servicios que responderán y las políticas que se aplicarán en el borde. Es importante distinguir esta decisión de un simple balanceador: el gateway puede transformar y enrutar solicitudes, además de aportar observabilidad y otras funciones transversales.

La base es el estado final de M07, con logging, health checks, resiliencia y comunicación mediante RabbitMQ. El catálogo que construiremos en 8.1 registra decisiones y restricciones; sus banderas de autenticación o limitación de tráfico no activan middleware. El proceso de proxy real aparecerá en los siguientes puntos, cuando podamos enviar peticiones HTTP a través de él y comprobar la respuesta del backend.

**Carpeta de trabajo:** `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway`.

**Objetivo.** Completar los 24 pasos, conservar la etapa anterior y comprobar el catálogo y el comportamiento que incorpora este punto.

### PASO 8.1.1 — Crear la estructura del módulo 8 y del punto 8.1

El primer punto de M08 parte de la solución completa de M07 7.4, conservando sus API y la mensajería. Trabajar sobre una copia permite identificar los cambios del gateway sin sobrescribir el material que el alumno ya ha probado.

1. En el Explorador de archivos localiza `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly` y comprueba que contiene `AceriaApp.sln`, `src`, `tests`, `docs` y `global.json`.
2. Crea la carpeta `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway` y copia allí el contenido de la etapa anterior, incluidas sus carpetas ocultas de configuración.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; abre únicamente la copia `8.1-introduccion-api-gateway` para realizar este punto.
4. Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Verificación concreta.** Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Diagnóstico si falla.**

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

- Copiar desde el punto 7.3 en lugar del 7.4: se pierde el contenido del último punto del módulo 7.

## PASO 8.1.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

La compilación comprueba que la etapa anterior está completa y que el SDK .NET 8 resuelve sus dependencias.

1. Abre en Visual Studio el archivo `AceriaApp.sln` de `8.1-introduccion-api-gateway`.
2. Abre Terminal en esa carpeta y ejecuta `dotnet sln AceriaApp.sln list`; deben aparecer 34 proyectos.
3. Ejecuta `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Verificación concreta.** Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Diagnóstico si falla.**

- Abrir la solución del punto 7.4 en lugar de la del punto 8.1: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 7.4 en ejecución: el puerto queda ocupado y la compilación falla.

## PASO 8.1.3 — Verificar que la carpeta `docs` contiene los documentos del punto 7.4

Esta etapa añade dos documentos sin sobrescribir las 42 fichas y decisiones anteriores.

1. Expande `docs` en Solution Explorer y ejecuta `(Get-ChildItem docs -Filter *.md).Count`.
2. Comprueba que hay 42 documentos heredados y que incluyen los ADR-010 a ADR-030.
3. Compara los nombres y contenidos con `docs` de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`.
4. Confirma que todavía no existen `docs/introduccion-api-gateway.md` ni `docs/ADR-031-introduccion-api-gateway.md`.

**Verificación concreta.** Confirma que todavía no existen `docs/introduccion-api-gateway.md` ni `docs/ADR-031-introduccion-api-gateway.md`.

**Diagnóstico si falla.**

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 7.4.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

## PASO 8.1.4 — Crear el documento de introducción al API Gateway en `docs`

La ficha de diseño debe describir las rutas públicas y los servicios que realmente existirán. Producción e Inventario son backends ejecutables; Clientes es todavía una alternativa conceptual y no debe presentarse como un tercer proceso activo.

1. Crea `docs/introduccion-api-gateway.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/introduccion-api-gateway.md`. **Operación:** Crear el contenido completo.

```
# Introducción al patrón API Gateway — Punto 8.1

## Definición

Componente que actúa como punto de entrada único a un conjunto de servicios.

## Responsabilidades

- Enrutamiento: dirigir la petición al servicio.
- Traducción: puede adaptar protocolos cuando se implementa esa capacidad; este laboratorio publica HTTP
mediante un proxy inverso.
- Agregación: puede combinar respuestas mediante una implementación específica; el proxy de este módulo no
incorpora agregación de negocio.
- Políticas transversales: puede concentrar autenticación, autorización y limitación de tasa si se configuran
sus componentes. Los flags del catálogo no las activan.

## Lo que no debe hacer

- Contener reglas de negocio.
- Transformar el modelo del dominio.
- Introducir un punto único de fallo sin medidas de disponibilidad. Una sola instancia del gateway sí
constituye ese riesgo; la redundancia requiere un despliegue adicional.
- Almacenar el estado de negocio de los servicios.
- Acceder directamente a la base de datos.

## Diferencia con balanceador

| Aspecto | Balanceador | Gateway |
|---------|-------------|---------|
| Distribuye entre | Instancias de un servicio o un conjunto de destinos | Servicios y, si se configura, sus
instancias |
| Capa | Puede operar en transporte o en aplicación | Aplicación |
| Conoce rutas | Un balanceador HTTP puede conocerlas | Usa rutas para seleccionar el servicio |

## Patrones

- Gateway único.
- Gateway por cliente.
- Gateway por dominio.
- BFF (Backend For Frontend).

## Funciones transversales

- Autenticación, autorización y limitación de tasa: capacidades posibles que requieren registro y
configuración; no están implementadas por los indicadores de los catálogos.
- Registro: el gateway ejecutable reutiliza el middleware JSON de M07.
- Monitorización: las señales del gateway complementan las de cada servicio; no sustituyen sus comprobaciones
de dependencias.

## Seguridad

Un gateway puede aplicar políticas de seguridad configuradas expresamente. En este laboratorio se delimitan
las rutas publicadas y se valida el identificador de correlación. No se han instalado autenticación,
autorización ni cuotas mediante los flags del catálogo. La validación de negocio sigue en los servicios;
impedir el acceso directo a los backends requiere controles de red adicionales.

## Observabilidad

El gateway registra sus propias peticiones y propaga el identificador de correlación validado por el
middleware de M07. Producción e Inventario conservan sus propios registros y sus responsabilidades de salud.
Una cabecera de correlación inválida se sustituye en el borde y el valor validado se reenvía a los servicios.

## Ventajas

- Simplificación del cliente.
- Centralización de políticas.
- Ocultación de la topología.
- Observabilidad.

## Inconvenientes

- Punto único de fallo.
- Latencia adicional.
```

```
- Complejidad operativa.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Gateway | Catálogo de diseño en 8.1; proceso AceriaApp.Gateway con YARP a partir de 8.2 |
| Rutas | Recursos de Producción e Inventario; Clientes es un ejemplo conceptual |
| Políticas | Registro y correlación implementados; autenticación y cuotas solo declaradas en el catálogo |
| Punto de entrada | Gateway público en 5100; los puertos de backend siguen accesibles en el entorno local |
| Validación | Tests del catálogo; desde 8.2 se añaden pruebas de proxy HTTP real |

## Alcance comprobable de esta etapa

Los flags AplicaAutenticacion y AplicaRateLimit son declaraciones del catálogo. No activan middleware.
Clientes.Api es un ejemplo de diseño; no existe un tercer backend implementado en este laboratorio.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

Define este encabezado en el documento que el alumno conservará junto al proyecto. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

Añade una fila a la tabla documental; sus columnas deben conservarse al copiarla.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

## PASO 8.1.5 — Crear la carpeta `ApiGateway` en el proyecto de Producción

La carpeta separa el modelo y su validador de las implementaciones HTTP y del dominio. El catálogo describe decisiones; no modifica por sí mismo el proxy.

1. En Solution Explorer expande `src/AceriaApp.Api`.
2. Crea dentro de ese proyecto una carpeta llamada `ApiGateway`.
3. Comprueba que su ruta es `src/AceriaApp.Api/ApiGateway` y que no está dentro de otra carpeta de catálogo.
4. Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Verificación concreta.** Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Diagnóstico si falla.**

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

## PASO 8.1.6 — Crear la clase `RutaGateway`

RutaGateway representa una propuesta de ruta junto con políticas declaradas. Sus propiedades no habilitan autenticación ni rate limiting; solo el middleware efectivamente registrado puede aplicar esas protecciones.

1. Crea `src/AceriaApp.Api/ApiGateway/RutaGateway.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ApiGateway/RutaGateway.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ApiGateway;

public class RutaGateway
{
    public string RutaExterna { get; init; } = string.Empty;
    public string ServicioDestino { get; init; } = string.Empty;
    public string RutaInterna { get; init; } = string.Empty;
    public bool AplicaAutenticacion { get; init; }
    public bool AplicaRateLimit { get; init; }
    public bool AplicaCorrelacion { get; init; }
}
```

Sitúa los tipos en un espacio de nombres propio, separado del resto de etapas y responsabilidades. Declara el tipo responsable de este comportamiento; sus miembros quedan dentro del bloque siguiente. Expone la propiedad indicada; init limita su asignación a la construcción del objeto y evita cambios posteriores accidentales.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

## PASO 8.1.7 — Crear la clase `ValidadorApiGateway`

ValidadorApiGateway analiza datos del catálogo, no intercepta solicitudes HTTP. Al probarlo hay que distinguir la coherencia de una ruta descrita de la capacidad de YARP para reenviar una petición real.

1. Crea `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ApiGateway;

public class ValidadorApiGateway
{
```

```
    public ResultadoApiGateway Validar(IEnumerable<RutaGateway> rutas)
    {
        var errores = new List<string>();
        var lista = rutas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay rutas declaradas.");
        }

        foreach (var ruta in lista)
        {
            if (string.IsNullOrWhiteSpace(ruta.RutaExterna))
            {
                errores.Add("Una ruta no declara la ruta externa.");
            }

            if (string.IsNullOrWhiteSpace(ruta.ServicioDestino))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara el servicio destino.");
            }

            if (string.IsNullOrWhiteSpace(ruta.RutaInterna))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara la ruta interna.");
            }

            if (!ruta.AplicaAutenticacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica autenticación.");
            }

            if (!ruta.AplicaRateLimit)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica limitación de tasa.");
            }

            if (!ruta.AplicaCorrelacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica correlación.");
            }
        }

        return new ResultadoApiGateway(errores.Count == 0, errores);
    }
}

public record ResultadoApiGateway(bool EsValida, IReadOnlyList<string> Errores);
```

Declara la operación, sus parámetros y su resultado; el bloque siguiente contiene su implementación. Obtiene o prepara este valor local para las comprobaciones y operaciones que siguen. Materializa la secuencia para consultar y recorrer los mismos elementos sin repetir su enumeración.

Evalúa esta condición antes de continuar; el bloque asociado trata específicamente ese caso. Acumula un incumplimiento del contrato didáctico; el resultado final indicará que la declaración no es válida. Recorre los elementos de la colección para aplicarles la misma comprobación.

Devuelve el resultado calculado al llamador y termina este recorrido del método. Define un contrato de datos con los campos que el consumidor utilizará para interpretar el resultado.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `ToList` no se resuelve.
- No comprobar el caso de lista vacía: el validador devuelve válido sin rutas.

## PASO 8.1.8 — Añadir los `using` necesarios en `ValidadorApiGateway`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. El método `ToList` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ApiGateway;

public class ValidadorApiGateway
{
    public ResultadoApiGateway Validar(IEnumerable<RutaGateway> rutas)
    {
        var errores = new List<string>();
        var lista = rutas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay rutas declaradas.");
        }

        foreach (var ruta in lista)
        {
            if (string.IsNullOrWhiteSpace(ruta.RutaExterna))
            {
                errores.Add("Una ruta no declara la ruta externa.");
            }

            if (string.IsNullOrWhiteSpace(ruta.ServicioDestino))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara el servicio destino.");
            }

            if (string.IsNullOrWhiteSpace(ruta.RutaInterna))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara la ruta interna.");
            }

            if (!ruta.AplicaAutenticacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica autenticación.");
            }

            if (!ruta.AplicaRateLimit)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica limitación de tasa.");
            }

            if (!ruta.AplicaCorrelacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica correlación.");
            }
        }

        return new ResultadoApiGateway(errores.Count == 0, errores);
    }
}
```

```
public record ResultadoApiGateway(bool EsValida, IReadOnlyList<string> Errores);
```

Importa el espacio de nombres requerido por los tipos y métodos utilizados en este archivo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `ToList` no se resuelve.
- Si un using no se resuelve, comprueba el espacio de nombres y la referencia del proyecto; usa la ubicación mostrada en el archivo completo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.1.9 — Crear la clase `CatalogoRutas`

CatalogoRutas reúne decisiones de diseño que ayudarán a preparar el gateway. La lista de nombres y flags no inicia el proxy ni acredita conectividad con Producción o Inventario.

1. Crea `src/AceriaApp.Api/ApiGateway/CatalogoRutas.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ApiGateway/CatalogoRutas.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ApiGateway;

public class CatalogoRutas
{
    public static List<RutaGateway> RutasPorDefecto()
    {
        return new List<RutaGateway>
        {
            new()
            {
                RutaExterna = "/api/v1/ordenes/{**catch-all}",
                ServicioDestino = "Ordenes.Api",
                RutaInterna = "/api/v1/ordenes/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
            },
            new()
            {
                RutaExterna = "/api/v1/inventario/{**catch-all}",
                ServicioDestino = "Inventario.Api",
                RutaInterna = "/api/v1/inventario/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
            },
            new()
            {
                RutaExterna = "/api/v1/clientes/{**catch-all}",
                ServicioDestino = "Clientes.Api",
                RutaInterna = "/api/v1/clientes/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
            }
        };
```

```
    }
}
```

Completa esta expresión o invocación dentro de CatalogoRutas.cs; se conserva íntegra para reproducir el comportamiento. Asigna el valor indicado a esta variable, propiedad u opción dentro de la operación actual.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Devolver `List<T>` mutable: expone la implementación interna.

## PASO 8.1.10 — Añadir el `using` necesario en `CatalogoRutas`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. Abre `src/AceriaApp.Api/ApiGateway/CatalogoRutas.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ApiGateway/CatalogoRutas.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
namespace AceriaApp.Api.ApiGateway;

public class CatalogoRutas
{
    public static List<RutaGateway> RutasPorDefecto()
    {
        return new List<RutaGateway>
        {
            new()
            {
                RutaExterna = "/api/v1/ordenes/{**catch-all}",
                ServicioDestino = "Ordenes.Api",
                RutaInterna = "/api/v1/ordenes/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
            },
            new()
            {
                RutaExterna = "/api/v1/inventario/{**catch-all}",
                ServicioDestino = "Inventario.Api",
                RutaInterna = "/api/v1/inventario/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
            },
            new()
            {
                RutaExterna = "/api/v1/clientes/{**catch-all}",
                ServicioDestino = "Clientes.Api",
                RutaInterna = "/api/v1/clientes/{**catch-all}",
                AplicaAutenticacion = true,
                AplicaRateLimit = true,
                AplicaCorrelacion = true
```

```
            }
        };
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.1.11 — Crear el proyecto de tests del punto 8.1

El punto 8.1 añade un proyecto de tests específico para las clases de API Gateway. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea manualmente `tests/AceriaApp.ApiGateway.Tests` y su archivo `.csproj`; al usar este bloque no se genera `UnitTest1.cs`.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj`. **Operación:** Crear el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Organiza propiedades o referencias del proyecto MSBuild; conserva la apertura y el cierre de los elementos. Configura .NET 8 y las opciones de compilación de este proyecto. Fija la dependencia y su versión para restaurar el mismo paquete en cada equipo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 7.4: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

## PASO 8.1.12 — Añadir la referencia del proyecto de tests a la API de Producción

La referencia permite que el proyecto de tests acceda a las clases de `ApiGateway` del proyecto de Producción.

1. Abre `tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj` y añade la referencia a `AceriaApp.Api` usando el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Referencia el proyecto indicado mediante una ruta relativa a este archivo .csproj.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

## PASO 8.1.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre `tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj` y fija FluentAssertions 6.12.2 con el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

## PASO 8.1.14 — Crear el archivo `ValidadorApiGatewayTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza las rutas que violan las reglas del gateway.

1. Crea `tests/AceriaApp.ApiGateway.Tests/ValidadorApiGatewayTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/ValidadorApiGatewayTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.ApiGateway;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ApiGateway.Tests;

public class ValidadorApiGatewayTests
{
    private readonly ValidadorApiGateway _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoRutas.RutasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinRutas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<RutaGateway>());

        resultado.EsValida.Should().BeFalse();
```

```
        resultado.Errores.Should().Contain(e => e.Contains("No hay rutas"));
    }

    [Fact]
    public void Validar_SinServicioDestino_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = string.Empty, RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el servicio destino"));
    }

    [Fact]
    public void Validar_SinRutaInterna_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = string.Empty,
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara la ruta interna"));
    }

    [Fact]
    public void Validar_SinAutenticacion_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = false, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica autenticación"));
    }

    [Fact]
    public void Validar_SinRateLimit_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = false, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica limitación de tasa"));
    }

    [Fact]
    public void Validar_SinCorrelacion_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = false }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica correlación"));
```

```
    }
}
```

Declara una prueba individual que xUnit debe descubrir y ejecutar. Comprueba esta condición observable; una diferencia hace fallar la prueba en vez de ocultar el problema.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.ApiGateway;`: las clases no se resuelven.
- La sintaxis FluentAssertions exige su paquete y su using; las aserciones equivalentes de xUnit también son válidas.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

## PASO 8.1.15 — Crear el archivo `CatalogoRutasTests.cs`

Los tests verifican que el catálogo por defecto contiene las rutas esperadas y que todas aplican las políticas requeridas.

1. Crea `tests/AceriaApp.ApiGateway.Tests/CatalogoRutasTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/CatalogoRutasTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.ApiGateway;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ApiGateway.Tests;

public class CatalogoRutasTests
{
    [Fact]
    public void RutasPorDefecto_ContieneTresRutas()
    {
        var rutas = CatalogoRutas.RutasPorDefecto();

        rutas.Should().HaveCount(3);
    }

    [Fact]
    public void RutasPorDefecto_ContieneOrdenesInventarioYClientes()
    {
        var rutas = CatalogoRutas.RutasPorDefecto();

        rutas.Should().Contain(r => r.ServicioDestino == "Ordenes.Api");
        rutas.Should().Contain(r => r.ServicioDestino == "Inventario.Api");
        rutas.Should().Contain(r => r.ServicioDestino == "Clientes.Api");
    }

    [Fact]
    public void RutasPorDefecto_TodasAplicanAutenticacion()
    {
        var rutas = CatalogoRutas.RutasPorDefecto();

        rutas.Should().OnlyContain(r => r.AplicaAutenticacion);
    }

    [Fact]
```

```
    public void RutasPorDefecto_TodasAplicanRateLimit()
    {
        var rutas = CatalogoRutas.RutasPorDefecto();

        rutas.Should().OnlyContain(r => r.AplicaRateLimit);
    }

    [Fact]
    public void RutasPorDefecto_TodasAplicanCorrelacion()
    {
        var rutas = CatalogoRutas.RutasPorDefecto();

        rutas.Should().OnlyContain(r => r.AplicaCorrelacion);
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.ApiGateway;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Contain` y `OnlyContain` no se resuelven.
- No verificar las políticas: alguna podría no comprobarse.

## PASO 8.1.16 — Compilar la solución completa

La compilación confirma la continuidad de M07 y la presencia del nuevo material de análisis. Todavía no acredita una petición reenviada a través de YARP, que se incorporará en 8.2.

1. Escribe todos los archivos de ampliación funcional mostrados a continuación antes de compilar; cuando se indica reemplazar, sustituye el contenido de la copia actual.
2. Desde la carpeta actual ejecuta `dotnet sln AceriaApp.sln add (Get-ChildItem src,tests -Recurse -Filter *.csproj | ForEach-Object FullName)` en PowerShell 7.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release -m:1`.
4. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 35 proyectos; el compilador debe terminar con cero errores.

**Verificación concreta.** Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 35 proyectos; el compilador debe terminar con cero errores.

**Diagnóstico si falla.**

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

## PASO 8.1.17 — Ejecutar todos los tests del punto 8.1

Una prueba de catálogo acredita su declaración. Las pruebas funcionales utilizan el proxy y servidores HTTP reales, pero la persistencia y el broker se comprueban con la suite y el recorrido completo.

1. Ejecuta `dotnet test tests/AceriaApp.ApiGateway.Tests/AceriaApp.ApiGateway.Tests.csproj -c Release --no-build` en la terminal.
2. Comprueba que se ejecutan 12 pruebas: 12 de catálogo y 0 funcionales.
3. Si hay un fallo, identifica su nombre y revisa la regla o la respuesta esperada antes de modificar código.
4. Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Verificación concreta.** Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Diagnóstico si falla.**

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

## PASO 8.1.18 — Ejecutar todos los tests de la solución

La suite completa comprueba que el nuevo punto no rompe la persistencia, la mensajería, el dominio, la observabilidad ni la resiliencia desarrollados antes.

1. Configura `ACERIA_TEST_SQLSERVER` para tu SQL Server Express/LocalDB y `ACERIA_TEST_RABBITMQ` para el broker; sigue los requisitos del inicio del manual.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build -m:1`.
3. Comprueba que pasan las 415 pruebas acumuladas, incluidas las heredadas de M07, sin omisiones.
4. Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Verificación concreta.** Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Diagnóstico si falla.**

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

## PASO 8.1.19 — Crear el README del punto 8.1

El README del punto 8.1 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 7.4.

1. Abre `README.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 8.1 — Introducción al patrón API Gateway

## Objetivo

Modelar rutas y responsabilidades del gateway, conservando las decisiones anteriores de AceriaApp.

## Estado del proyecto

Etapa acumulativa con 35 proyectos, 23 proyectos de tests y 44 documentos. El cierre exige 415 pruebas
aprobadas sin omisiones; los resultados se registran en los informes del módulo.

## Cómo ejecutarlo

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test
AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite
completa.
```

```
## Qué se ha añadido respecto al punto anterior

- Modelo RutaGateway, validador ValidadorApiGateway y catálogo CatalogoRutas.
- 12 pruebas de catálogo y 0 pruebas funcionales nuevas.
- Documento temático y ADR-031.
- Reto optativo en copia separada, con una prueba adicional.

## Documentación

- `docs/introduccion-api-gateway.md`.
- `docs/ADR-031-introduccion-api-gateway.md`.
- Las 42 fichas y decisiones anteriores permanecen idénticas.
```

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite completa. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. - `docs/introduccion-api-gateway.md`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `docs/ADR-031-introduccion-api-gateway.md`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar el README del punto 7.4: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 7.4: rompe la separación.

## PASO 8.1.20 — Crear el ADR-031 del punto 8.1

El ADR-031 documenta la decisión de adoptar el patrón API Gateway. Es un artefacto nuevo del punto 8.1 que no modifica los ADR existentes.

1. Crea `docs/ADR-031-introduccion-api-gateway.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/ADR-031-introduccion-api-gateway.md`. **Operación:** Crear el contenido completo.

```
# ADR-031: Adopción del patrón API Gateway

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha cerrado el módulo 7 con observabilidad y resiliencia. Es necesario definir un punto de entrada único para los servicios y centralizar las preocupaciones transversales.

## Decisión
Se adopta el patrón API Gateway como entrada pública de los recursos de Producción e Inventario. El punto 8.1 modela sus responsabilidades y desde 8.2 se ejecuta un proceso YARP independiente. Reutiliza el registro JSON y la correlación de M07; autenticación, autorización y limitación de tasa requieren componentes adicionales y no se implementan mediante flags del catálogo. Clientes es un ejemplo conceptual, sin tercer backend. El gateway no contiene reglas de negocio, no transforma el dominio, no persiste estado de negocio ni accede a las bases. Una instancia única sí introduce un riesgo de disponibilidad; la redundancia es una medida adicional de despliegue. Los tests del catálogo comprueban las declaraciones y las pruebas de proxy verifican el comportamiento HTTP implementado.

## Alternativas consideradas
- Exposición directa de los servicios: descartado por falta de centralización.
```

```
- Gateway por cliente: descartado por simplicidad del proyecto.
- Balanceador sin gateway: no se elige como sustituto de la publicación por rutas y las responsabilidades de entrada adoptadas.

## Consecuencias
- Positivas: simplificación del cliente, centralización de políticas, ocultación de la topología, observabilidad.
- Negativas: punto único de fallo, latencia adicional, complejidad operativa.
- Mitigación prevista: redundancia del gateway, monitorización y despliegue con varias instancias. El laboratorio local no despliega esa redundancia ni bloquea por sí mismo el acceso directo a los backends.

## Alcance comprobable de esta etapa

Los flags AplicaAutenticacion y AplicaRateLimit son declaraciones del catálogo. No activan middleware.
Clientes.Api es un ejemplo de diseño; no existe un tercer backend implementado en este laboratorio.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos indicados en el manual.
```

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-031`: rompe la convención.

## PASO 8.1.21 — Verificar que no se ha modificado ningún archivo del punto 7.4

La progresión modifica copias independientes. Los nuevos archivos o configuraciones de una etapa no deben alterar un punto ya entregado.

1. Abre las carpetas `docs` de la etapa actual y de `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly`.
2. Compara con Get-FileHash los archivos que existían en la etapa anterior; cada hash SHA256 debe coincidir.
3. Comprueba también que los archivos de tests heredados conservan su contenido; solo se incorporan los tests del punto.
4. Verifica que `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.1-introduccion-api-gateway`.

**Verificación concreta.** Verifica que `M07_OBSERVABILIDAD_RESILIENCIA/PROYECTO/7.4-retry-circuit-breaker-polly` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.1-introduccion-api-gateway`.

**Diagnóstico si falla.**

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

## PASO 8.1.22 — Verificar la estructura del punto 8.1

El inventario convierte la comprobación de estructura en una condición observable y detecta omisiones que una compilación aislada podría pasar por alto.

1. Ejecuta `Get-ChildItem -Recurse -File` y revisa las carpetas src, tests y docs.
2. Verifica 35 proyectos, 23 proyectos de pruebas y 44 documentos.

3. Comprueba la presencia de las tres clases de `ApiGateway`, sus tests, el documento temático y ADR-031.
4. Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Verificación concreta.** Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Diagnóstico si falla.**

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 7.4: viola la regla de no modificar.
- Omitir la carpeta `ApiGateway`: los tests no encuentran las clases.

## PASO 8.1.23 — Verificar el `.gitignore` del punto 8.1

Las exclusiones conservan un repositorio reproducible sin binarios temporales ni datos específicos de Visual Studio.

1. Abre `.gitignore` de la etapa actual.
2. Comprueba que contiene bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
3. Compara su contenido con el archivo heredado y conserva sus exclusiones.
4. Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Verificación concreta.** Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Diagnóstico si falla.**

- Modificar el `.gitignore` copiado del punto 7.4: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

## PASO 8.1.24 — Hacer el commit del punto 8.1 y verificar el estado final

Un commit conserva el estado comprobado. El número de commits no tiene que coincidir con el de puntos: las correcciones y revisiones también forman parte del historial.

1. Desde la raíz del repositorio ejecuta `git status --short` y revisa todos los cambios de la etapa.
2. Añade exclusivamente los archivos docentes y del proyecto con `git add M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway`.
3. Ejecuta `git commit -m "Punto 8.1: Introducción al patrón API Gateway"` después de comprobar las pruebas.
4. Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Verificación concreta.** Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Diagnóstico si falla.**

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

# Reto de ampliación (optativo)

añadir un método `Resumen` a `ValidadorApiGateway` que devuelva el número total de rutas, el número de servicios destino distintos y el número de rutas que aplican autenticación, y un test que verifique el cálculo con el catálogo por defecto.

Crea una copia de `8.1-introduccion-api-gateway` llamada `reto-8-1` fuera de las cuatro carpetas obligatorias. La copia anterior permanece intacta. Aplica las tres sustituciones completas siguientes en orden.

## Ampliación 1: añadir el contrato de resumen

**Archivo:** `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ApiGateway;

public class ValidadorApiGateway
{
    public ResultadoApiGateway Validar(IEnumerable<RutaGateway> rutas)
    {
        var errores = new List<string>();
        var lista = rutas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay rutas declaradas.");
        }

        foreach (var ruta in lista)
        {
            if (string.IsNullOrWhiteSpace(ruta.RutaExterna))
            {
                errores.Add("Una ruta no declara la ruta externa.");
            }

            if (string.IsNullOrWhiteSpace(ruta.ServicioDestino))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara el servicio destino.");
            }

            if (string.IsNullOrWhiteSpace(ruta.RutaInterna))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara la ruta interna.");
            }

            if (!ruta.AplicaAutenticacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica autenticación.");
            }

            if (!ruta.AplicaRateLimit)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica limitación de tasa.");
            }

            if (!ruta.AplicaCorrelacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica correlación.");
            }
        }

        return new ResultadoApiGateway(errores.Count == 0, errores);
    }
}

public record ResultadoApiGateway(bool EsValida, IReadOnlyList<string> Errores);
```

```
public record ResumenApiGateway(int Total, int ServiciosDistintos, int ConAutenticacion);
```

## Ampliación 2: implementar el cálculo del resumen

**Archivo:** `src/AceriaApp.Api/ApiGateway/ValidadorApiGateway.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ApiGateway;

public class ValidadorApiGateway
{
    public ResultadoApiGateway Validar(IEnumerable<RutaGateway> rutas)
    {
        var errores = new List<string>();
        var lista = rutas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay rutas declaradas.");
        }

        foreach (var ruta in lista)
        {
            if (string.IsNullOrWhiteSpace(ruta.RutaExterna))
            {
                errores.Add("Una ruta no declara la ruta externa.");
            }

            if (string.IsNullOrWhiteSpace(ruta.ServicioDestino))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara el servicio destino.");
            }

            if (string.IsNullOrWhiteSpace(ruta.RutaInterna))
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no declara la ruta interna.");
            }

            if (!ruta.AplicaAutenticacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica autenticación.");
            }

            if (!ruta.AplicaRateLimit)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica limitación de tasa.");
            }

            if (!ruta.AplicaCorrelacion)
            {
                errores.Add($"La ruta '{ruta.RutaExterna}' no aplica correlación.");
            }
        }

        return new ResultadoApiGateway(errores.Count == 0, errores);
    }

    public static ResumenApiGateway Resumen(IEnumerable<RutaGateway> rutas)
    {
        var lista = rutas.ToList();
        var servicios = lista.Select(r => r.ServicioDestino).Distinct().Count();
        var conAutenticacion = lista.Count(r => r.AplicaAutenticacion);
        return new ResumenApiGateway(lista.Count, servicios, conAutenticacion);
    }
}

public record ResultadoApiGateway(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenApiGateway(int Total, int ServiciosDistintos, int ConAutenticacion);
```

## Ampliación 3: verificar el resultado con el catálogo

**Archivo:** `tests/AceriaApp.ApiGateway.Tests/ValidadorApiGatewayTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.ApiGateway;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ApiGateway.Tests;

public class ValidadorApiGatewayTests
{
    private readonly ValidadorApiGateway _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoRutas.RutasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinRutas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<RutaGateway>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay rutas"));
    }

    [Fact]
    public void Validar_SinServicioDestino_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = string.Empty, RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el servicio destino"));
    }

    [Fact]
    public void Validar_SinRutaInterna_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = string.Empty,
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara la ruta interna"));
    }

    [Fact]
    public void Validar_SinAutenticacion_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = false, AplicaRateLimit = true, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);
```

```
        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica autenticación"));
    }

    [Fact]
    public void Validar_SinRateLimit_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = false, AplicaCorrelacion = true }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica limitación de tasa"));
    }

    [Fact]
    public void Validar_SinCorrelacion_DevuelveError()
    {
        var rutas = new List<RutaGateway>
        {
            new() { RutaExterna = "/api/v1/x", ServicioDestino = "X.Api", RutaInterna = "/api/v1/x",
AplicaAutenticacion = true, AplicaRateLimit = true, AplicaCorrelacion = false }
        };

        var resultado = _validador.Validar(rutas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplica correlación"));
    }

    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorApiGateway.Resumen(CatalogoRutas.RutasPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.ServiciosDistintos.Should().Be(3);
        resumen.ConAutenticacion.Should().Be(3);
    }
}
```

**Ejecución y resultado esperado.** En la copia optativa ejecuta `dotnet test AceriaApp.sln -c Release -m:1`. Deben pasar 416 pruebas; el proyecto `AceriaApp.ApiGateway.Tests` debe ejecutar 13. Verifica que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que los cuatro puntos obligatorios no cambiaron. Registra el reto en un commit independiente cuando lo hayas comprobado.

## Resultado esperado del punto

- Solución autónoma con 35 proyectos y 23 proyectos de pruebas.
- 44 documentos, incluidos el tema del punto y ADR-031.
- 12 pruebas nuevas de catálogo y 0 funcionales; 415 acumuladas sin omisiones.
- Etapa anterior, documentación y tests heredados conservados; el reto sigue separado.

## Conclusión y continuidad

El punto 8.1 ha definido el patrón API Gateway como punto de entrada único a los servicios. El proyecto AceriaApp dispone ahora de un validador automático de rutas del gateway que comprueba la ruta externa, el servicio destino, la ruta interna y las políticas transversales, y doce tests nuevos. Esta base permite avanzar hacia el punto 8.2, donde se estudiará la configuración básica de YARP como implementación concreta del patrón.

---

**Fin del punto 8.1.**

# Punto 8.2 — Configuración básica de YARP

## Levantar YARP y comprobar que una ruta llega al backend correcto

YARP actúa como proxy inverso y utiliza rutas y clusters para decidir a qué destino enviar una petición. Una configuración válida debe indicar qué URL pública acepta el gateway y qué dirección real tiene el backend, sin crear un servicio ficticio para completar el diagrama. En esta etapa conectaremos los procesos Windows de Producción e Inventario al gateway y comprobaremos el reenvío con solicitudes reales.

El catálogo de clusters ayuda a detectar inconsistencias de diseño, pero no demuestra que un destino responda. La evidencia funcional requiere iniciar el proxy, consultar una ruta publicada, comprobar el código recibido y relacionarlo con el servicio al que se envió. Mantendremos la mensajería y SQL Server del proyecto heredado sin trasladar el dominio al gateway.

**Carpeta de trabajo:** `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp`.

**Objetivo.** Completar los 24 pasos, conservar la etapa anterior y comprobar el catálogo y el comportamiento que incorpora este punto.

### PASO 8.2.1 — Crear la carpeta del punto 8.2 a partir del punto 8.1

La copia conserva 35 proyectos y 415 pruebas antes de incorporar los cambios del punto.

1. En el Explorador de archivos localiza `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway` y comprueba que contiene `AceriaApp.sln`, `src`, `tests`, `docs` y `global.json`.
2. Crea la carpeta `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp` y copia allí el contenido de la etapa anterior, incluidas sus carpetas ocultas de configuración.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; abre únicamente la copia `8.2-configuracion-basica-yarp` para realizar este punto.
4. Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Verificación concreta.** Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Diagnóstico si falla.**

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

### PASO 8.2.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

La compilación comprueba que la etapa anterior está completa y que el SDK .NET 8 resuelve sus dependencias.

1. Abre en Visual Studio el archivo `AceriaApp.sln` de `8.2-configuracion-basica-yarp`.
2. Abre Terminal en esa carpeta y ejecuta `dotnet sln AceriaApp.sln list`; deben aparecer 35 proyectos.
3. Ejecuta `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Verificación concreta.** Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Diagnóstico si falla.**

- Abrir la solución del punto 8.1 en lugar de la del punto 8.2: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 8.1 en ejecución: el puerto queda ocupado y la compilación falla.

## PASO 8.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 8.1

Esta etapa añade dos documentos sin sobrescribir las 44 fichas y decisiones anteriores.

1. Expande `docs` en Solution Explorer y ejecuta `(Get-ChildItem docs -Filter *.md).Count`.
2. Comprueba que hay 44 documentos heredados y que incluyen los ADR-010 a ADR-031.
3. Compara los nombres y contenidos con `docs` de `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway`.
4. Confirma que todavía no existen `docs/configuracion-basica-yarp.md` ni `docs/ADR-032-configuracion-basica-yarp.md`.

**Verificación concreta.** Confirma que todavía no existen `docs/configuracion-basica-yarp.md` ni `docs/ADR-032-configuracion-basica-yarp.md`.

**Diagnóstico si falla.**

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 8.1.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

## PASO 8.2.4 — Crear el documento de configuración básica de YARP en `docs`

La configuración básica de YARP debe relacionar rutas públicas con clusters y destinos reales. Conviene documentar las direcciones y comprobar que cada backend puede responder directamente antes de situar el proxy delante.

1. Crea `docs/configuracion-basica-yarp.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/configuracion-basica-yarp.md`. **Operación:** Crear el contenido completo.

```
# Configuración básica de YARP — Punto 8.2

## YARP

Biblioteca de .NET para construir proxies inversos y gateways. El proyecto AceriaApp.Gateway de este módulo
utiliza Yarp.ReverseProxy 2.2.0 y tiene como destino net8.0. Se ejecuta como proceso independiente, sin
acceso a las bases ni referencias al dominio.

## Ruta

Relación entre una petición entrante y un cluster destino.
```

```
## Cluster

Conjunto de destinos a los que se enruta una petición.

## Relación ruta-cluster

Varias rutas pueden apuntar al mismo cluster.

## Integración con ASP.NET Core

    builder.Services.AddReverseProxy()
        .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

    app.MapReverseProxy();


## Estructura de la configuración

    {
      "ReverseProxy": {
        "Routes": {
          "ordenes": {
            "ClusterId": "produccion-cluster",
            "Match": { "Path": "/api/v1/ordenes/{**resto}" }
          },
          "materiales": {
            "ClusterId": "inventario-cluster",
            "Match": { "Path": "/api/v1/materiales/{**resto}", "Methods": [ "GET", "HEAD" ] }
          }
        },
        "Clusters": {
          "produccion-cluster": {
            "Destinations": {
              "produccion": { "Address": "http://localhost:5105/" }
            }
          },
          "inventario-cluster": {
            "Destinations": {
              "inventario-1": { "Address": "http://localhost:5106/" }
            }
          }
        }
      }
    }


## Transformaciones

Adaptan la petición antes de enviarla al destino o la respuesta antes de devolverla al cliente. Las
transformaciones declarativas se configuran en `Transforms` de la ruta. En 8.2 la propagación del
identificador validado y el ajuste de Location se implementan en C#; el middleware de M07 valida la
correlación antes de reenviarla. Las transformaciones del alias de Inventario se incorporan en 8.3.

## Configuración por entorno

Puede complementarse con `appsettings.Development.json`, `appsettings.Production.json` y variables de entorno
según el despliegue. La configuración entregada del gateway está en `appsettings.json`; las direcciones de
destino se pueden sustituir con claves como `ReverseProxy__Clusters__produccion-
cluster__Destinations__produccion__Address`. El recorrido local utiliza Gateway 5100, Producción 5105 e
Inventario 5106. Los nombres DNS ASCII de un entorno de contenedores solo funcionan si esa red los resuelve.

## Buenas prácticas

1. Identificadores descriptivos.
2. Configuración separada por entorno.
3. Transformaciones solo cuando son necesarias.
4. Documentar en el ADR.
5. Validar con un test.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Proyecto gateway | AceriaApp.Gateway |
| Configuración | appsettings.json |
| Rutas | Órdenes y comunicación de Producción; materiales y planificaciones de Inventario |
```

```
| Clusters | produccion-cluster hacia 5105 e inventario-cluster hacia 5106 en 8.2 |
| Transformaciones | Propagación de correlación validada y Location público; el alias de prefijo se añade en
8.3 |
| Validación | Tests del catálogo y del proxy con Kestrel; la integración SQL y RabbitMQ se acredita por
separado |

## Alcance comprobable de esta etapa

El proyecto ejecutable AceriaApp.Gateway se construye en el paso 8.2.16. El catálogo es una descripción
separada, no el archivo que YARP carga. La configuración completa incluye también las rutas de Producción y
comunicación y las planificaciones de Inventario. Clientes.Api sigue siendo conceptual y no existe como
tercer backend del laboratorio. En 8.4 se añade un segundo destino de Inventario en 5107; Producción conserva
un destino.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

Define este encabezado en el documento que el alumno conservará junto al proyecto. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. `Adaptan la petición antes de enviarla al destino o la respuesta antes de devolverla al cliente. Las transformaciones declarativas se configuran en `Transforms` de la ruta. En 8.2 la propagación del identificador validado y el ajuste de Location se implementan en C#; el middleware de M07 valida la correlación antes de reenviarla. Las transformaciones del alias de Inventario se incorporan en 8.3.` Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

`Puede complementarse con `appsettings.Development.json`, `appsettings.Production.json` y variables de entorno según el despliegue. La configuración entregada del gateway está en `appsettings.json`; las direcciones de destino se pueden sustituir con claves como `ReverseProxy__Clusters__produccion-cluster__Destinations__produccion__Address`. El recorrido local utiliza Gateway 5100, Producción 5105 e Inventario 5106. Los nombres DNS ASCII de un entorno de contenedores solo funcionan si esa red los resuelve.` Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Añade una fila a la tabla documental; sus columnas deben conservarse al copiarla.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

## PASO 8.2.5 — Crear la carpeta `ConfiguracionYarp` en el proyecto de Producción

La carpeta separa el modelo y su validador de las implementaciones HTTP y del dominio. El catálogo describe decisiones; no modifica por sí mismo el proxy.

1. En Solution Explorer expande `src/AceriaApp.Api`.
2. Crea dentro de ese proyecto una carpeta llamada `ConfiguracionYarp`.
3. Comprueba que su ruta es `src/AceriaApp.Api/ConfiguracionYarp` y que no está dentro de otra carpeta de catálogo.
4. Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Verificación concreta.** Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Diagnóstico si falla.**

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.

- Crear la carpeta con otro nombre: rompe la convención del curso.

## PASO 8.2.6 — Crear la clase `ClusterYarp`

ClusterYarp describe una agrupación de destinos para el catálogo docente. El proceso ejecutable depende de la configuración de ReverseProxy cargada en el gateway, no de la mera existencia de este objeto.

1. Crea `src/AceriaApp.Api/ConfiguracionYarp/ClusterYarp.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/ClusterYarp.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ConfiguracionYarp;

public class ClusterYarp
{
    public string Identificador { get; init; } = string.Empty;
    public IReadOnlyList<string> Destinos { get; init; } = Array.Empty<string>();
    public bool TieneDireccionValida { get; init; }
    public bool TieneConfiguracionPorEntorno { get; init; }
}
```

Sitúa los tipos en un espacio de nombres propio, separado del resto de etapas y responsabilidades. Declara el tipo responsable de este comportamiento; sus miembros quedan dentro del bloque siguiente. Expone la propiedad indicada; init limita su asignación a la construcción del objeto y evita cambios posteriores accidentales.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar `Identificador` con `string.Empty`: el compilador emite advertencia.
- No inicializar `Destinos` con `Array.Empty<string>()`: el compilador emite advertencia.

## PASO 8.2.7 — Crear la clase `ValidadorConfiguracionYarp`

El validador puede detectar un destino mal descrito, pero no determina su disponibilidad en red. Las pruebas de YARP deben solicitar la ruta publicada y observar el servicio que realmente responde.

1. Crea `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ConfiguracionYarp;

public class ValidadorConfiguracionYarp
{
    public ResultadoConfiguracionYarp Validar(IEnumerable<ClusterYarp> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (cluster.Destinos.Count == 0)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no declara destinos.");
            }

            if (!cluster.TieneDireccionValida)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene dirección válida.");
            }

            if (!cluster.TieneConfiguracionPorEntorno)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene configuración por entorno.");
            }
        }

        return new ResultadoConfiguracionYarp(errores.Count == 0, errores);
    }
}

public record ResultadoConfiguracionYarp(bool EsValida, IReadOnlyList<string> Errores);
```

Declara la operación, sus parámetros y su resultado; el bloque siguiente contiene su implementación. Obtiene o prepara este valor local para las comprobaciones y operaciones que siguen. Materializa la secuencia para consultar y recorrer los mismos elementos sin repetir su enumeración.

Evalúa esta condición antes de continuar; el bloque asociado trata específicamente ese caso. Acumula un incumplimiento del contrato didáctico; el resultado final indicará que la declaración no es válida. Recorre los elementos de la colección para aplicarles la misma comprobación.

Devuelve el resultado calculado al llamador y termina este recorrido del método. Define un contrato de datos con los campos que el consumidor utilizará para interpretar el resultado.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `ToList` no se resuelve.
- No comprobar el caso de lista vacía: el validador devuelve válido sin clusters.

## PASO 8.2.8 — Añadir los `using` necesarios en `ValidadorConfiguracionYarp`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. El método `ToList` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ConfiguracionYarp;

public class ValidadorConfiguracionYarp
{
    public ResultadoConfiguracionYarp Validar(IEnumerable<ClusterYarp> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (cluster.Destinos.Count == 0)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no declara destinos.");
            }

            if (!cluster.TieneDireccionValida)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene dirección válida.");
            }

            if (!cluster.TieneConfiguracionPorEntorno)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene configuración por entorno.");
            }
        }

        return new ResultadoConfiguracionYarp(errores.Count == 0, errores);
    }
}

public record ResultadoConfiguracionYarp(bool EsValida, IReadOnlyList<string> Errores);
```

Importa el espacio de nombres requerido por los tipos y métodos utilizados en este archivo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `ToList` no se resuelve.
- Si un using no se resuelve, comprueba el espacio de nombres y la referencia del proyecto; usa la ubicación mostrada en el archivo completo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.2.9 — Crear la clase `CatalogoClusters`

CatalogoClusters permite analizar varias configuraciones, aunque sus registros no sustituyen el archivo que utiliza el middleware de proxy. Contrasta ambos elementos durante la práctica.

1. Crea `src/AceriaApp.Api/ConfiguracionYarp/CatalogoClusters.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/CatalogoClusters.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.ConfiguracionYarp;

public class CatalogoClusters
{
    public static List<ClusterYarp> ClustersPorDefecto()
    {
        return new List<ClusterYarp>
        {
            new() { Identificador = "ordenes-cluster", Destinos = new[] { "http://aceria-ordenes:5001/" },
TieneDireccionValida = true, TieneConfiguracionPorEntorno = true },
            new() { Identificador = "inventario-cluster", Destinos = new[] { "http://aceria-
inventario:5002/" }, TieneDireccionValida = true, TieneConfiguracionPorEntorno = true },
            new() { Identificador = "clientes-cluster", Destinos = new[] { "http://aceria-clientes:5003/" },
TieneDireccionValida = true, TieneConfiguracionPorEntorno = true }
        };
    }
}
```

Asigna el valor indicado a esta variable, propiedad u opción dentro de la operación actual.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Devolver `List<T>` mutable: expone la implementación interna.

## PASO 8.2.10 — Añadir el `using` necesario en `CatalogoClusters`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. Abre `src/AceriaApp.Api/ConfiguracionYarp/CatalogoClusters.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.

4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/CatalogoClusters.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
namespace AceriaApp.Api.ConfiguracionYarp;

public class CatalogoClusters
{
    public static List<ClusterYarp> ClustersPorDefecto()
    {
        return new List<ClusterYarp>
        {
            new() { Identificador = "ordenes-cluster", Destinos = new[] { "http://aceria-ordenes:5001/" },
TieneDireccionValida = true, TieneConfiguracionPorEntorno = true },
            new() { Identificador = "inventario-cluster", Destinos = new[] { "http://aceria-
inventario:5002/" }, TieneDireccionValida = true, TieneConfiguracionPorEntorno = true },
            new() { Identificador = "clientes-cluster", Destinos = new[] { "http://aceria-clientes:5003/" },
TieneDireccionValida = true, TieneConfiguracionPorEntorno = true }
        };
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.2.11 — Crear el proyecto de tests del punto 8.2

El punto 8.2 añade un proyecto de tests específico para las clases de configuración de YARP. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea manualmente `tests/AceriaApp.ConfiguracionYarp.Tests` y su archivo `.csproj`; al usar este bloque no se genera `UnitTest1.cs`.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj`. **Operación:** Crear el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Organiza propiedades o referencias del proyecto MSBuild; conserva la apertura y el cierre de los elementos. Configura .NET 8 y las opciones de compilación de este proyecto. Fija la dependencia y su versión para restaurar el mismo paquete en cada equipo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 8.1: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

## PASO 8.2.12 — Añadir la referencia del proyecto de tests a la API de Producción

La referencia permite que el proyecto de tests acceda a las clases de `ConfiguracionYarp` del proyecto de Producción.

1. Abre `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj` y añade la referencia a `AceriaApp.Api` usando el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj`.
**Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Referencia el proyecto indicado mediante una ruta relativa a este archivo .csproj.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

## PASO 8.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj` y fija FluentAssertions 6.12.2 con el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj`.
**Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

## PASO 8.2.14 — Crear el archivo `ValidadorConfiguracionYarpTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los clusters que violan las reglas.

1. Crea `tests/AceriaApp.ConfiguracionYarp.Tests/ValidadorConfiguracionYarpTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/ValidadorConfiguracionYarpTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.ConfiguracionYarp;
using FluentAssertions;
using Xunit;
```

```
namespace AceriaApp.ConfiguracionYarp.Tests;

public class ValidadorConfiguracionYarpTests
{
    private readonly ValidadorConfiguracionYarp _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoClusters.ClustersPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinClusters_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ClusterYarp>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay clusters"));
    }

    [Fact]
    public void Validar_SinDestinos_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = Array.Empty<string>(), TieneDireccionValida = true,
TieneConfiguracionPorEntorno = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara destinos"));
    }

    [Fact]
    public void Validar_DireccionInvalida_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = new[] { "http://x" }, TieneDireccionValida = false,
TieneConfiguracionPorEntorno = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene dirección válida"));
    }

    [Fact]
    public void Validar_SinConfiguracionPorEntorno_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = new[] { "http://x" }, TieneDireccionValida = true,
TieneConfiguracionPorEntorno = false }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene configuración por entorno"));
    }
}
```

Declara una prueba individual que xUnit debe descubrir y ejecutar. Comprueba esta condición observable; una diferencia hace fallar la prueba en vez de ocultar el problema.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.ConfiguracionYarp;`: las clases no se resuelven.
- La sintaxis FluentAssertions exige su paquete y su using; las aserciones equivalentes de xUnit también son válidas.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

## PASO 8.2.15 — Crear el archivo `CatalogoClustersTests.cs`

Los tests verifican que el catálogo por defecto contiene los clusters esperados y que todos cumplen las reglas de configuración.

1. Crea `tests/AceriaApp.ConfiguracionYarp.Tests/CatalogoClustersTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/CatalogoClustersTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.ConfiguracionYarp;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ConfiguracionYarp.Tests;

public class CatalogoClustersTests
{
    [Fact]
    public void ClustersPorDefecto_ContieneTresClusters()
    {
        var clusters = CatalogoClusters.ClustersPorDefecto();

        clusters.Should().HaveCount(3);
    }

    [Fact]
    public void ClustersPorDefecto_ContieneOrdenesInventarioYClientes()
    {
        var clusters = CatalogoClusters.ClustersPorDefecto();

        clusters.Should().Contain(c => c.Identificador == "ordenes-cluster");
        clusters.Should().Contain(c => c.Identificador == "inventario-cluster");
        clusters.Should().Contain(c => c.Identificador == "clientes-cluster");
    }

    [Fact]
    public void ClustersPorDefecto_TodosTienenDestinos()
    {
        var clusters = CatalogoClusters.ClustersPorDefecto();

        clusters.Should().OnlyContain(c => c.Destinos.Count > 0);
    }

    [Fact]
    public void ClustersPorDefecto_TodosTienenDireccionValida()
    {
        var clusters = CatalogoClusters.ClustersPorDefecto();

        clusters.Should().OnlyContain(c => c.TieneDireccionValida);
    }
```

```
    [Fact]
    public void ClustersPorDefecto_TodosTienenConfiguracionPorEntorno()
    {
        var clusters = CatalogoClusters.ClustersPorDefecto();

        clusters.Should().OnlyContain(c => c.TieneConfiguracionPorEntorno);
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.ConfiguracionYarp;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Contain` y `OnlyContain` no se resuelven.
- No verificar la dirección válida: la regla no se comprueba.

## PASO 8.2.16 — Compilar la solución completa

Al conectar YARP, las comprobaciones funcionales deben asegurar que el gateway arranca y reenvía solicitudes a los procesos de Producción e Inventario. Una compilación correcta sin tráfico HTTP no demuestra enrutamiento.

1. Escribe todos los archivos de ampliación funcional mostrados a continuación antes de compilar; cuando se indica reemplazar, sustituye el contenido de la copia actual.
2. Desde la carpeta actual ejecuta `dotnet sln AceriaApp.sln add (Get-ChildItem src,tests -Recurse -Filter *.csproj | ForEach-Object FullName)` en PowerShell 7.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release -m:1`.
4. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 37 proyectos; el compilador debe terminar con cero errores.

**Archivo:** `src/AceriaApp.Gateway/AceriaApp.Gateway.csproj`. **Operación:** Crear el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Yarp.ReverseProxy" Version="2.2.0" />
    <ProjectReference Include="../AceriaApp.Observabilidad/AceriaApp.Observabilidad.csproj" />
  </ItemGroup>
</Project>
```

**Archivo:** `src/AceriaApp.Gateway/GatewayProgram.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Observabilidad;
using Yarp.ReverseProxy.Transforms;

namespace AceriaApp.Gateway;

public sealed class GatewayProgram
{
    public static Task Main(string[] args) => CreateApp(args).RunAsync();

    public static WebApplication CreateApp(string[] args, Action<WebApplicationBuilder>? configurar = null)
    {
        var builder = WebApplication.CreateBuilder(args);
        configurar?.Invoke(builder);
        builder.Services.AddLoggingAceria("Gateway", builder.Configuration);
        builder.Services.AddErroresAceria();
        builder.Services.AddReverseProxy()
            .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"))
            .AddTransforms(ruta =>
            {
                ruta.AddRequestTransform(contexto =>
```

```
                {
                    // La cabecera entrante puede ser inválida: se usa el valor ya validado en el borde.
                    contexto.ProxyRequest.Headers.Remove(ContextoCorrelacion.Cabecera);
                    contexto.ProxyRequest.Headers.TryAddWithoutValidation(ContextoCorrelacion.Cabecera,
                        ContextoCorrelacion.Obtener(contexto.HttpContext));
                    return ValueTask.CompletedTask;
                });
                ruta.AddResponseTransform(contexto =>
                {
                    var location = contexto.ProxyResponse?.Headers.Location;
                    var destino = contexto.ProxyResponse?.RequestMessage?.RequestUri;
                    if (location is { IsAbsoluteUri: true } && destino is not null &&
                        string.Equals(location.Scheme, destino.Scheme, StringComparison.OrdinalIgnoreCase) &&
                        string.Equals(location.Authority, destino.Authority,
StringComparison.OrdinalIgnoreCase))
                    {
                        // Los recursos publicados conservan su path: Location sigue siendo público.
                        contexto.HttpContext.Response.Headers.Location = location.PathAndQuery +
location.Fragment;
                    }
                    return ValueTask.CompletedTask;
                });
            });
        var app = builder.Build();
        app.UseLoggingAceria();
        app.UseErroresAceria();
        // Liveness acredita este proceso, no la disponibilidad de SQL, RabbitMQ o los destinos.
        app.MapGet("/health/live", () => Results.Ok(new { servicio = "Gateway", estado = "vivo" }));
        // La sobrecarga predeterminada incluye selección de destinos y middleware de balanceo.
        app.MapReverseProxy();
        return app;
    }
}
```

Completa esta expresión o invocación dentro de GatewayProgram.cs; se conserva íntegra para reproducir el comportamiento. Registra los servicios YARP que seleccionan destinos y reenvían peticiones HTTP. Carga rutas y clusters desde la configuración del proceso y permite su actualización.

Trabaja con la petición concreta sin convertirla en una operación de negocio nueva. Documenta la intención de la instrucción próxima sin alterar su ejecución. Utiliza el identificador ya validado en el borde para que gateway y backend compartan la misma correlación.

Configura o comprueba la respuesta que recibe el cliente, conservando la semántica HTTP. Lee o escribe la cabecera HTTP indicada; su valor debe coincidir con el contrato de transformación. Publica las rutas YARP con su canalización de selección, balanceo y reenvío.

**Archivo:** `src/AceriaApp.Gateway/appsettings.json`. **Operación:** Crear el contenido completo.

```
{
  "Urls": "http://localhost:5100",
  "Observabilidad": {
    "NivelMinimo": "Information"
  },
  "ReverseProxy": {
    "Routes": {
      "ordenes": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/ordenes/{**resto}"
        }
      },
      "produccion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/produccion/{**resto}"
        }
      },
      "comunicacion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/comunicacion/{**resto}"
```

```
        }
      },
      "materiales": {
        "ClusterId": "inventario-cluster",
        "Match": {
          "Path": "/api/v1/materiales/{**resto}",
          "Methods": [
            "GET",
            "HEAD"
          ]
        }
      },
      "planificaciones": {
        "ClusterId": "inventario-cluster",
        "Match": {
          "Path": "/api/v1/planificaciones/{**resto}",
          "Methods": [
            "GET",
            "HEAD"
          ]
        }
      }
    },
    "Clusters": {
      "produccion-cluster": {
        "Destinations": {
          "produccion": {
            "Address": "http://localhost:5105/"
          }
        }
      },
      "inventario-cluster": {
        "Destinations": {
          "inventario-1": {
            "Address": "http://localhost:5106/"
          }
        }
      }
    }
  }
}
```

Abre o cierra esta estructura JSON; las comas y delimitadores mantienen válido el documento. Establece la dirección de escucha de este proceso; conserva el nombre y el tipo json. Asigna este valor a la opción observabilidad; conserva el nombre y el tipo json.

Asigna este valor a la opción nivelminimo; conserva el nombre y el tipo json. Reúne la configuración que lee addreverseproxy; conserva el nombre y el tipo json. Declara las rutas públicas que yarp puede seleccionar; conserva el nombre y el tipo json.

Asigna este valor a la opción ordenes; conserva el nombre y el tipo json. Vincula la ruta con un cluster existente; conserva el nombre y el tipo json. Asigna este valor a la opción match; conserva el nombre y el tipo json.

Define el patrón público o la ruta del sondeo según su sección; conserva el nombre y el tipo json. Asigna este valor a la opción produccion; conserva el nombre y el tipo json. Asigna este valor a la opción comunicacion; conserva el nombre y el tipo json.

Asigna este valor a la opción materiales; conserva el nombre y el tipo json. Restringe los métodos que esta ruta admite; conserva el nombre y el tipo json. Asigna este valor a la opción planificaciones; conserva el nombre y el tipo json.

Agrupa destinos que ofrecen el mismo contrato http; conserva el nombre y el tipo json. Asigna este valor a la opción produccion-cluster; conserva el nombre y el tipo json. Enumera instancias del servicio, no servicios con contratos diferentes; conserva el nombre y el tipo json.

Establece la url interna del destino; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-cluster; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-1; conserva el nombre y el tipo json.

**Archivo:** `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddLoggingAceria("Inventario", builder.Configuration);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        builder.Services.AddSaludAceria(builder.Configuration, inventario: true);
        builder.Services.AddErroresAceria();
        var app = builder.Build();
        app.UseLoggingAceria();
        app.UseErroresAceria();
        app.MapDiagnosticoAceria();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapMethods("/api/v1/materiales/{material}", new[] { "GET", "HEAD" }, async (string material,
InventarioDbContext db, CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapMethods("/api/v1/planificaciones/{ordenId:guid}", new[] { "GET", "HEAD" }, async (Guid
ordenId, InventarioDbContext db, CancellationToken ct) =>
        {
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
valor.CantidadTotal });
        });
        app.MapSaludAceria();
        await app.RunAsync();
    }
}
```

Completa esta expresión o invocación dentro de InventarioProgram.cs; se conserva íntegra para reproducir el comportamiento. Delimita la vida del recurso y garantiza su disposición al salir del ámbito, también si ocurre una excepción; await using permite la disposición asíncrona. Espera la operación asíncrona sin bloquear el hilo; un fallo se propaga para poder detectarlo.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/GatewayConfiguracionFuncionalTests.cs`. **Operación:** Crear el contenido completo.

```
using System.Net;
using System.Net.Http.Json;
using System.Text;
using AceriaApp.Pruebas.Gateway;
using Xunit;

namespace AceriaApp.ConfiguracionYarp.Tests;

public sealed class GatewayConfiguracionFuncionalTests
{
    // Detecta destinos cruzados y pérdida del método/cuerpo/query al atravesar YARP real.
    [Theory]
    [InlineData("GET", "/api/v1/ordenes/42", "produccion")]
    [InlineData("POST", "/api/v1/ordenes", "produccion")]
    [InlineData("PUT", "/api/v1/produccion/ordenes/42", "produccion")]
    [InlineData("PATCH", "/api/v1/comunicacion/ordenes/42", "produccion")]
    [InlineData("DELETE", "/api/v1/ordenes/42", "produccion")]
    [InlineData("GET", "/api/v1/materiales/Acero", "inventario-1")]
    [InlineData("GET", "/api/v1/planificaciones/42", "inventario-1")]
    public async Task Publica_solo_el_servicio_y_conserva_la_peticion(string metodo, string ruta, string
instancia)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var peticion = new HttpRequestMessage(new HttpMethod(metodo), ruta + "?
material=Acero%20inoxidable&pagina=2")
        { Content = new StringContent("{\"cantidad\":17}", Encoding.UTF8, "application/json") };
        using var respuesta = await prueba.Cliente.SendAsync(peticion);
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        var eco = await respuesta.Content.ReadFromJsonAsync<EcoPeticion>();
        Assert.Equal(instancia, eco!.Instancia);
        Assert.Equal(metodo, eco.Metodo);
        Assert.Equal(ruta, eco.Ruta);
        Assert.Equal("?material=Acero%20inoxidable&pagina=2", eco.Query);
        Assert.Equal("{\"cantidad\":17}", eco.Cuerpo);
    }

    [Theory]
    [InlineData(400)] [InlineData(404)] [InlineData(409)] [InlineData(422)]
    public async Task Conserva_el_estado_y_el_ProblemDetails_del_backend(int estado)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.GetAsync($"/api/v1/ordenes/42?estado={estado}");
        Assert.Equal(estado, (int)respuesta.StatusCode);
        Assert.Equal("application/problem+json", respuesta.Content.Headers.ContentType!.MediaType);
        Assert.Equal($"{{\"type\":\"https://aceria.test/problemas/regla\",\"title\":\"Regla del servicio\",
\"status\":{estado},\"detail\":\"Respuesta original\",\"codigo\":\"ACERIA-001\"}}", await
respuesta.Content.ReadAsStringAsync());
    }

    [Fact]
    public async Task El_Location_absoluto_interno_del_201_se_puede_seguir_por_el_gateway()
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.PostAsync("/api/v1/ordenes?estado=201", new
StringContent("{}"));
        Assert.Equal(HttpStatusCode.Created, respuesta.StatusCode);
        Assert.Equal("/api/v1/ordenes/creada?version=1", respuesta.Headers.Location!.OriginalString);
        using var creada = await prueba.Cliente.GetAsync(respuesta.Headers.Location);
        Assert.Equal(HttpStatusCode.OK, creada.StatusCode);
        Assert.Equal("produccion", (await creada.Content.ReadFromJsonAsync<EcoPeticion>())!.Instancia);
    }

    [Theory]
    [InlineData("curso_42-A", true)] [InlineData("invalida con espacios", false)] [InlineData("", false)]
    public async Task La_correlacion_validada_en_el_borde_es_la_que_recibe_el_backend(string entrada, bool
```

```
valida)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var peticion = new HttpRequestMessage(HttpMethod.Get, "/api/v1/ordenes");
        if (entrada.Length > 0) peticion.Headers.TryAddWithoutValidation("X-Correlation-ID", entrada);
        using var respuesta = await prueba.Cliente.SendAsync(peticion);
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        var correlacion = Assert.Single(respuesta.Headers.GetValues("X-Correlation-ID"));
        Assert.Equal(correlacion, (await respuesta.Content.ReadFromJsonAsync<EcoPeticion>())!.Correlacion);
        if (valida) Assert.Equal(entrada, correlacion);
        else { Assert.NotEqual(entrada, correlacion); Assert.Matches("^[a-f0-9]{32}$", correlacion); }
    }

    [Theory]
    [InlineData("/diagnostico")] [InlineData("/diagnostico/error")]
    [InlineData("/health/ready")] [InlineData("/swagger/index.html")]
    [InlineData("/desconocida")] [InlineData("/api/v1/clientes")]
    public async Task Una_ruta_no_publicada_no_llega_a_ningun_backend(string ruta)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.GetAsync(ruta);
        Assert.Equal(HttpStatusCode.NotFound, respuesta.StatusCode);
        Assert.Equal(0, prueba.Produccion.Peticiones + prueba.Inventario1.Peticiones);
    }

    [Theory]
    [InlineData("POST")] [InlineData("PUT")] [InlineData("PATCH")] [InlineData("DELETE")]
    public async Task Inventario_no_publica_escrituras(string metodo)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.SendAsync(new HttpRequestMessage(new HttpMethod(metodo),
"/api/v1/materiales/Acero"));
        Assert.Equal(HttpStatusCode.MethodNotAllowed, respuesta.StatusCode);
        Assert.Equal(0, prueba.Inventario1.Peticiones);
    }

    [Fact]
    public async Task Live_responde_del_gateway_aunque_los_backends_esten_detenidos()
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        await prueba.Produccion.DetenerAsync();
        await prueba.Inventario1.DetenerAsync();
        using var respuesta = await prueba.Cliente.GetAsync("/health/live");
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Contains("Gateway", await respuesta.Content.ReadAsStringAsync());
    }
}
```

Declara una prueba parametrizada; cada conjunto de datos produce una ejecución. Proporciona un caso de entrada concreto para la prueba parametrizada.

**Archivo:** `tests/Compartido/GatewayPruebas.cs`. **Operación:** Crear el contenido completo.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Gateway;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Hosting.Server;
using Microsoft.AspNetCore.Hosting.Server.Features;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaApp.Pruebas.Gateway;

// Servidores HTTP reales de contrato. No sustituyen las pruebas SQL/RabbitMQ de integración.
public sealed class GatewayPruebas : IAsyncDisposable
{
    public BackendPruebas Produccion { get; private set; } = null!;
    public BackendPruebas Inventario1 { get; private set; } = null!;
    public BackendPruebas? Inventario2 { get; private set; }
```

```
    public WebApplication Proxy { get; private set; } = null!;
    public HttpClient Cliente { get; private set; } = null!;

    public static async Task<GatewayPruebas> CrearAsync(bool segundaInstancia = false,
Action<WebApplicationBuilder>? configurar = null)
    {
        var prueba = new GatewayPruebas();
        try
        {
            prueba.Produccion = await BackendPruebas.CrearAsync("produccion");
            prueba.Inventario1 = await BackendPruebas.CrearAsync("inventario-1");
            if (segundaInstancia) prueba.Inventario2 = await BackendPruebas.CrearAsync("inventario-2");
            prueba.Proxy = GatewayProgram.CreateApp([], builder =>
            {
                builder.Configuration.Sources.Clear();
                builder.Configuration.AddJsonFile(Path.Combine(AppContext.BaseDirectory,
"gateway.appsettings.json"), false, false);
                builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
                {
                    ["Urls"] = "http://127.0.0.1:0",
                    ["Observabilidad:NivelMinimo"] = "Fatal",
                    ["ReverseProxy:Clusters:produccion-cluster:Destinations:produccion:Address"] =
prueba.Produccion.Direccion,
                    ["ReverseProxy:Clusters:inventario-cluster:Destinations:inventario-1:Address"] =
prueba.Inventario1.Direccion
                });
                if (prueba.Inventario2 is not null ||
builder.Configuration["ReverseProxy:Clusters:inventario-cluster:Destinations:inventario-2:Address"] is not
null)
                    builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
                    {
                        ["ReverseProxy:Clusters:inventario-cluster:Destinations:inventario-2:Address"] =
prueba.Inventario2?.Direccion ?? prueba.Inventario1.Direccion
                    });
                builder.Logging.ClearProviders();
                configurar?.Invoke(builder);
            });
            await prueba.Proxy.StartAsync();
            prueba.Cliente = new HttpClient(new HttpClientHandler { AllowAutoRedirect = false })
            {
                BaseAddress = new Uri(DireccionDe(prueba.Proxy)), Timeout = TimeSpan.FromSeconds(10)
            };
            return prueba;
        }
        catch { await prueba.DisposeAsync(); throw; }
    }

    public async Task ReiniciarInventario2Async()
    {
        var direccion = Inventario2!.Direccion;
        await Inventario2.DisposeAsync();
        Inventario2 = await BackendPruebas.CrearAsync("inventario-2", direccion);
    }

    public static string DireccionDe(WebApplication app) => app.Services.GetRequiredService<IServer>()
        .Features.Get<IServerAddressesFeature>()!.Addresses.Single();

    public static async Task EsperarAsync(Func<Task<bool>> condicion, string motivo)
    {
        using var plazo = new CancellationTokenSource(TimeSpan.FromSeconds(20));
        while (!plazo.IsCancellationRequested)
        {
            if (await condicion()) return;
            try { await Task.Delay(100, plazo.Token); }
            catch (OperationCanceledException) { break; }
        }
        throw new TimeoutException(motivo);
    }

    public async ValueTask DisposeAsync()
    {
        Cliente?.Dispose();
        if (Proxy is not null) { await Proxy.StopAsync(); await Proxy.DisposeAsync(); }
        if (Produccion is not null) await Produccion.DisposeAsync();
        if (Inventario1 is not null) await Inventario1.DisposeAsync();
        if (Inventario2 is not null) await Inventario2.DisposeAsync();
```

```
    }
}

public sealed record EcoPeticion(string Instancia, string Metodo, string Ruta, string Query,
    string Cuerpo, string Correlacion, string Canal, string RutaSeleccionada);

public sealed class BackendPruebas : IAsyncDisposable
{
    private readonly WebApplication app;
    private int peticiones;
    private int escrituras;
    private int sondeos;
    private bool detenido;
    public string Direccion { get; }
    public int Peticiones => Volatile.Read(ref peticiones);
    public int Escrituras => Volatile.Read(ref escrituras);
    public int Sondeos => Volatile.Read(ref sondeos);
    public bool Listo { get; set; } = true;

    private BackendPruebas(WebApplication app)
    {
        this.app = app;
        Direccion = GatewayPruebas.DireccionDe(app);
    }

    public static async Task<BackendPruebas> CrearAsync(string instancia, string? direccion = null)
    {
        var builder = WebApplication.CreateBuilder();
        builder.WebHost.UseUrls(direccion ?? "http://127.0.0.1:0");
        builder.Logging.ClearProviders();
        var app = builder.Build();
        BackendPruebas? backend = null;
        app.MapGet("/health/ready", () =>
        {
            Interlocked.Increment(ref backend!.sondeos);
            return Results.StatusCode(backend.Listo ? 200 : 503);
        });
        app.Map("/{**ruta}", async (HttpContext contexto) =>
        {
            Interlocked.Increment(ref backend!.peticiones);
            if (contexto.Request.Method is not ("GET" or "HEAD")) Interlocked.Increment(ref
backend.escrituras);
            contexto.Response.Headers["X-Instancia"] = instancia;
            if (contexto.Request.Query.ContainsKey("abortar")) { contexto.Abort(); return; }
            var estado = int.TryParse(contexto.Request.Query["estado"], out var numero) ? numero : 200;
            contexto.Response.StatusCode = estado;
            if (estado is 400 or 404 or 409 or 422)
            {
                contexto.Response.ContentType = "application/problem+json";
                await contexto.Response.WriteAsync($"{{\"type\":\"https://aceria.test/problemas/regla\",
\"title\":\"Regla del servicio\",\"status\":{estado},\"detail\":\"Respuesta original\",\"codigo\":
\"ACERIA-001\"}}");
                return;
            }
            if (estado == 201) contexto.Response.Headers.Location =
                $"{contexto.Request.Scheme}://{contexto.Request.Host}/api/v1/ordenes/creada?version=1";
            using var lector = new StreamReader(contexto.Request.Body);
            var cuerpo = await lector.ReadToEndAsync();
            await contexto.Response.WriteAsJsonAsync(new EcoPeticion(instancia, contexto.Request.Method,
                contexto.Request.Path, contexto.Request.QueryString.Value ?? "", cuerpo,
                contexto.Request.Headers["X-Correlation-ID"].ToString(),
                contexto.Request.Headers["X-Canal"].ToString(),
                contexto.Request.Headers["X-Ruta"].ToString()));
        });
        await app.StartAsync();
        backend = new BackendPruebas(app);
        return backend;
    }

    public async Task DetenerAsync()
    {
        if (detenido) return;
        detenido = true;
        await app.StopAsync();
    }
```

```
    public async ValueTask DisposeAsync() { await DetenerAsync(); await app.DisposeAsync(); }
}
```

Declara una propiedad con lectura; un accesor set, cuando aparece, permite modificar su valor según su visibilidad. Delimita una operación que necesita gestión explícita de errores o liberación de recursos. Completa esta expresión o invocación dentro de GatewayPruebas.cs; se conserva íntegra para reproducir el comportamiento.

Trata exclusivamente el error capturado en esta rama; los demás errores deben seguir siendo visibles. Interrumpe la operación con un error explícito; no transforma un fallo en un resultado satisfactorio.

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj`.
**Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Gateway/AceriaApp.Gateway.csproj" />
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <Compile Include="../Compartido/GatewayPruebas.cs" Link="Compartido/GatewayPruebas.cs" />
    <Content Include="../../src/AceriaApp.Gateway/appsettings.json" Link="gateway.appsettings.json"
CopyToOutputDirectory="PreserveNewest" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Hace disponibles las API del framework ASP.NET Core para los servidores de prueba. Incluye el soporte de pruebas compartido en la compilación de este proyecto.

**Verificación concreta.** Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 37 proyectos; el compilador debe terminar con cero errores.

**Diagnóstico si falla.**

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

## PASO 8.2.17 — Ejecutar todos los tests del punto 8.2

Una prueba de catálogo acredita su declaración. Las pruebas funcionales utilizan el proxy y servidores HTTP reales, pero la persistencia y el broker se comprueban con la suite y el recorrido completo.

1. Ejecuta `dotnet test tests/AceriaApp.ConfiguracionYarp.Tests/AceriaApp.ConfiguracionYarp.Tests.csproj -c Release --no-build` en la terminal.
2. Comprueba que se ejecutan 36 pruebas: 10 de catálogo y 26 funcionales.
3. Si hay un fallo, identifica su nombre y revisa la regla o la respuesta esperada antes de modificar código.
4. Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Verificación concreta.** Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Diagnóstico si falla.**

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

## PASO 8.2.18 — Ejecutar todos los tests de la solución

La suite completa comprueba que el nuevo punto no rompe la persistencia, la mensajería, el dominio, la observabilidad ni la resiliencia desarrollados antes.

1. Configura `ACERIA_TEST_SQLSERVER` para tu SQL Server Express/LocalDB y `ACERIA_TEST_RABBITMQ` para el broker; sigue los requisitos del inicio del manual.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build -m:1`.
3. Comprueba que pasan las 451 pruebas acumuladas, incluidas las heredadas de M07, sin omisiones.
4. Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Verificación concreta.** Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Diagnóstico si falla.**

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

## PASO 8.2.19 — Crear el README del punto 8.2

El README del punto 8.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 8.1.

1. Abre `README.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 8.2 — Configuración básica de YARP

## Objetivo

Configurar y ejecutar un gateway YARP independiente hacia Producción e Inventario.

## Estado del proyecto

Etapa acumulativa con 37 proyectos, 24 proyectos de tests y 46 documentos. El cierre exige 451 pruebas
aprobadas sin omisiones; los resultados se registran en los informes del módulo.

## Cómo ejecutarlo

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test
AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite
completa.
```

```
El recorrido funcional inicia Gateway en 5100, Producción en 5105 e Inventario en 5106. La etapa 8.4 añade
Inventario en 5107.

## Qué se ha añadido respecto al punto anterior

- Modelo ClusterYarp, validador ValidadorConfiguracionYarp y catálogo CatalogoClusters.
- 10 pruebas de catálogo y 26 pruebas funcionales nuevas.
- Documento temático y ADR-032.
- Reto optativo en copia separada, con una prueba adicional.

## Documentación

- `docs/configuracion-basica-yarp.md`.
- `docs/ADR-032-configuracion-basica-yarp.md`.
- Las 44 fichas y decisiones anteriores permanecen idénticas.
```

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite completa. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `docs/configuracion-basica-yarp.md`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

- `docs/ADR-032-configuracion-basica-yarp.md`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar el README del punto 8.1: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 8.1: rompe la separación.

## PASO 8.2.20 — Crear el ADR-032 del punto 8.2

El ADR-032 documenta la decisión de adoptar YARP. Es un artefacto nuevo del punto 8.2 que no modifica los ADR existentes.

1. Crea `docs/ADR-032-configuracion-basica-yarp.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/ADR-032-configuracion-basica-yarp.md`. **Operación:** Crear el contenido completo.

```
# ADR-032: Configuración básica de YARP

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado el patrón API Gateway. Es necesario definir la implementación concreta del
gateway.

## Decisión
Se adopta Yarp.ReverseProxy 2.2.0 en AceriaApp.Gateway, con destino net8.0 y proceso independiente.
`appsettings.json` contiene `ReverseProxy`, con `Routes` y `Clusters`. Se publican las órdenes y comunicación
de Producción y los materiales y planificaciones de Inventario; Clientes no es un backend implementado. En
8.2 el gateway escucha en 5100 y apunta a Producción 5105 e Inventario 5106; en 8.4 Inventario añade 5107 y
```

```
Producción mantiene un destino. Las direcciones se pueden sustituir por configuración de entorno. La
correlación se valida mediante el middleware de M07 y se propaga mediante código; Location conserva una
referencia pública. Los alias y la eliminación de /inventario se incorporan en 8.3. Se comprueban tanto el
catálogo como el proxy real con Kestrel; SQL y RabbitMQ se acreditan mediante integración.

## Alternativas consideradas
- Ocelot: no se adopta para mantener la biblioteca YARP elegida en este laboratorio ASP.NET Core.
- Nginx: descartado por estar fuera del stack .NET.
- Gateway personalizado: descartado por complejidad.

## Consecuencias
- Positivas: integración con ASP.NET Core, configuración declarativa, coherencia con el stack.
- Negativas: la configuración requiere mantenimiento por entorno.
- Mitigación: la validación automática y las buenas prácticas documentadas.

## Alcance comprobable de esta etapa

El proyecto ejecutable AceriaApp.Gateway se construye en el paso 8.2.16. El catálogo es una descripción
separada, no el archivo que YARP carga. Los nombres DNS ASCII para contenedores requieren una red que los
resuelva; el recorrido local utiliza localhost y puertos explícitos. Aceptado expresa esta decisión
arquitectónica, no un PASS de CI.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

`Se adopta Yarp.ReverseProxy 2.2.0 en AceriaApp.Gateway, con destino net8.0 y proceso independiente. `appsettings.json` contiene `ReverseProxy`, con `Routes` y `Clusters`. Se publican las órdenes y comunicación de Producción y los materiales y planificaciones de Inventario; Clientes no es un backend implementado. En 8.2 el gateway escucha en 5100 y apunta a Producción 5105 e Inventario 5106; en 8.4 Inventario añade 5107 y Producción mantiene un destino. Las direcciones se pueden sustituir por configuración de entorno. La correlación se valida mediante el middleware de M07 y se propaga mediante código; Location conserva una referencia pública. Los alias y la eliminación de /inventario se incorporan en 8.3. Se comprueban tanto el catálogo como el proxy real con Kestrel; SQL y RabbitMQ se acreditan mediante integración`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-032`: rompe la convención.

## PASO 8.2.21 — Verificar que no se ha modificado ningún archivo del punto 8.1

La progresión modifica copias independientes. Los nuevos archivos o configuraciones de una etapa no deben alterar un punto ya entregado.

1. Abre las carpetas `docs` de la etapa actual y de `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway`.
2. Compara con Get-FileHash los archivos que existían en la etapa anterior; cada hash SHA256 debe coincidir.
3. Comprueba también que los archivos de tests heredados conservan su contenido; solo se incorporan los tests del punto.
4. Verifica que `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.2-configuracion-basica-yarp`.

**Verificación concreta.** Verifica que `M08_API_GATEWAY/PROYECTO/8.1-introduccion-api-gateway` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.2-configuracion-basica-yarp`.

**Diagnóstico si falla.**

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

## PASO 8.2.22 — Verificar la estructura del punto 8.2

El inventario convierte la comprobación de estructura en una condición observable y detecta omisiones que una compilación aislada podría pasar por alto.

1. Ejecuta `Get-ChildItem -Recurse -File` y revisa las carpetas src, tests y docs.
2. Verifica 37 proyectos, 24 proyectos de pruebas y 46 documentos.
3. Comprueba la presencia de las tres clases de `ConfiguracionYarp`, sus tests, el documento temático y ADR-032.
4. Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Verificación concreta.** Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Diagnóstico si falla.**

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 8.1: viola la regla de no modificar.
- Omitir la carpeta `ConfiguracionYarp`: los tests no encuentran las clases.

## PASO 8.2.23 — Verificar el `.gitignore` del punto 8.2

Las exclusiones conservan un repositorio reproducible sin binarios temporales ni datos específicos de Visual Studio.

1. Abre `.gitignore` de la etapa actual.
2. Comprueba que contiene bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
3. Compara su contenido con el archivo heredado y conserva sus exclusiones.
4. Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Verificación concreta.** Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Diagnóstico si falla.**

- Modificar el `.gitignore` copiado del punto 8.1: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

## PASO 8.2.24 — Hacer el commit del punto 8.2 y verificar el estado final

Un commit conserva el estado comprobado. El número de commits no tiene que coincidir con el de puntos: las correcciones y revisiones también forman parte del historial.

1. Desde la raíz del repositorio ejecuta `git status --short` y revisa todos los cambios de la etapa.
2. Añade exclusivamente los archivos docentes y del proyecto con `git add M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp`.

3. Ejecuta `git commit -m "Punto 8.2: Configuración básica de YARP"` después de comprobar las pruebas.
4. Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Verificación concreta.** Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Diagnóstico si falla.**

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

## Reto de ampliación (optativo)

añadir un método `Resumen` a `ValidadorConfiguracionYarp` que devuelva el número total de clusters, el número total de destinos y el número de clusters que tienen configuración por entorno, y un test que verifique el cálculo con el catálogo por defecto.

Crea una copia de `8.2-configuracion-basica-yarp` llamada `reto-8-2` fuera de las cuatro carpetas obligatorias. La copia anterior permanece intacta. Aplica las tres sustituciones completas siguientes en orden.

### Ampliación 1: añadir el contrato de resumen

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ConfiguracionYarp;

public class ValidadorConfiguracionYarp
{
    public ResultadoConfiguracionYarp Validar(IEnumerable<ClusterYarp> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (cluster.Destinos.Count == 0)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no declara destinos.");
            }

            if (!cluster.TieneDireccionValida)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene dirección válida.");
            }

            if (!cluster.TieneConfiguracionPorEntorno)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene configuración por entorno.");
            }
```

```
        }

        return new ResultadoConfiguracionYarp(errores.Count == 0, errores);
    }
}

public record ResultadoConfiguracionYarp(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenConfiguracionYarp(int Total, int TotalDestinos, int ConConfiguracionPorEntorno);
```

## Ampliación 2: implementar el cálculo del resumen

**Archivo:** `src/AceriaApp.Api/ConfiguracionYarp/ValidadorConfiguracionYarp.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.ConfiguracionYarp;

public class ValidadorConfiguracionYarp
{
    public ResultadoConfiguracionYarp Validar(IEnumerable<ClusterYarp> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (cluster.Destinos.Count == 0)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no declara destinos.");
            }

            if (!cluster.TieneDireccionValida)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene dirección válida.");
            }

            if (!cluster.TieneConfiguracionPorEntorno)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene configuración por entorno.");
            }
        }

        return new ResultadoConfiguracionYarp(errores.Count == 0, errores);
    }

    public static ResumenConfiguracionYarp Resumen(IEnumerable<ClusterYarp> clusters)
    {
        var lista = clusters.ToList();
        var totalDestinos = lista.Sum(c => c.Destinos.Count);
        var conConfiguracion = lista.Count(c => c.TieneConfiguracionPorEntorno);
        return new ResumenConfiguracionYarp(lista.Count, totalDestinos, conConfiguracion);
    }
}

public record ResultadoConfiguracionYarp(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenConfiguracionYarp(int Total, int TotalDestinos, int ConConfiguracionPorEntorno);
```

## Ampliación 3: verificar el resultado con el catálogo

**Archivo:** `tests/AceriaApp.ConfiguracionYarp.Tests/ValidadorConfiguracionYarpTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.ConfiguracionYarp;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ConfiguracionYarp.Tests;

public class ValidadorConfiguracionYarpTests
{
    private readonly ValidadorConfiguracionYarp _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoClusters.ClustersPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinClusters_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ClusterYarp>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay clusters"));
    }

    [Fact]
    public void Validar_SinDestinos_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = Array.Empty<string>(), TieneDireccionValida = true,
TieneConfiguracionPorEntorno = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara destinos"));
    }

    [Fact]
    public void Validar_DireccionInvalida_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = new[] { "http://x" }, TieneDireccionValida = false,
TieneConfiguracionPorEntorno = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene dirección válida"));
    }

    [Fact]
    public void Validar_SinConfiguracionPorEntorno_DevuelveError()
    {
        var clusters = new List<ClusterYarp>
        {
            new() { Identificador = "X", Destinos = new[] { "http://x" }, TieneDireccionValida = true,
TieneConfiguracionPorEntorno = false }
        };

        var resultado = _validador.Validar(clusters);
```

```
        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene configuración por entorno"));
    }

    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorConfiguracionYarp.Resumen(CatalogoClusters.ClustersPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.TotalDestinos.Should().Be(3);
        resumen.ConConfiguracionPorEntorno.Should().Be(3);
    }
}
```

**Ejecución y resultado esperado.** En la copia optativa ejecuta `dotnet test AceriaApp.sln -c Release -m:1`. Deben pasar 452 pruebas; el proyecto `AceriaApp.ConfiguracionYarp.Tests` debe ejecutar 37. Verifica que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que los cuatro puntos obligatorios no cambiaron. Registra el reto en un commit independiente cuando lo hayas comprobado.

### Resultado esperado del punto

- Solución autónoma con 37 proyectos y 24 proyectos de pruebas.
- 46 documentos, incluidos el tema del punto y ADR-032.
- 10 pruebas nuevas de catálogo y 26 funcionales; 451 acumuladas sin omisiones.
- Etapa anterior, documentación y tests heredados conservados; el reto sigue separado.

### Conclusión y continuidad

El punto 8.2 ha definido la configuración básica de YARP como implementación concreta del patrón API Gateway. El proyecto AceriaApp dispone ahora de un validador automático de clusters que comprueba el identificador, los destinos, la dirección válida y la configuración por entorno, y diez tests nuevos. Esta base permite avanzar hacia el punto 8.3, donde se estudiará el enrutamiento de peticiones con detalle: coincidencia de rutas, transformaciones y ordenación de rutas.

---

**Fin del punto 8.2.**

---

# Punto 8.3 — Enrutamiento de peticiones

## Entender cómo se transforman las rutas y se conserva la correlación

Cuando el cliente utiliza un prefijo público, el backend puede esperar un camino diferente. Las reglas de YARP deben transformar la ruta sin perder el método, los datos necesarios ni los identificadores que permiten seguir la petición en los logs. Un error en el prefijo puede producir un 404 aunque el backend esté disponible; una cabecera de correlación no validada también puede romper el recorrido de observabilidad.

La práctica trabajará sobre reglas de enrutamiento y pruebas de proxy HTTP real. Los catálogos no realizan transformaciones por sí solos: la configuración efectiva y los middleware determinan lo que recibe Producción o Inventario. Las comprobaciones relacionarán la solicitud pública con la ruta del servicio y con los registros de observabilidad conservados desde M07.

**Carpeta de trabajo:** `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones`.

**Objetivo.** Completar los 24 pasos, conservar la etapa anterior y comprobar el catálogo y el comportamiento que incorpora este punto.

## PASO 8.3.1 — Crear la carpeta del punto 8.3 a partir del punto 8.2

La copia conserva 37 proyectos y 451 pruebas antes de incorporar los cambios del punto.

1. En el Explorador de archivos localiza `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp` y comprueba que contiene `AceriaApp.sln`, `src`, `tests`, `docs` y `global.json`.
2. Crea la carpeta `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones` y copia allí el contenido de la etapa anterior, incluidas sus carpetas ocultas de configuración.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; abre únicamente la copia `8.3-enrutamiento-peticiones` para realizar este punto.
4. Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Verificación concreta.** Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Diagnóstico si falla.**

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

## PASO 8.3.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

La compilación comprueba que la etapa anterior está completa y que el SDK .NET 8 resuelve sus dependencias.

1. Abre en Visual Studio el archivo `AceriaApp.sln` de `8.3-enrutamiento-peticiones`.
2. Abre Terminal en esa carpeta y ejecuta `dotnet sln AceriaApp.sln list`; deben aparecer 37 proyectos.
3. Ejecuta `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Verificación concreta.** Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Diagnóstico si falla.**

- Abrir la solución del punto 8.2 en lugar de la del punto 8.3: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 8.2 en ejecución: el puerto queda ocupado y la compilación falla.

## PASO 8.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 8.2

Esta etapa añade dos documentos sin sobrescribir las 46 fichas y decisiones anteriores.

1. Expande `docs` en Solution Explorer y ejecuta `(Get-ChildItem docs -Filter *.md).Count`.
2. Comprueba que hay 46 documentos heredados y que incluyen los ADR-010 a ADR-032.
3. Compara los nombres y contenidos con `docs` de `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp`.

4. Confirma que todavía no existen `docs/enrutamiento-peticiones.md` ni `docs/ADR-033-enrutamiento-peticiones.md`.

**Verificación concreta.** Confirma que todavía no existen `docs/enrutamiento-peticiones.md` ni `docs/ADR-033-enrutamiento-peticiones.md`.

**Diagnóstico si falla.**

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 8.2.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

## PASO 8.3.4 — Crear el documento de enrutamiento de peticiones en `docs`

El documento de enrutamiento de peticiones es el artefacto de `docs` específico del punto 8.3. No modifica los 46 documentos existentes del punto 8.2.

1. Crea `docs/enrutamiento-peticiones.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/enrutamiento-peticiones.md`. **Operación:** Crear el contenido completo.

```
# Enrutamiento de peticiones — Punto 8.3

## Proceso de coincidencia

YARP utiliza el enrutamiento de endpoints de ASP.NET Core para seleccionar una ruta que satisfaga sus
condiciones. Primero interviene el valor de `Order`: los valores menores tienen prioridad, incluidos los
negativos. Entre rutas con el mismo `Order`, interviene la especificidad del patrón. La posición de una
entrada en el objeto JSON no establece su prioridad; deben evitarse coincidencias ambiguas.

## Patrones de ruta

Sintaxis de ASP.NET Core. Segmentos literales, parámetros, comodines y restricciones.

    {
      "Match": {
        "Path": "/api/v1/ordenes/{id:guid}",
        "Methods": [ "GET" ]
      }
    }


## Precedencia

Un `Order` menor prevalece sobre uno mayor antes de comparar la especificidad. Con el mismo `Order`, un
patrón específico tiene mayor precedencia que otro más general. En el alias de Inventario se utiliza `Order:
-10`; no hay una restricción general de YARP que obligue a usar órdenes no negativos.

## Transformaciones de ruta

- `PathRemovePrefix`: elimina un prefijo.
- `PathSet`: establece una ruta fija.
- `PathPattern`: aplica un patrón.

## Transformaciones de cabecera

- `RequestHeader` con `Set` o `Append`: fija o añade un valor de cabecera de petición. Para eliminarla se
utiliza `RequestHeaderRemove`.
- `ResponseHeader` con `Set` o `Append`: fija o añade un valor de cabecera de respuesta. Para eliminarla se
utiliza `ResponseHeaderRemove`.
- `Set` en estas transformaciones asigna un literal. Por ejemplo, `Set: generated` enviaría el texto
```

```
generated; no genera un identificador. La correlación dinámica se obtiene del middleware de M07 y se propaga
mediante código.

## Transformaciones de consulta

- `QueryRouteParameter`: establece el nombre del parámetro de consulta; su `Set` o `Append` indica el nombre
del valor de ruta del que se toma el dato.
- `QueryValueParameter`: establece el nombre del parámetro de consulta; su `Set` o `Append` contiene un valor
literal. En el alias se utiliza `QueryValueParameter: canal` con `Set: gateway`.
- Copiar dinámicamente un valor arbitrario de cabecera a la consulta requeriría una transformación
personalizada; no se configura mediante una transformación integrada con un nombre inventado.

## Orden de aplicación

Las transformaciones de petición se aplican en el orden declarado dentro de `Transforms`; no existe una
reordenación automática por categorías de ruta, cabecera y consulta. El alias docente declara:

1. `PathRemovePrefix: /inventario`.
2. `RequestHeader: X-Canal` con `Set: Gateway`.
3. `QueryValueParameter: canal` con `Set: gateway`.

Así, `/inventario/api/v1/materiales/Acero` se reenvía como `/api/v1/materiales/Acero`, conservando `/api/v1`.
El proxy añade X-Canal: Gateway y fija canal=gateway; no elimina el prefijo de versión del backend.

## Enrutamiento por cabecera

La ruta se selecciona según el valor de una cabecera declarada en `Match.Headers`.

## Buenas prácticas

1. Documentar primero `Order` y después la especificidad; no confiar en el orden de escritura del JSON.
2. Restricciones en los parámetros.
3. Transformaciones solo cuando son necesarias.
4. Documentar el orden de las rutas.
5. Validar con un test.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Orden de rutas | Order menor antes; especificidad cuando Order coincide |
| Transformaciones ruta | Eliminar solo /inventario en los dos alias publicados |
| Transformaciones cabecera | X-Canal: Gateway; correlación validada por middleware y propagada por código |
| Transformaciones consulta | canal=gateway mediante QueryValueParameter y Set literal |
| Métodos | GET y HEAD en materiales, planificaciones y sus alias; las escrituras a esas rutas se rechazan |
| Validación | Tests del catálogo y peticiones Kestrel sobre coincidencia, métodos y transformaciones |

## Alcance comprobable de esta etapa

La restricción Orden >= 0 del validador de catálogo y sus transformaciones obligatorias son reglas del
ejercicio de análisis, no del contrato general de YARP. El gateway utiliza rutas con Order negativo y otras
sin transformaciones. Las rutas públicas no incluyen un comodín global: los endpoints internos de diagnóstico
no se exponen por el alias. Las reglas de negocio permanecen en Producción e Inventario.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

Define este encabezado en el documento que el alumno conservará junto al proyecto. YARP utiliza el enrutamiento de endpoints de ASP.NET Core para seleccionar una ruta que satisfaga sus condiciones. Primero interviene el valor de `Order`: los valores menores tienen prioridad, incluidos los negativos. Entre rutas con el mismo `Order`, interviene la especificidad del patrón. La posición de una entrada en el objeto JSON no establece su prioridad; deben evitarse coincidencias ambiguas. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

Un `Order` menor prevalece sobre uno mayor antes de comparar la especificidad. Con el mismo `Order`, un patrón específico tiene mayor precedencia que otro más general. En el alias de Inventario se utiliza `Order: -10`; no hay una restricción general de YARP que obligue a usar órdenes no negativos. Conserva esta explicación o instrucción operativa dentro del documento de la

etapa. - `PathRemovePrefix`: elimina un prefijo. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `PathSet`: establece una ruta fija. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

- `PathPattern`: aplica un patrón. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `RequestHeader` con `Set` o `Append`: fija o añade un valor de cabecera de petición. Para eliminarla se utiliza `RequestHeaderRemove`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `ResponseHeader` con `Set` o `Append`: fija o añade un valor de cabecera de respuesta. Para eliminarla se utiliza `ResponseHeaderRemove`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

- `Set` en estas transformaciones asigna un literal. Por ejemplo, `Set: generated` enviaría el texto generated; no genera un identificador. La correlación dinámica se obtiene del middleware de M07 y se propaga mediante código. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `QueryRouteParameter`: establece el nombre del parámetro de consulta; su `Set` o `Append` indica el nombre del valor de ruta del que se toma el dato. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. - `QueryValueParameter`: establece el nombre del parámetro de consulta; su `Set` o `Append` contiene un valor literal. En el alias se utiliza `QueryValueParameter: canal` con `Set: gateway`. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

Registra una condición o una decisión que podrá contrastarse al revisar la etapa. Las transformaciones de petición se aplican en el orden declarado dentro de `Transforms`; no existe una reordenación automática por categorías de ruta, cabecera y consulta. El alias docente declara: Conserva esta explicación o instrucción operativa dentro del documento de la etapa. 1. `PathRemovePrefix: /inventario`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

2. `RequestHeader: X-Canal` con `Set: Gateway`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. 3. `QueryValueParameter: canal` con `Set: gateway`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Así, `/inventario/api/v1/materiales/Acero` se reenvía como `/api/v1/materiales/Acero`, conservando `/api/v1`. El proxy añade X-Canal: Gateway y fija canal=gateway; no elimina el prefijo de versión del backend. Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

La ruta se selecciona según el valor de una cabecera declarada en `Match.Headers`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. 1. Documentar primero `Order` y después la especificidad; no confiar en el orden de escritura del JSON. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Añade una fila a la tabla documental; sus columnas deben conservarse al copiarla.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

## PASO 8.3.5 — Crear la carpeta `Enrutamiento` en el proyecto de Producción

La carpeta separa el modelo y su validador de las implementaciones HTTP y del dominio. El catálogo describe decisiones; no modifica por sí mismo el proxy.

1. En Solution Explorer expande `src/AceriaApp.Api`.
2. Crea dentro de ese proyecto una carpeta llamada `Enrutamiento`.

3. Comprueba que su ruta es `src/AceriaApp.Api/Enrutamiento` y que no está dentro de otra carpeta de catálogo.
4. Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Verificación concreta.** Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Diagnóstico si falla.**

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

## PASO 8.3.6 — Crear la clase `ReglaEnrutamiento`

ReglaEnrutamiento describe la correspondencia entre una ruta pública y su destino. En la implementación real hay que comprobar también las transformaciones del prefijo y los parámetros de ruta que llegan al backend.

1. Crea `src/AceriaApp.Api/Enrutamiento/ReglaEnrutamiento.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Enrutamiento/ReglaEnrutamiento.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Enrutamiento;

public class ReglaEnrutamiento
{
    public string Patron { get; init; } = string.Empty;
    public string MetodoHttp { get; init; } = string.Empty;
    public int Orden { get; init; }
    public bool TieneTransformacionDeRuta { get; init; }
    public bool TieneTransformacionDeCabecera { get; init; }
    public bool TieneTransformacionDeConsulta { get; init; }
}
```

Sitúa los tipos en un espacio de nombres propio, separado del resto de etapas y responsabilidades. Declara el tipo responsable de este comportamiento; sus miembros quedan dentro del bloque siguiente. Expone la propiedad indicada; init limita su asignación a la construcción del objeto y evita cambios posteriores accidentales.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

## PASO 8.3.7 — Crear la clase `ValidadorEnrutamiento`

El validador de reglas estudia conflictos y coherencia de las descripciones. La prueba de tráfico verifica otra propiedad: que la ruta pública se transforma y llega al endpoint esperado sin perder datos.

1. Crea `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Enrutamiento;

public class ValidadorEnrutamiento
{
    private static readonly string[] MetodosPermitidos = { "GET", "POST", "PUT", "PATCH", "DELETE", "HEAD",
"OPTIONS" };

    public ResultadoEnrutamiento Validar(IEnumerable<ReglaEnrutamiento> reglas)
    {
        var errores = new List<string>();
        var lista = reglas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay reglas declaradas.");
        }

        foreach (var regla in lista)
        {
            if (string.IsNullOrWhiteSpace(regla.Patron))
            {
                errores.Add("Una regla no declara el patrón.");
            }

            if (!MetodosPermitidos.Contains(regla.MetodoHttp))
            {
                errores.Add($"Método HTTP no permitido en '{regla.Patron}': '{regla.MetodoHttp}'.");
            }

            if (regla.Orden < 0)
            {
                errores.Add($"El orden de la regla '{regla.Patron}' es negativo: '{regla.Orden}'.");
            }

            if (!regla.TieneTransformacionDeRuta &&
                !regla.TieneTransformacionDeCabecera &&
                !regla.TieneTransformacionDeConsulta)
            {
                errores.Add($"La regla '{regla.Patron}' no declara ninguna transformación.");
            }
        }

        return new ResultadoEnrutamiento(errores.Count == 0, errores);
    }
}

public record ResultadoEnrutamiento(bool EsValida, IReadOnlyList<string> Errores);
```

Asigna el valor indicado a esta variable, propiedad u opción dentro de la operación actual. Declara la operación, sus parámetros y su resultado; el bloque siguiente contiene su implementación. Obtiene o prepara este valor local para las comprobaciones y operaciones que siguen.

Materializa la secuencia para consultar y recorrer los mismos elementos sin repetir su enumeración. Evalúa esta condición antes de continuar; el bloque asociado trata específicamente ese caso. Acumula un incumplimiento del contrato didáctico; el resultado final indicará que la declaración no es válida.

Recorre los elementos de la colección para aplicarles la misma comprobación. Completa esta expresión o invocación dentro de ValidadorEnrutamiento.cs; se conserva íntegra para reproducir el comportamiento. Devuelve el resultado calculado al llamador y termina este recorrido del método.

Define un contrato de datos con los campos que el consumidor utilizará para interpretar el resultado.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin reglas.

## PASO 8.3.8 — Añadir los `using` necesarios en `ValidadorEnrutamiento`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Enrutamiento;

public class ValidadorEnrutamiento
{
    private static readonly string[] MetodosPermitidos = { "GET", "POST", "PUT", "PATCH", "DELETE", "HEAD",
"OPTIONS" };

    public ResultadoEnrutamiento Validar(IEnumerable<ReglaEnrutamiento> reglas)
    {
        var errores = new List<string>();
        var lista = reglas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay reglas declaradas.");
        }

        foreach (var regla in lista)
        {
            if (string.IsNullOrWhiteSpace(regla.Patron))
            {
                errores.Add("Una regla no declara el patrón.");
            }

            if (!MetodosPermitidos.Contains(regla.MetodoHttp))
            {
                errores.Add($"Método HTTP no permitido en '{regla.Patron}': '{regla.MetodoHttp}'.");
            }
```

```
            if (regla.Orden < 0)
            {
                errores.Add($"El orden de la regla '{regla.Patron}' es negativo: '{regla.Orden}'.");
            }

            if (!regla.TieneTransformacionDeRuta &&
                !regla.TieneTransformacionDeCabecera &&
                !regla.TieneTransformacionDeConsulta)
            {
                errores.Add($"La regla '{regla.Patron}' no declara ninguna transformación.");
            }
        }

        return new ResultadoEnrutamiento(errores.Count == 0, errores);
    }
}

public record ResultadoEnrutamiento(bool EsValida, IReadOnlyList<string> Errores);
```

Importa el espacio de nombres requerido por los tipos y métodos utilizados en este archivo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Si un using no se resuelve, comprueba el espacio de nombres y la referencia del proyecto; usa la ubicación mostrada en el archivo completo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.3.9 — Crear la clase `CatalogoReglasEnrutamiento`

CatalogoReglasEnrutamiento facilita comparar propuestas. Su resultado no cambia la tabla de rutas activa en YARP hasta que se modifique y recargue la configuración real.

1. Crea `src/AceriaApp.Api/Enrutamiento/CatalogoReglasEnrutamiento.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Enrutamiento/CatalogoReglasEnrutamiento.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Enrutamiento;

public class CatalogoReglasEnrutamiento
{
    public static List<ReglaEnrutamiento> ReglasPorDefecto()
    {
        return new List<ReglaEnrutamiento>
        {
            new()
            {
                Patron = "/api/v1/ordenes/{id:guid}",
                MetodoHttp = "GET",
                Orden = 1,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = false
            },
            new()
            {
```

```
                Patron = "/api/v1/ordenes/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 2,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = false
            },
            new()
            {
                Patron = "/api/v1/inventario/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 3,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = true
            },
            new()
            {
                Patron = "/api/v1/clientes/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 4,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = true
            }
        };
    }
}
```

Completa esta expresión o invocación dentro de CatalogoReglasEnrutamiento.cs; se conserva íntegra para reproducir el comportamiento.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Devolver `List<T>` mutable: expone la implementación interna.

## PASO 8.3.10 — Añadir el `using` necesario en `CatalogoReglasEnrutamiento`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. Abre `src/AceriaApp.Api/Enrutamiento/CatalogoReglasEnrutamiento.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Enrutamiento/CatalogoReglasEnrutamiento.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
namespace AceriaApp.Api.Enrutamiento;

public class CatalogoReglasEnrutamiento
{
    public static List<ReglaEnrutamiento> ReglasPorDefecto()
    {
        return new List<ReglaEnrutamiento>
        {
            new()
```

```
            {
                Patron = "/api/v1/ordenes/{id:guid}",
                MetodoHttp = "GET",
                Orden = 1,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = false
            },
            new()
            {
                Patron = "/api/v1/ordenes/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 2,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = false
            },
            new()
            {
                Patron = "/api/v1/inventario/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 3,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = true
            },
            new()
            {
                Patron = "/api/v1/clientes/{**catch-all}",
                MetodoHttp = "GET",
                Orden = 4,
                TieneTransformacionDeRuta = true,
                TieneTransformacionDeCabecera = true,
                TieneTransformacionDeConsulta = true
            }
        };
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.3.11 — Crear el proyecto de tests del punto 8.3

El punto 8.3 añade un proyecto de tests específico para las clases de enrutamiento. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea manualmente `tests/AceriaApp.Enrutamiento.Tests` y su archivo `.csproj`; al usar este bloque no se genera `UnitTest1.cs`.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj`. **Operación:** Crear el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
```

```
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Organiza propiedades o referencias del proyecto MSBuild; conserva la apertura y el cierre de los elementos. Configura .NET 8 y las opciones de compilación de este proyecto. Fija la dependencia y su versión para restaurar el mismo paquete en cada equipo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 8.2: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

## PASO 8.3.12 — Añadir la referencia del proyecto de tests a la API de Producción

La referencia permite que el proyecto de tests acceda a las clases de `Enrutamiento` del proyecto de Producción.

1. Abre `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj` y añade la referencia a `AceriaApp.Api` usando el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Referencia el proyecto indicado mediante una ruta relativa a este archivo .csproj.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

## PASO 8.3.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj` y fija FluentAssertions 6.12.2 con el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

## PASO 8.3.14 — Crear el archivo `ValidadorEnrutamientoTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza las reglas que violan las reglas de enrutamiento.

1. Crea `tests/AceriaApp.Enrutamiento.Tests/ValidadorEnrutamientoTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/ValidadorEnrutamientoTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.Enrutamiento;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Enrutamiento.Tests;

public class ValidadorEnrutamientoTests
{
    private readonly ValidadorEnrutamiento _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValida()
    {
        var resultado = _validador.Validar(CatalogoReglasEnrutamiento.ReglasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinReglas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ReglaEnrutamiento>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay reglas"));
    }

    [Fact]
    public void Validar_MetodoNoPermitido_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "TRACE", Orden = 1, TieneTransformacionDeRuta = true }
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Método HTTP no permitido"));
    }

    [Fact]
    public void Validar_OrdenNegativo_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "GET", Orden = -1, TieneTransformacionDeRuta = true }
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("orden de la regla"));
    }

    [Fact]
    public void Validar_SinTransformaciones_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "GET", Orden = 1 }
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara ninguna transformación"));
    }
}
```

Declara una prueba individual que xUnit debe descubrir y ejecutar. Comprueba esta condición observable; una diferencia hace fallar la prueba en vez de ocultar el problema.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.Enrutamiento;`: las clases no se resuelven.
- La sintaxis FluentAssertions exige su paquete y su using; las aserciones equivalentes de xUnit también son válidas.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

## PASO 8.3.15 — Crear el archivo `CatalogoReglasEnrutamientoTests.cs`

Los tests verifican que el catálogo por defecto contiene las reglas esperadas, ordenadas ascendentemente y con las transformaciones requeridas.

1. Crea `tests/AceriaApp.Enrutamiento.Tests/CatalogoReglasEnrutamientoTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/CatalogoReglasEnrutamientoTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.Enrutamiento;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Enrutamiento.Tests;

public class CatalogoReglasEnrutamientoTests
{
    [Fact]
    public void ReglasPorDefecto_ContieneCuatroReglas()
    {
        var reglas = CatalogoReglasEnrutamiento.ReglasPorDefecto();

        reglas.Should().HaveCount(4);
    }

    [Fact]
    public void ReglasPorDefecto_OrdenadasAscendentemente()
    {
        var reglas = CatalogoReglasEnrutamiento.ReglasPorDefecto();

        var ordenes = reglas.Select(r => r.Orden).ToList();
        ordenes.Should().BeInAscendingOrder();
    }

    [Fact]
    public void ReglasPorDefecto_TodasTienenTransformacionDeRuta()
    {
        var reglas = CatalogoReglasEnrutamiento.ReglasPorDefecto();

        reglas.Should().OnlyContain(r => r.TieneTransformacionDeRuta);
    }

    [Fact]
    public void ReglasPorDefecto_TodasTienenTransformacionDeCabecera()
    {
        var reglas = CatalogoReglasEnrutamiento.ReglasPorDefecto();
```

```
        reglas.Should().OnlyContain(r => r.TieneTransformacionDeCabecera);
    }

    [Fact]
    public void ReglasPorDefecto_PrimeraReglaEsOrdenesPorId()
    {
        var reglas = CatalogoReglasEnrutamiento.ReglasPorDefecto();

        reglas[0].Patron.Should().Be("/api/v1/ordenes/{id:guid}");
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.Enrutamiento;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Select` y `OnlyContain` no se resuelven.
- No verificar el orden ascendente: la precedencia no se comprueba.

## PASO 8.3.16 — Compilar la solución completa

El ensayo de enrutamiento debe relacionar respuesta, código HTTP y backend seleccionado. Una cabecera de correlación válida tiene que propagarse según la política configurada, y las rutas heredadas deben seguir funcionando.

1. Escribe todos los archivos de ampliación funcional mostrados a continuación antes de compilar; cuando se indica reemplazar, sustituye el contenido de la copia actual.
2. Desde la carpeta actual ejecuta `dotnet sln AceriaApp.sln add (Get-ChildItem src,tests -Recurse -Filter *.csproj | ForEach-Object FullName)` en PowerShell 7.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release -m:1`.
4. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 38 proyectos; el compilador debe terminar con cero errores.

**Archivo:** `src/AceriaApp.Gateway/appsettings.json`. **Operación:** Reemplazar el contenido completo.

```
{
  "Urls": "http://localhost:5100",
  "Observabilidad": {
    "NivelMinimo": "Information"
  },
  "ReverseProxy": {
    "Routes": {
      "ordenes": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/ordenes/{**resto}"
        }
      },
      "produccion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/produccion/{**resto}"
        }
      },
      "comunicacion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/comunicacion/{**resto}"
        }
      },
      "materiales": {
        "ClusterId": "inventario-cluster",
        "Match": {
          "Path": "/api/v1/materiales/{**resto}",
```

```
        "Methods": [
          "GET",
          "HEAD"
        ]
      }
    },
    "planificaciones": {
      "ClusterId": "inventario-cluster",
      "Match": {
        "Path": "/api/v1/planificaciones/{**resto}",
        "Methods": [
          "GET",
          "HEAD"
        ]
      }
    },
    "inventario-materiales": {
      "ClusterId": "inventario-cluster",
      "Order": -10,
      "Match": {
        "Path": "/inventario/api/v1/materiales/{**resto}",
        "Methods": [
          "GET",
          "HEAD"
        ]
      },
      "Transforms": [
        {
          "PathRemovePrefix": "/inventario"
        },
        {
          "RequestHeader": "X-Canal",
          "Set": "Gateway"
        },
        {
          "QueryValueParameter": "canal",
          "Set": "gateway"
        }
      ]
    },
    "inventario-planificaciones": {
      "ClusterId": "inventario-cluster",
      "Order": -10,
      "Match": {
        "Path": "/inventario/api/v1/planificaciones/{**resto}",
        "Methods": [
          "GET",
          "HEAD"
        ]
      },
      "Transforms": [
        {
          "PathRemovePrefix": "/inventario"
        },
        {
          "RequestHeader": "X-Canal",
          "Set": "Gateway"
        },
        {
          "QueryValueParameter": "canal",
          "Set": "gateway"
        }
      ]
    }
  },
  "Clusters": {
    "produccion-cluster": {
      "Destinations": {
        "produccion": {
          "Address": "http://localhost:5105/"
        }
      }
    },
    "inventario-cluster": {
      "Destinations": {
        "inventario-1": {
```

```
            "Address": "http://localhost:5106/"
          }
        }
      }
    }
  }
}
```

Abre o cierra esta estructura JSON; las comas y delimitadores mantienen válido el documento. Establece la dirección de escucha de este proceso; conserva el nombre y el tipo json. Asigna este valor a la opción observabilidad; conserva el nombre y el tipo json.

Asigna este valor a la opción nivelminimo; conserva el nombre y el tipo json. Reúne la configuración que lee addreverseproxy; conserva el nombre y el tipo json. Declara las rutas públicas que yarp puede seleccionar; conserva el nombre y el tipo json.

Asigna este valor a la opción ordenes; conserva el nombre y el tipo json. Vincula la ruta con un cluster existente; conserva el nombre y el tipo json. Asigna este valor a la opción match; conserva el nombre y el tipo json.

Define el patrón público o la ruta del sondeo según su sección; conserva el nombre y el tipo json. Asigna este valor a la opción produccion; conserva el nombre y el tipo json. Asigna este valor a la opción comunicacion; conserva el nombre y el tipo json.

Asigna este valor a la opción materiales; conserva el nombre y el tipo json. Restringe los métodos que esta ruta admite; conserva el nombre y el tipo json. Asigna este valor a la opción planificaciones; conserva el nombre y el tipo json.

Asigna este valor a la opción inventario-materiales; conserva el nombre y el tipo json. Asigna prioridad explícita; los valores menores se evalúan antes; conserva el nombre y el tipo json. Declara cambios ordenados sobre la petición reenviada; conserva el nombre y el tipo json.

Elimina únicamente el prefijo configurado; conserva el nombre y el tipo json. Selecciona la cabecera que se transforma; conserva el nombre y el tipo json. Fija un valor literal; no ejecuta código ni genera identificadores; conserva el nombre y el tipo json.

Selecciona un parámetro de consulta para el valor literal configurado; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-planificaciones; conserva el nombre y el tipo json. Agrupa destinos que ofrecen el mismo contrato http; conserva el nombre y el tipo json.

Asigna este valor a la opción produccion-cluster; conserva el nombre y el tipo json. Enumera instancias del servicio, no servicios con contratos diferentes; conserva el nombre y el tipo json. Establece la url interna del destino; conserva el nombre y el tipo json.

Asigna este valor a la opción inventario-cluster; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-1; conserva el nombre y el tipo json.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/GatewayEnrutamientoFuncionalTests.cs`. **Operación:** Crear el contenido completo.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Pruebas.Gateway;
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.Configuration;
using Xunit;

namespace AceriaApp.Enrutamiento.Tests;

public sealed class GatewayEnrutamientoFuncionalTests
{
    // Detecta prefijo sin retirar y Set convertido por error en Append o literal no admitido.
    [Theory]
```

```
    [InlineData("materiales/Acero")] [InlineData("planificaciones/42")]
    public async Task El_alias_retira_el_prefijo_y_fija_cabecera_y_query(string recurso)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var peticion = new HttpRequestMessage(HttpMethod.Get, "/inventario/api/v1/" + recurso + "?
pagina=2&canal=cliente");
        peticion.Headers.Add("X-Canal", "Cliente");
        using var respuesta = await prueba.Cliente.SendAsync(peticion);
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        var eco = await respuesta.Content.ReadFromJsonAsync<EcoPeticion>();
        Assert.Equal("inventario-1", eco!.Instancia);
        Assert.Equal("/api/v1/" + recurso, eco.Ruta);
        Assert.Equal("Gateway", eco.Canal);
        var query = Microsoft.AspNetCore.WebUtilities.QueryHelpers.ParseQuery(eco.Query);
        Assert.Equal("2", query["pagina"].ToString());
        Assert.Equal("gateway", Assert.Single(query["canal"]));
    }

    [Fact]
    public async Task La_ruta_publica_directa_conserva_la_query_y_la_cabecera_originales()
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var peticion = new HttpRequestMessage(HttpMethod.Get, "/api/v1/materiales/Acero?
canal=cliente");
        peticion.Headers.Add("X-Canal", "Cliente");
        using var respuesta = await prueba.Cliente.SendAsync(peticion);
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        var eco = await respuesta.Content.ReadFromJsonAsync<EcoPeticion>();
        Assert.Equal("?canal=cliente", eco!.Query);
        Assert.Equal("Cliente", eco.Canal);
    }

    [Theory]
    [InlineData("/inventario/diagnostico")] [InlineData("/inventario/diagnostico/error")]
    [InlineData("/inventario/health/ready")] [InlineData("/inventario/health/live")]
    [InlineData("/inventario/api/v1/ordenes")] [InlineData("/inventario/desconocida")]
    public async Task El_alias_no_abre_rutas_internas_ni_recursos_no_publicados(string ruta)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.GetAsync(ruta);
        Assert.Equal(HttpStatusCode.NotFound, respuesta.StatusCode);
        Assert.Equal(0, prueba.Produccion.Peticiones + prueba.Inventario1.Peticiones);
    }

    [Theory]
    [InlineData("POST")] [InlineData("PUT")] [InlineData("PATCH")] [InlineData("DELETE")]
    public async Task El_alias_de_inventario_sigue_siendo_de_solo_lectura(string metodo)
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.SendAsync(new HttpRequestMessage(new HttpMethod(metodo),
"/inventario/api/v1/materiales/Acero"));
        Assert.Equal(HttpStatusCode.MethodNotAllowed, respuesta.StatusCode);
        Assert.Equal(0, prueba.Inventario1.Escrituras);
    }

    [Fact]
    public async Task El_orden_explicito_del_alias_precede_a_una_ruta_competidora()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(configurar: builder =>
            builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["ReverseProxy:Routes:competidora:ClusterId"] = "produccion-cluster",
                ["ReverseProxy:Routes:competidora:Order"] = "-5",
                ["ReverseProxy:Routes:competidora:Match:Path"] = "/inventario/{**resto}"
            }));
        using var respuesta = await prueba.Cliente.GetAsync("/inventario/api/v1/materiales/Acero");
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal("inventario-1", (await respuesta.Content.ReadFromJsonAsync<EcoPeticion>())!.Instancia);
    }

    [Fact]
    public async Task El_alias_acepta_HEAD_y_no_devuelve_cuerpo()
    {
        await using var prueba = await GatewayPruebas.CrearAsync();
        using var respuesta = await prueba.Cliente.SendAsync(new HttpRequestMessage(HttpMethod.Head, "/
inventario/api/v1/materiales/Acero"));
```

```
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        Assert.Equal("inventario-1", Assert.Single(respuesta.Headers.GetValues("X-Instancia")));
        Assert.Equal("", await respuesta.Content.ReadAsStringAsync());
        Assert.Equal(1, prueba.Inventario1.Peticiones);
    }
}
```

Documenta la intención de la instrucción próxima sin alterar su ejecución. Declara una prueba parametrizada; cada conjunto de datos produce una ejecución. Proporciona un caso de entrada concreto para la prueba parametrizada.

Delimita la vida del recurso y garantiza su disposición al salir del ámbito, también si ocurre una excepción; await using permite la disposición asíncrona. Lee o escribe la cabecera HTTP indicada; su valor debe coincidir con el contrato de transformación. Espera la operación asíncrona sin bloquear el hilo; un fallo se propaga para poder detectarlo.

Completa esta expresión o invocación dentro de GatewayEnrutamientoFuncionalTests.cs; se conserva íntegra para reproducir el comportamiento.

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Gateway/AceriaApp.Gateway.csproj" />
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <Compile Include="../Compartido/GatewayPruebas.cs" Link="Compartido/GatewayPruebas.cs" />
    <Content Include="../../src/AceriaApp.Gateway/appsettings.json" Link="gateway.appsettings.json"
CopyToOutputDirectory="PreserveNewest" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Hace disponibles las API del framework ASP.NET Core para los servidores de prueba. Incluye el soporte de pruebas compartido en la compilación de este proyecto.

**Verificación concreta.** Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 38 proyectos; el compilador debe terminar con cero errores.

**Diagnóstico si falla.**

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

## PASO 8.3.17 — Ejecutar todos los tests del punto 8.3

Una prueba de catálogo acredita su declaración. Las pruebas funcionales utilizan el proxy y servidores HTTP reales, pero la persistencia y el broker se comprueban con la suite y el recorrido completo.

1. Ejecuta `dotnet test tests/AceriaApp.Enrutamiento.Tests/AceriaApp.Enrutamiento.Tests.csproj -c Release --no-build` en la terminal.

2. Comprueba que se ejecutan 25 pruebas: 10 de catálogo y 15 funcionales.
3. Si hay un fallo, identifica su nombre y revisa la regla o la respuesta esperada antes de modificar código.
4. Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Verificación concreta.** Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Diagnóstico si falla.**

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

## PASO 8.3.18 — Ejecutar todos los tests de la solución

La suite completa comprueba que el nuevo punto no rompe la persistencia, la mensajería, el dominio, la observabilidad ni la resiliencia desarrollados antes.

1. Configura `ACERIA_TEST_SQLSERVER` para tu SQL Server Express/LocalDB y `ACERIA_TEST_RABBITMQ` para el broker; sigue los requisitos del inicio del manual.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build -m:1`.
3. Comprueba que pasan las 476 pruebas acumuladas, incluidas las heredadas de M07, sin omisiones.
4. Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Verificación concreta.** Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Diagnóstico si falla.**

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

## PASO 8.3.19 — Crear el README del punto 8.3

El README del punto 8.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 8.2.

1. Abre `README.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 8.3 — Enrutamiento de peticiones

## Objetivo

Aplicar coincidencias de ruta, restricciones de método y transformaciones observables de petición.

## Estado del proyecto

Etapa acumulativa con 38 proyectos, 25 proyectos de tests y 48 documentos. El cierre exige 476 pruebas
```

```
aprobadas sin omisiones; los resultados se registran en los informes del módulo.

## Cómo ejecutarlo

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test
AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite
completa.

El recorrido funcional inicia Gateway en 5100, Producción en 5105 e Inventario en 5106. La etapa 8.4 añade
Inventario en 5107.

## Qué se ha añadido respecto al punto anterior

- Modelo ReglaEnrutamiento, validador ValidadorEnrutamiento y catálogo CatalogoReglasEnrutamiento.
- 10 pruebas de catálogo y 15 pruebas funcionales nuevas.
- Documento temático y ADR-033.
- Reto optativo en copia separada, con una prueba adicional.

## Documentación

- `docs/enrutamiento-peticiones.md`.
- `docs/ADR-033-enrutamiento-peticiones.md`.
- Las 46 fichas y decisiones anteriores permanecen idénticas.
```

`Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite completa`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. `- `docs/enrutamiento-peticiones.md`` . Registra una condición o una decisión que podrá contrastarse al revisar la etapa. `- `docs/ADR-033-enrutamiento-peticiones.md`` . Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar el README del punto 8.2: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 8.2: rompe la separación.

## PASO 8.3.20 — Crear el ADR-033 del punto 8.3

El ADR-033 documenta la decisión sobre el enrutamiento de peticiones. Es un artefacto nuevo del punto 8.3 que no modifica los ADR existentes.

1. Crea `docs/ADR-033-enrutamiento-peticiones.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/ADR-033-enrutamiento-peticiones.md`. **Operación:** Crear el contenido completo.

```
# ADR-033: Enrutamiento de peticiones

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado YARP como gateway. Es necesario definir las reglas de enrutamiento y las
transformaciones aplicadas a las peticiones.

## Decisión
Se documenta la selección de endpoints de ASP.NET Core: primero el menor `Order`, después la especificidad
```

```
cuando Order coincide. Los órdenes negativos son válidos y la posición en el objeto JSON no establece la prioridad. Los recursos de Inventario y sus alias permiten GET y HEAD. Los alias `/inventario/api/v1/materiales/{**resto}` y `/inventario/api/v1/planificaciones/{**resto}` usan Order -10 y transformaciones en su orden declarado: eliminar solo `/inventario`, fijar `X-Canal: Gateway` y fijar `canal=gateway` mediante `QueryValueParameter`. No se elimina `/api/v1`. `QueryRouteParameter` obtiene un valor de ruta; `QueryValueParameter` usa un literal con Set o Append. Set en una cabecera también asigna un literal, no genera valores dinámicos. La correlación validada procede del middleware de M07 y se propaga por código. Los tests Kestrel comprueban coincidencia, métodos y transformaciones; el catálogo mantiene reglas pedagógicas propias.

## Alternativas consideradas
- Confiar en la posición de las rutas en JSON: descartado porque no es el criterio de prioridad de endpoints. Usar el mismo Order puede ser válido si la especificidad permite una selección no ambigua.
- Transformaciones en cada servicio: descartado por duplicación.
- Enrutamiento por host: descartado por simplicidad del proyecto.

## Consecuencias
- Positivas: enrutamiento predecible, transformaciones centralizadas, coherencia.
- Negativas: la configuración requiere mantener el orden de las rutas.
- Mitigación: la validación automática y las buenas prácticas documentadas.

## Alcance comprobable de esta etapa

Orden >= 0 y las transformaciones obligatorias son reglas del catálogo, no requisitos generales de YARP. No se publica un comodín global ni el diagnóstico interno. Copiar una cabecera arbitraria a la consulta requeriría una transformación personalizada. Aceptado registra la decisión; la ejecución y sus evidencias deben comprobarse por separado.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos indicados en el manual.
```

`Se documenta la selección de endpoints de ASP.NET Core: primero el menor `Order`, después la especificidad cuando Order coincide. Los órdenes negativos son válidos y la posición en el objeto JSON no establece la prioridad. Los recursos de Inventario y sus alias permiten GET y HEAD. Los alias `/inventario/api/v1/materiales/{**resto}` y `/inventario/api/v1/planificaciones/{**resto}` usan Order -10 y transformaciones en su orden declarado: eliminar solo `/inventario`, fijar `X-Canal: Gateway` y fijar `canal=gateway` mediante `QueryValueParameter`. No se elimina `/api/v1`. `QueryRouteParameter` obtiene un valor de ruta; `QueryValueParameter` usa un literal con Set o Append. Set en una cabecera también asigna un literal, no genera valores dinámicos. La correlación validada procede del middleware de M07 y se propaga por código. Los tests Kestrel comprueban coincidencia, métodos y transformaciones; el catálogo mantiene reglas pedagógicas propias.` Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-033`: rompe la convención.

## PASO 8.3.21 — Verificar que no se ha modificado ningún archivo del punto 8.2

La progresión modifica copias independientes. Los nuevos archivos o configuraciones de una etapa no deben alterar un punto ya entregado.

1. Abre las carpetas `docs` de la etapa actual y de `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp`.
2. Compara con Get-FileHash los archivos que existían en la etapa anterior; cada hash SHA256 debe coincidir.
3. Comprueba también que los archivos de tests heredados conservan su contenido; solo se incorporan los tests del punto.

4. Verifica que `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.3-enrutamiento-peticiones`.

**Verificación concreta.** Verifica que `M08_API_GATEWAY/PROYECTO/8.2-configuracion-basica-yarp` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.3-enrutamiento-peticiones`.

**Diagnóstico si falla.**

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

## PASO 8.3.22 — Verificar la estructura del punto 8.3

El inventario convierte la comprobación de estructura en una condición observable y detecta omisiones que una compilación aislada podría pasar por alto.

1. Ejecuta `Get-ChildItem -Recurse -File` y revisa las carpetas src, tests y docs.
2. Verifica 38 proyectos, 25 proyectos de pruebas y 48 documentos.
3. Comprueba la presencia de las tres clases de `Enrutamiento`, sus tests, el documento temático y ADR-033.
4. Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Verificación concreta.** Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable.

**Diagnóstico si falla.**

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 8.2: viola la regla de no modificar.
- Omitir la carpeta `Enrutamiento`: los tests no encuentran las clases.

## PASO 8.3.23 — Verificar el `.gitignore` del punto 8.3

Las exclusiones conservan un repositorio reproducible sin binarios temporales ni datos específicos de Visual Studio.

1. Abre `.gitignore` de la etapa actual.
2. Comprueba que contiene bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
3. Compara su contenido con el archivo heredado y conserva sus exclusiones.
4. Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Verificación concreta.** Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Diagnóstico si falla.**

- Modificar el `.gitignore` copiado del punto 8.2: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

### PASO 8.3.24 — Hacer el commit del punto 8.3 y verificar el estado final

Un commit conserva el estado comprobado. El número de commits no tiene que coincidir con el de puntos: las correcciones y revisiones también forman parte del historial.

1. Desde la raíz del repositorio ejecuta `git status --short` y revisa todos los cambios de la etapa.
2. Añade exclusivamente los archivos docentes y del proyecto con `git add M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones`.
3. Ejecuta `git commit -m "Punto 8.3: Enrutamiento de peticiones"` después de comprobar las pruebas.
4. Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Verificación concreta.** Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Diagnóstico si falla.**

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

## Reto de ampliación (optativo)

añadir un método `Resumen` a `ValidadorEnrutamiento` que devuelva el número total de reglas, el número de reglas con transformación de ruta, el número con transformación de cabecera y el número con transformación de consulta, y un test que verifique el cálculo con el catálogo por defecto.

Crea una copia de `8.3-enrutamiento-peticiones` llamada `reto-8-3` fuera de las cuatro carpetas obligatorias. La copia anterior permanece intacta. Aplica las tres sustituciones completas siguientes en orden.

### Ampliación 1: añadir el contrato de resumen

**Archivo:** `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Enrutamiento;

public class ValidadorEnrutamiento
{
    private static readonly string[] MetodosPermitidos = { "GET", "POST", "PUT", "PATCH", "DELETE", "HEAD",
"OPTIONS" };

    public ResultadoEnrutamiento Validar(IEnumerable<ReglaEnrutamiento> reglas)
    {
        var errores = new List<string>();
        var lista = reglas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay reglas declaradas.");
        }

        foreach (var regla in lista)
        {
            if (string.IsNullOrWhiteSpace(regla.Patron))
            {
                errores.Add("Una regla no declara el patrón.");
            }

            if (!MetodosPermitidos.Contains(regla.MetodoHttp))
```

```
            {
                errores.Add($"Método HTTP no permitido en '{regla.Patron}': '{regla.MetodoHttp}'.");
            }

            if (regla.Orden < 0)
            {
                errores.Add($"El orden de la regla '{regla.Patron}' es negativo: '{regla.Orden}'.");
            }

            if (!regla.TieneTransformacionDeRuta &&
                !regla.TieneTransformacionDeCabecera &&
                !regla.TieneTransformacionDeConsulta)
            {
                errores.Add($"La regla '{regla.Patron}' no declara ninguna transformación.");
            }
        }

        return new ResultadoEnrutamiento(errores.Count == 0, errores);
    }
}

public record ResultadoEnrutamiento(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenEnrutamiento(int Total, int ConTransformacionRuta, int ConTransformacionCabecera, int
ConTransformacionConsulta);
```

## Ampliación 2: implementar el cálculo del resumen

**Archivo:** `src/AceriaApp.Api/Enrutamiento/ValidadorEnrutamiento.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Enrutamiento;

public class ValidadorEnrutamiento
{
    private static readonly string[] MetodosPermitidos = { "GET", "POST", "PUT", "PATCH", "DELETE", "HEAD",
"OPTIONS" };

    public ResultadoEnrutamiento Validar(IEnumerable<ReglaEnrutamiento> reglas)
    {
        var errores = new List<string>();
        var lista = reglas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay reglas declaradas.");
        }

        foreach (var regla in lista)
        {
            if (string.IsNullOrWhiteSpace(regla.Patron))
            {
                errores.Add("Una regla no declara el patrón.");
            }

            if (!MetodosPermitidos.Contains(regla.MetodoHttp))
            {
                errores.Add($"Método HTTP no permitido en '{regla.Patron}': '{regla.MetodoHttp}'.");
            }

            if (regla.Orden < 0)
            {
                errores.Add($"El orden de la regla '{regla.Patron}' es negativo: '{regla.Orden}'.");
            }

            if (!regla.TieneTransformacionDeRuta &&
                !regla.TieneTransformacionDeCabecera &&
                !regla.TieneTransformacionDeConsulta)
            {
                errores.Add($"La regla '{regla.Patron}' no declara ninguna transformación.");
            }
```

```
        }

        return new ResultadoEnrutamiento(errores.Count == 0, errores);
    }

    public static ResumenEnrutamiento Resumen(IEnumerable<ReglaEnrutamiento> reglas)
    {
        var lista = reglas.ToList();
        var conRuta = lista.Count(r => r.TieneTransformacionDeRuta);
        var conCabecera = lista.Count(r => r.TieneTransformacionDeCabecera);
        var conConsulta = lista.Count(r => r.TieneTransformacionDeConsulta);
        return new ResumenEnrutamiento(lista.Count, conRuta, conCabecera, conConsulta);
    }
}

public record ResultadoEnrutamiento(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenEnrutamiento(int Total, int ConTransformacionRuta, int ConTransformacionCabecera, int
ConTransformacionConsulta);
```

## Ampliación 3: verificar el resultado con el catálogo

**Archivo:** `tests/AceriaApp.Enrutamiento.Tests/ValidadorEnrutamientoTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Enrutamiento;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Enrutamiento.Tests;

public class ValidadorEnrutamientoTests
{
    private readonly ValidadorEnrutamiento _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValida()
    {
        var resultado = _validador.Validar(CatalogoReglasEnrutamiento.ReglasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinReglas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ReglaEnrutamiento>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay reglas"));
    }

    [Fact]
    public void Validar_MetodoNoPermitido_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "TRACE", Orden = 1, TieneTransformacionDeRuta = true }
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Método HTTP no permitido"));
    }

    [Fact]
    public void Validar_OrdenNegativo_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "GET", Orden = -1, TieneTransformacionDeRuta = true }
```

```
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("orden de la regla"));
    }

    [Fact]
    public void Validar_SinTransformaciones_DevuelveError()
    {
        var reglas = new List<ReglaEnrutamiento>
        {
            new() { Patron = "/api/v1/x", MetodoHttp = "GET", Orden = 1 }
        };

        var resultado = _validador.Validar(reglas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara ninguna transformación"));
    }

    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorEnrutamiento.Resumen(CatalogoReglasEnrutamiento.ReglasPorDefecto());

        resumen.Total.Should().Be(4);
        resumen.ConTransformacionRuta.Should().Be(4);
        resumen.ConTransformacionCabecera.Should().Be(4);
        resumen.ConTransformacionConsulta.Should().Be(2);
    }
}
```

**Ejecución y resultado esperado.** En la copia optativa ejecuta `dotnet test AceriaApp.sln -c Release -m:1`. Deben pasar 477 pruebas; el proyecto `AceriaApp.Enrutamiento.Tests` debe ejecutar 26. Verifica que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que los cuatro puntos obligatorios no cambiaron. Registra el reto en un commit independiente cuando lo hayas comprobado.

## Resultado esperado del punto

- Solución autónoma con 38 proyectos y 25 proyectos de pruebas.
- 48 documentos, incluidos el tema del punto y ADR-033.
- 10 pruebas nuevas de catálogo y 15 funcionales; 476 acumuladas sin omisiones.
- Etapa anterior, documentación y tests heredados conservados; el reto sigue separado.

## Conclusión y continuidad

El punto 8.3 ha definido el enrutamiento de peticiones en YARP con patrones, precedencia, transformaciones y orden de aplicación. El proyecto AceriaApp dispone ahora de un validador automático de reglas de enrutamiento que comprueba el patrón, el método HTTP, el orden y las transformaciones, y diez tests nuevos. Esta base permite avanzar hacia el punto 8.4, donde se estudiará el balanceo de carga básico como mecanismo para distribuir el tráfico entre varias instancias de un servicio, cerrando el módulo 8 y el curso con el resumen final del estado del proyecto.

---

**Fin del punto 8.3.**

---

# Punto 8.4 — Balanceo de carga básico

## Distribuir solicitudes entre destinos sin atribuir alta disponibilidad automática

El balanceo de carga decide qué instancia recibe cada nueva petición cuando existen varios destinos equivalentes. Esto no significa que una regla RoundRobin repare por sí sola una instancia caída ni que el gateway garantice alta disponibilidad del sistema completo. Conviene diferenciar selección de destino, salud de las instancias y recuperación ante fallos.

En el cierre de M08 combinaremos la configuración de YARP con ensayos de distribución y regresión de las rutas anteriores. Una lista de clusters válida es solo el punto de partida; el comportamiento debe observarse con backends accesibles, respuestas verificables y registros que permitan identificar qué destino atendió cada solicitud. El alcance docente no incluye una solución productiva completa de identidad, TLS ni cuotas.

**Carpeta de trabajo:** `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`.

**Objetivo.** Completar los 24 pasos, conservar la etapa anterior y comprobar el catálogo y el comportamiento que incorpora este punto.

### PASO 8.4.1 — Crear la carpeta del punto 8.4 a partir del punto 8.3

La copia conserva 38 proyectos y 476 pruebas antes de incorporar los cambios del punto.

1. En el Explorador de archivos localiza `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones` y comprueba que contiene `AceriaApp.sln`, `src`, `tests`, `docs` y `global.json`.
2. Crea la carpeta `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga` y copia allí el contenido de la etapa anterior, incluidas sus carpetas ocultas de configuración.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; abre únicamente la copia `8.4-balanceo-carga` para realizar este punto.
4. Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Verificación concreta.** Ejecuta `Get-ChildItem -Force` en PowerShell dentro de la copia y comprueba que las carpetas y la solución siguen presentes.

**Diagnóstico si falla.**

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

### PASO 8.4.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

La compilación comprueba que la etapa anterior está completa y que el SDK .NET 8 resuelve sus dependencias.

1. Abre en Visual Studio el archivo `AceriaApp.sln` de `8.4-balanceo-carga`.
2. Abre Terminal en esa carpeta y ejecuta `dotnet sln AceriaApp.sln list`; deben aparecer 38 proyectos.
3. Ejecuta `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Verificación concreta.** Comprueba que la salida termina con compilación correcta y cero errores antes de continuar.

**Diagnóstico si falla.**

- Abrir la solución del punto 8.3 en lugar de la del punto 8.4: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 8.3 en ejecución: el puerto queda ocupado y la compilación falla.

## PASO 8.4.3 — Verificar que la carpeta `docs` contiene los documentos del punto 8.3

Esta etapa añade dos documentos sin sobrescribir las 48 fichas y decisiones anteriores.

1. Expande `docs` en Solution Explorer y ejecuta `(Get-ChildItem docs -Filter *.md).Count`.
2. Comprueba que hay 48 documentos heredados y que incluyen los ADR-010 a ADR-033.
3. Compara los nombres y contenidos con `docs` de `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones`.
4. Confirma que todavía no existen `docs/balanceo-carga.md` ni `docs/ADR-034-balanceo-carga.md`.

**Verificación concreta.** Confirma que todavía no existen `docs/balanceo-carga.md` ni `docs/ADR-034-balanceo-carga.md`.

**Diagnóstico si falla.**

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 8.3.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

## PASO 8.4.4 — Crear el documento de balanceo de carga en `docs`

El balanceo se estudia como selección de un destino entre varias instancias equivalentes. No hay que confundir esa política con recuperación garantizada si los servidores no están preparados o carecen de health checks.

1. Crea `docs/balanceo-carga.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/balanceo-carga.md`. **Operación:** Crear el contenido completo.

```
# Balanceo de carga básico — Punto 8.4

## Definición

Distribución del tráfico entre varias instancias de un servicio.

## Algoritmos de balanceo

- RoundRobin: distribución cíclica.
- Random: distribución aleatoria.
- LeastRequests: instancia con menos peticiones.
- PowerOfTwoChoices: mejor entre dos aleatorias.
- Ponderado: concepto de distribución según pesos; requiere una política personalizada y no está implementado en este laboratorio.
```

```
## Configuración de destinos múltiples

    {
      "ReverseProxy": {
        "Clusters": {
          "inventario-cluster": {
            "LoadBalancingPolicy": "RoundRobin",
            "Destinations": {
              "inventario-1": { "Address": "http://localhost:5106/" },
              "inventario-2": { "Address": "http://localhost:5107/" }
            }
          },
          "produccion-cluster": {
            "Destinations": {
              "produccion": { "Address": "http://localhost:5105/" }
            }
          }
        }
      }
    }


## Pesos de destino

YARP no incorpora una propiedad nativa `Weight` en la configuración de destinos utilizada aquí. Un dato
Weight en metadatos solo tendría efecto si una extensión lo interpretara y una política personalizada
seleccionara los destinos según ese dato. El indicador TienePesos del catálogo expresa una decisión
conceptual; no instala esa extensión. El laboratorio utiliza dos destinos equivalentes de Inventario sin
ponderación.

## Algoritmo en YARP

La propiedad `LoadBalancingPolicy` del cluster configura el algoritmo. Este módulo fija `RoundRobin`
expresamente en inventario-cluster; no depende del algoritmo predeterminado de YARP. La sobrecarga sin
argumentos de `MapReverseProxy()` incorpora el middleware de balanceo. El reparto cíclico se aplica a
solicitudes nuevas entre destinos disponibles; no garantiza igual duración de trabajo ni reintenta
escrituras.

## Balanceo y health checks

El cluster de Inventario activa sondeos a `/health/ready`, con intervalo y timeout de un segundo, política
`ConsecutiveFailures` y umbral de dos fallos consecutivos. `AvailableDestinationsPolicy: HealthyAndUnknown`
permite destinos sanos o todavía no clasificados y excluye los marcados Unhealthy. Si no queda ninguno
disponible, devuelve 503; una petición ya asignada puede fallar durante la detección. Un sondeo satisfactorio
permite reincorporar la instancia recuperada sin reiniciar el gateway. Estos checks deben configurarse:
declarar su intención en el catálogo no los activa.

## Balanceo en contenedores

- Kubernetes: Service balancea entre pods.
- Docker Swarm: Service balancea entre tasks.
- Gateway: balancea entre instancias del servicio.

## Estrategias de escalado

- Horizontal: añade instancias.
- Vertical: aumenta recursos.
- Automático: ajusta instancias según métricas.

## Buenas prácticas

1. Disponer de varios destinos en el servicio cuyo reparto se quiere comprobar. Aquí Inventario tiene dos y
Producción conserva uno; no se declara alta disponibilidad completa del sistema.
2. Algoritmo adecuado al contexto.
3. Combinar con health checks.
4. Registrar métricas por instancia.
5. Validar con un test.

## Cierre del módulo 8

El cierre del módulo exige comprobar el estado siguiente en los proyectos reconstruidos y sus informes:

- Patrón API Gateway con punto de entrada único.
- Configuración de YARP con rutas y clusters.
- Enrutamiento con precedencia, transformaciones y orden.
```

```
- Balanceo RoundRobin entre dos destinos de Inventario y un destino de Producción.
- Inventario final de 39 proyectos, 26 proyectos de pruebas y 50 documentos.
- Objetivo de 491 pruebas acumuladas sin omisiones en la etapa 8.4; se acredita ejecutándolas, no mediante
esta descripción.
- Cobertura docente de ocho módulos y 34 puntos; el cierre requiere las evidencias Linux, Windows y PDF y su
revisión.

## Cierre del curso

El recorrido del curso reúne los módulos siguientes:

| Módulo | Tema | Puntos |
|--------|------|-------:|
| 1 | Introducción a los microservicios | 5 |
| 2 | APIs REST con ASP.NET Core | 4 |
| 3 | Arquitectura hexagonal | 4 |
| 4 | Patrones de arquitectura | 5 |
| 5 | DDD y CQRS | 5 |
| 6 | Comunicación entre microservicios | 3 |
| 7 | Observabilidad y resiliencia | 4 |
| 8 | Publicación mediante API Gateway | 4 |
| Total | Ocho módulos | 34 |

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Clusters | Inventario: 5106 y 5107; Producción: 5105 |
| Política | RoundRobin explícito en Inventario; sin pesos |
| Health checks | Sondeos activos de Inventario y HealthyAndUnknown; todos Unhealthy produce 503 |
| Identidad | X-Instancia distingue inventario-1 e inventario-2; ambas usan la misma base SQL |
| Validación | Tests del catálogo y Kestrel; integración real de SQL, RabbitMQ y LocalDB acreditada por sus
informes |

## Alcance comprobable de esta etapa

El comportamiento a verificar es reparto entre ambas identidades, retirada de un destino caído, respuesta 503
cuando todos están no disponibles y reincorporación al recuperarse. Los informes deben corresponder a los
mismos archivos y revisión que se entregan. Los recuentos esperados y el estado Aceptado de los ADR no
equivalen a un resultado PASS de CI.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

Define este encabezado en el documento que el alumno conservará junto al proyecto. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

YARP no incorpora una propiedad nativa `Weight` en la configuración de destinos utilizada aquí. Un dato Weight en metadatos solo tendría efecto si una extensión lo interpretara y una política personalizada seleccionara los destinos según ese dato. El indicador TienePesos del catálogo expresa una decisión conceptual; no instala esa extensión. El laboratorio utiliza dos destinos equivalentes de Inventario sin ponderación. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. La propiedad `LoadBalancingPolicy` del cluster configura el algoritmo. Este módulo fija `RoundRobin` expresamente en inventario-cluster; no depende del algoritmo predeterminado de YARP. La sobrecarga sin argumentos de `MapReverseProxy()` incorpora el middleware de balanceo. El reparto cíclico se aplica a solicitudes nuevas entre destinos disponibles; no garantiza igual duración de trabajo ni reintenta escrituras. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. El cluster de Inventario activa sondeos a `/health/ready`, con intervalo y timeout de un segundo, política `ConsecutiveFailures` y umbral de dos fallos consecutivos. `AvailableDestinationsPolicy: HealthyAndUnknown` permite destinos sanos o todavía no clasificados y excluye los marcados Unhealthy. Si no queda ninguno disponible, devuelve 503; una petición ya asignada puede fallar durante la detección. Un sondeo satisfactorio permite reincorporar la instancia recuperada sin reiniciar el gateway. Estos checks deben configurarse: declarar su intención en el catálogo no los activa. Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

Añade una fila a la tabla documental; sus columnas deben conservarse al copiarla.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

## PASO 8.4.5 — Crear la carpeta `Balanceo` en el proyecto de Producción

La carpeta separa el modelo y su validador de las implementaciones HTTP y del dominio. El catálogo describe decisiones; no modifica por sí mismo el proxy.

1. En Solution Explorer expande `src/AceriaApp.Api`.
2. Crea dentro de ese proyecto una carpeta llamada `Balanceo`.
3. Comprueba que su ruta es `src/AceriaApp.Api/Balanceo` y que no está dentro de otra carpeta de catálogo.
4. Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Verificación concreta.** Conserva las carpetas anteriores; esta carpeta agrupa únicamente el ejercicio de análisis del punto.

**Diagnóstico si falla.**

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

## PASO 8.4.6 — Crear la clase `ClusterBalanceo`

ClusterBalanceo contiene las características que queremos comparar. Los algoritmos reales actúan sobre destinos configurados en YARP; un campo que declara RoundRobin no distribuye por sí mismo las solicitudes.

1. Crea `src/AceriaApp.Api/Balanceo/ClusterBalanceo.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Balanceo/ClusterBalanceo.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Balanceo;

public class ClusterBalanceo
{
    public string Identificador { get; init; } = string.Empty;
    public string Politica { get; init; } = string.Empty;
    public int NumeroDeDestinos { get; init; }
    public bool TienePesos { get; init; }
    public bool CombinadoConHealthChecks { get; init; }
}
```

Sitúa los tipos en un espacio de nombres propio, separado del resto de etapas y responsabilidades. Declara el tipo responsable de este comportamiento; sus miembros quedan dentro del bloque siguiente. Expone la propiedad indicada; init limita su asignación a la construcción del objeto y evita cambios posteriores accidentales.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

## PASO 8.4.7 — Crear la clase `ValidadorBalanceo`

El validador de balanceo comprueba la descripción del escenario, mientras que una prueba de tráfico permite observar a qué instancia llegó cada petición. La segunda evidencia no puede deducirse de la primera.

1. Crea `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Balanceo;

public class ValidadorBalanceo
{
    private static readonly string[] PoliticasPermitidas = { "RoundRobin", "Random", "LeastRequests",
"PowerOfTwoChoices" };

    public ResultadoBalanceo Validar(IEnumerable<ClusterBalanceo> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (!PoliticasPermitidas.Contains(cluster.Politica))
            {
                errores.Add($"Política no permitida en '{cluster.Identificador}': '{cluster.Politica}'.");
            }

            if (cluster.NumeroDeDestinos < 2)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene al menos dos destinos.");
            }

            if (!cluster.CombinadoConHealthChecks)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no está combinado con health checks.");
```

```
            }
        }

        return new ResultadoBalanceo(errores.Count == 0, errores);
    }
}

public record ResultadoBalanceo(bool EsValida, IReadOnlyList<string> Errores);
```

Trabaja con la petición concreta sin convertirla en una operación de negocio nueva. Declara la operación, sus parámetros y su resultado; el bloque siguiente contiene su implementación. Obtiene o prepara este valor local para las comprobaciones y operaciones que siguen.

Materializa la secuencia para consultar y recorrer los mismos elementos sin repetir su enumeración. Evalúa esta condición antes de continuar; el bloque asociado trata específicamente ese caso. Acumula un incumplimiento del contrato didáctico; el resultado final indicará que la declaración no es válida.

Recorre los elementos de la colección para aplicarles la misma comprobación. Devuelve el resultado calculado al llamador y termina este recorrido del método. Define un contrato de datos con los campos que el consumidor utilizará para interpretar el resultado.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin clusters.

## PASO 8.4.8 — Añadir los `using` necesarios en `ValidadorBalanceo`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Balanceo;

public class ValidadorBalanceo
{
    private static readonly string[] PoliticasPermitidas = { "RoundRobin", "Random", "LeastRequests",
"PowerOfTwoChoices" };

    public ResultadoBalanceo Validar(IEnumerable<ClusterBalanceo> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
```

```
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (!PoliticasPermitidas.Contains(cluster.Politica))
            {
                errores.Add($"Política no permitida en '{cluster.Identificador}': '{cluster.Politica}'.");
            }

            if (cluster.NumeroDeDestinos < 2)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene al menos dos destinos.");
            }

            if (!cluster.CombinadoConHealthChecks)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no está combinado con health checks.");
            }
        }

        return new ResultadoBalanceo(errores.Count == 0, errores);
    }
}

public record ResultadoBalanceo(bool EsValida, IReadOnlyList<string> Errores);
```

Importa el espacio de nombres requerido por los tipos y métodos utilizados en este archivo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Si un using no se resuelve, comprueba el espacio de nombres y la referencia del proyecto; usa la ubicación mostrada en el archivo completo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.4.9 — Crear la clase `CatalogoClustersBalanceo`

CatalogoClustersBalanceo clasifica alternativas para el análisis. Conviene comprobar que existe más de un destino accesible antes de interpretar un supuesto reparto de carga como un resultado real.

1. Crea `src/AceriaApp.Api/Balanceo/CatalogoClustersBalanceo.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Balanceo/CatalogoClustersBalanceo.cs`. **Operación:** Crear el contenido completo.

```
namespace AceriaApp.Api.Balanceo;

public class CatalogoClustersBalanceo
{
    public static List<ClusterBalanceo> ClustersPorDefecto()
    {
        return new List<ClusterBalanceo>
        {
```

```
            new() { Identificador = "ordenes-cluster", Politica = "RoundRobin", NumeroDeDestinos = 3,
TienePesos = false, CombinadoConHealthChecks = true },
            new() { Identificador = "inventario-cluster", Politica = "PowerOfTwoChoices", NumeroDeDestinos =
2, TienePesos = true, CombinadoConHealthChecks = true },
            new() { Identificador = "clientes-cluster", Politica = "LeastRequests", NumeroDeDestinos = 2,
TienePesos = false, CombinadoConHealthChecks = true }
        };
    }
}
```

Asigna el valor indicado a esta variable, propiedad u opción dentro de la operación actual.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Devolver `List<T>` mutable: expone la implementación interna.

## PASO 8.4.10 — Añadir el `using` necesario en `CatalogoClustersBalanceo`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. Abre `src/AceriaApp.Api/Balanceo/CatalogoClustersBalanceo.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `src/AceriaApp.Api/Balanceo/CatalogoClustersBalanceo.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
namespace AceriaApp.Api.Balanceo;

public class CatalogoClustersBalanceo
{
    public static List<ClusterBalanceo> ClustersPorDefecto()
    {
        return new List<ClusterBalanceo>
        {
            new() { Identificador = "ordenes-cluster", Politica = "RoundRobin", NumeroDeDestinos = 3,
TienePesos = false, CombinadoConHealthChecks = true },
            new() { Identificador = "inventario-cluster", Politica = "PowerOfTwoChoices", NumeroDeDestinos =
2, TienePesos = true, CombinadoConHealthChecks = true },
            new() { Identificador = "clientes-cluster", Politica = "LeastRequests", NumeroDeDestinos = 2,
TienePesos = false, CombinadoConHealthChecks = true }
        };
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

## PASO 8.4.11 — Crear el proyecto de tests del punto 8.4

El punto 8.4 añade un proyecto de tests específico para las clases de balanceo. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea manualmente `tests/AceriaApp.Balanceo.Tests` y su archivo `.csproj`; al usar este bloque no se genera `UnitTest1.cs`.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj`. **Operación:** Crear el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Organiza propiedades o referencias del proyecto MSBuild; conserva la apertura y el cierre de los elementos. Configura .NET 8 y las opciones de compilación de este proyecto. Fija la dependencia y su versión para restaurar el mismo paquete en cada equipo.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 8.3: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

## PASO 8.4.12 — Añadir la referencia del proyecto de tests a la API de Producción

La referencia permite que el proyecto de tests acceda a las clases de `Balanceo` del proyecto de Producción.

1. Abre `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj` y añade la referencia a `AceriaApp.Api` usando el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Referencia el proyecto indicado mediante una ruta relativa a este archivo .csproj.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

## PASO 8.4.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj` y fija FluentAssertions 6.12.2 con el archivo completo siguiente.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

## PASO 8.4.14 — Crear el archivo `ValidadorBalanceoTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los clusters que violan las reglas del balanceo.

1. Crea `tests/AceriaApp.Balanceo.Tests/ValidadorBalanceoTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/ValidadorBalanceoTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.Balanceo;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Balanceo.Tests;

public class ValidadorBalanceoTests
{
    private readonly ValidadorBalanceo _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoClustersBalanceo.ClustersPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinClusters_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ClusterBalanceo>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay clusters"));
    }

    [Fact]
    public void Validar_PoliticaNoPermitida_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "PoliticaInventada", NumeroDeDestinos = 2,
CombinadoConHealthChecks = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Política no permitida"));
    }

    [Fact]
```

```
    public void Validar_MenosDeDosDestinos_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "RoundRobin", NumeroDeDestinos = 1,
CombinadoConHealthChecks = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene al menos dos destinos"));
    }

    [Fact]
    public void Validar_SinHealthChecks_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "RoundRobin", NumeroDeDestinos = 2,
CombinadoConHealthChecks = false }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está combinado con health checks"));
    }
}
```

Declara una prueba individual que xUnit debe descubrir y ejecutar. Comprueba esta condición observable; una diferencia hace fallar la prueba en vez de ocultar el problema.

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.Balanceo;`: las clases no se resuelven.
- La sintaxis FluentAssertions exige su paquete y su using; las aserciones equivalentes de xUnit también son válidas.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

## PASO 8.4.15 — Crear el archivo `CatalogoClustersBalanceoTests.cs`

Los tests verifican que el catálogo por defecto contiene los clusters esperados y que todos cumplen las reglas del balanceo.

1. Crea `tests/AceriaApp.Balanceo.Tests/CatalogoClustersBalanceoTests.cs` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; ejecuta `dotnet build tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release`.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/CatalogoClustersBalanceoTests.cs`. **Operación:** Crear el contenido completo.

```
using AceriaApp.Api.Balanceo;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Balanceo.Tests;

public class CatalogoClustersBalanceoTests
{
```

```
    [Fact]
    public void ClustersPorDefecto_ContieneTresClusters()
    {
        var clusters = CatalogoClustersBalanceo.ClustersPorDefecto();

        clusters.Should().HaveCount(3);
    }

    [Fact]
    public void ClustersPorDefecto_TodosTienenAlMenosDosDestinos()
    {
        var clusters = CatalogoClustersBalanceo.ClustersPorDefecto();

        clusters.Should().OnlyContain(c => c.NumeroDeDestinos >= 2);
    }

    [Fact]
    public void ClustersPorDefecto_TodosCombinadosConHealthChecks()
    {
        var clusters = CatalogoClustersBalanceo.ClustersPorDefecto();

        clusters.Should().OnlyContain(c => c.CombinadoConHealthChecks);
    }

    [Fact]
    public void ClustersPorDefecto_ContieneOrdenesInventarioYClientes()
    {
        var clusters = CatalogoClustersBalanceo.ClustersPorDefecto();

        clusters.Should().Contain(c => c.Identificador == "ordenes-cluster");
        clusters.Should().Contain(c => c.Identificador == "inventario-cluster");
        clusters.Should().Contain(c => c.Identificador == "clientes-cluster");
    }

    [Fact]
    public void ClustersPorDefecto_ContieneAlgunoConPesos()
    {
        var clusters = CatalogoClustersBalanceo.ClustersPorDefecto();

        clusters.Should().Contain(c => c.TienePesos);
    }
}
```

**Verificación concreta.** Los archivos indicados contienen todos los bloques y la compilación del proyecto termina sin errores.

**Diagnóstico si falla.**

- Olvidar el `using AceriaApp.Api.Balanceo;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Contain` y `OnlyContain` no se resuelven.
- No verificar la combinación con health checks: la regla no se comprueba.

## PASO 8.4.16 — Compilar la solución completa

Para terminar hay que compilar, arrancar el gateway, comprobar las rutas anteriores y observar la distribución entre destinos equivalentes. Las políticas declaradas no sustituyen la evidencia de peticiones y respuestas.

1. Escribe todos los archivos de ampliación funcional mostrados a continuación antes de compilar; cuando se indica reemplazar, sustituye el contenido de la copia actual.
2. Desde la carpeta actual ejecuta `dotnet sln AceriaApp.sln add (Get-ChildItem src,tests -Recurse -Filter *.csproj | ForEach-Object FullName)` en PowerShell 7.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release -m:1`.
4. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 39 proyectos; el compilador debe terminar con cero errores.

**Archivo:** `src/AceriaApp.Gateway/appsettings.json`. **Operación:** Reemplazar el contenido completo.

```
{
  "Urls": "http://localhost:5100",
  "Observabilidad": {
    "NivelMinimo": "Information"
  },
  "ReverseProxy": {
    "Routes": {
      "ordenes": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/ordenes/{**resto}"
        }
      },
      "produccion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/produccion/{**resto}"
        }
      },
      "comunicacion": {
        "ClusterId": "produccion-cluster",
        "Match": {
          "Path": "/api/v1/comunicacion/{**resto}"
        }
      },
      "materiales": {
        "ClusterId": "inventario-cluster",
        "Match": {
          "Path": "/api/v1/materiales/{**resto}",
          "Methods": [
            "GET",
            "HEAD"
          ]
        }
      },
      "planificaciones": {
        "ClusterId": "inventario-cluster",
        "Match": {
          "Path": "/api/v1/planificaciones/{**resto}",
          "Methods": [
            "GET",
            "HEAD"
          ]
        }
      },
      "inventario-materiales": {
        "ClusterId": "inventario-cluster",
        "Order": -10,
        "Match": {
          "Path": "/inventario/api/v1/materiales/{**resto}",
          "Methods": [
            "GET",
            "HEAD"
          ]
        },
        "Transforms": [
          {
            "PathRemovePrefix": "/inventario"
          },
          {
            "RequestHeader": "X-Canal",
            "Set": "Gateway"
          },
          {
            "QueryValueParameter": "canal",
            "Set": "gateway"
          }
        ]
      },
      "inventario-planificaciones": {
        "ClusterId": "inventario-cluster",
        "Order": -10,
        "Match": {
          "Path": "/inventario/api/v1/planificaciones/{**resto}",
          "Methods": [
```

```
            "GET",
            "HEAD"
          ]
        },
        "Transforms": [
          {
            "PathRemovePrefix": "/inventario"
          },
          {
            "RequestHeader": "X-Canal",
            "Set": "Gateway"
          },
          {
            "QueryValueParameter": "canal",
            "Set": "gateway"
          }
        ]
      }
    },
    "Clusters": {
      "produccion-cluster": {
        "Destinations": {
          "produccion": {
            "Address": "http://localhost:5105/"
          }
        }
      },
      "inventario-cluster": {
        "Destinations": {
          "inventario-1": {
            "Address": "http://localhost:5106/"
          },
          "inventario-2": {
            "Address": "http://localhost:5107/"
          }
        },
        "LoadBalancingPolicy": "RoundRobin",
        "HealthCheck": {
          "AvailableDestinationsPolicy": "HealthyAndUnknown",
          "Active": {
            "Enabled": true,
            "Interval": "00:00:01",
            "Timeout": "00:00:01",
            "Policy": "ConsecutiveFailures",
            "Path": "/health/ready"
          }
        },
        "Metadata": {
          "ConsecutiveFailuresHealthPolicy.Threshold": "2"
        }
      }
    }
  }
}
```

Abre o cierra esta estructura JSON; las comas y delimitadores mantienen válido el documento. Establece la dirección de escucha de este proceso; conserva el nombre y el tipo json. Asigna este valor a la opción observabilidad; conserva el nombre y el tipo json.

Asigna este valor a la opción nivelminimo; conserva el nombre y el tipo json. Reúne la configuración que lee addreverseproxy; conserva el nombre y el tipo json. Declara las rutas públicas que yarp puede seleccionar; conserva el nombre y el tipo json.

Asigna este valor a la opción ordenes; conserva el nombre y el tipo json. Vincula la ruta con un cluster existente; conserva el nombre y el tipo json. Asigna este valor a la opción match; conserva el nombre y el tipo json.

Define el patrón público o la ruta del sondeo según su sección; conserva el nombre y el tipo json. Asigna este valor a la opción produccion; conserva el nombre y el tipo json. Asigna este valor a la opción comunicacion; conserva el nombre y el tipo json.

Asigna este valor a la opción materiales; conserva el nombre y el tipo json. Restringe los métodos que esta ruta admite; conserva el nombre y el tipo json. Asigna este valor a la opción planificaciones; conserva el nombre y el tipo json.

Asigna este valor a la opción inventario-materiales; conserva el nombre y el tipo json. Asigna prioridad explícita; los valores menores se evalúan antes; conserva el nombre y el tipo json. Declara cambios ordenados sobre la petición reenviada; conserva el nombre y el tipo json.

Elimina únicamente el prefijo configurado; conserva el nombre y el tipo json. Selecciona la cabecera que se transforma; conserva el nombre y el tipo json. Fija un valor literal; no ejecuta código ni genera identificadores; conserva el nombre y el tipo json.

Selecciona un parámetro de consulta para el valor literal configurado; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-planificaciones; conserva el nombre y el tipo json. Agrupa destinos que ofrecen el mismo contrato http; conserva el nombre y el tipo json.

Asigna este valor a la opción produccion-cluster; conserva el nombre y el tipo json. Enumera instancias del servicio, no servicios con contratos diferentes; conserva el nombre y el tipo json. Establece la url interna del destino; conserva el nombre y el tipo json.

Asigna este valor a la opción inventario-cluster; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-1; conserva el nombre y el tipo json. Asigna este valor a la opción inventario-2; conserva el nombre y el tipo json.

Elige el algoritmo con el que se reparte entre destinos disponibles; conserva el nombre y el tipo json. Configura la evaluación de disponibilidad de destinos; conserva el nombre y el tipo json. Asigna este valor a la opción availabledestinationspolicy; conserva el nombre y el tipo json.

Habilita sondeos independientes de las peticiones del cliente; conserva el nombre y el tipo json. Activa o desactiva la función de su sección; conserva el nombre y el tipo json. Fija el intervalo entre sondeos de disponibilidad; conserva el nombre y el tipo json.

Limita la espera del sondeo; conserva el nombre y el tipo json. Selecciona la política que interpreta los resultados de salud; conserva el nombre y el tipo json. Asigna este valor a la opción metadata; conserva el nombre y el tipo json.

Asigna este valor a la opción consecutivefailureshealthpolicy.threshold; conserva el nombre y el tipo json.

**Archivo:** `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Observabilidad;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddLoggingAceria("Inventario", builder.Configuration);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
```

```
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        builder.Services.AddSaludAceria(builder.Configuration, inventario: true);
        builder.Services.AddErroresAceria();
        var app = builder.Build();
        app.UseLoggingAceria();
        var instancia = builder.Configuration["Inventario:Instancia"] ?? "inventario-1";
        app.Use(async (contexto, siguiente) =>
        {
            contexto.Response.Headers["X-Instancia"] = instancia;
            await siguiente();
        });
        app.UseErroresAceria();
        app.MapDiagnosticoAceria();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapMethods("/api/v1/materiales/{material}", new[] { "GET", "HEAD" }, async (string material,
InventarioDbContext db, CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapMethods("/api/v1/planificaciones/{ordenId:guid}", new[] { "GET", "HEAD" }, async (Guid
ordenId, InventarioDbContext db, CancellationToken ct) =>
        {
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
valor.CantidadTotal });
        });
        app.MapSaludAceria();
        await app.RunAsync();
    }
}
```

Completa esta expresión o invocación dentro de InventarioProgram.cs; se conserva íntegra para reproducir el comportamiento. Lee o escribe la cabecera HTTP indicada; su valor debe coincidir con el contrato de transformación. Espera la operación asíncrona sin bloquear el hilo; un fallo se propaga para poder detectarlo.

Delimita la vida del recurso y garantiza su disposición al salir del ámbito, también si ocurre una excepción; await using permite la disposición asíncrona.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/GatewayBalanceoFuncionalTests.cs`. **Operación:** Crear el contenido completo.

```
using System.Net;
using AceriaApp.Pruebas.Gateway;
using Xunit;
using Microsoft.Extensions.DependencyInjection;
using Yarp.ReverseProxy;
using Yarp.ReverseProxy.Model;

namespace AceriaApp.Balanceo.Tests;

public sealed class GatewayBalanceoFuncionalTests
{
    // Detecta pérdida del middleware de balanceo o sustitución accidental de RoundRobin.
    [Fact]
```

```
    public async Task RoundRobin_alterna_los_dos_destinos_sanos()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(segundaInstancia: true);
        var instancias = await LeerInstanciasAsync(prueba, 8);
        Assert.Equal(4, instancias.Count(x => x == "inventario-1"));
        Assert.Equal(4, instancias.Count(x => x == "inventario-2"));
        for (var i = 1; i < instancias.Count; i++) Assert.NotEqual(instancias[i - 1], instancias[i]);
        await GatewayPruebas.EsperarAsync(() => Task.FromResult(prueba.Inventario1.Sondeos > 0 &&
prueba.Inventario2!.Sondeos > 0), "No se observaron sondeos activos en ambos servidores.");
    }

    [Fact]
    public async Task Tras_caida_fisica_y_recuperacion_la_instancia_vuelve_al_reparto()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(segundaInstancia: true);
        Assert.Contains("inventario-2", await LeerInstanciasAsync(prueba, 4));
        await prueba.Inventario2!.DetenerAsync();
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Unhealthy);
        Assert.All(await LeerInstanciasAsync(prueba, 8), x => Assert.Equal("inventario-1", x));
        await prueba.ReiniciarInventario2Async();
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Healthy);
        var recuperadas = await LeerInstanciasAsync(prueba, 8);
        Assert.Equal(4, recuperadas.Count(x => x == "inventario-2"));
    }

    [Fact]
    public async Task Todos_los_destinos_detenidos_dan_503_y_un_destino_recuperado_restaura_servicio()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(segundaInstancia: true);
        await prueba.Inventario1.DetenerAsync();
        await prueba.Inventario2!.DetenerAsync();
        await EsperarSaludAsync(prueba, "inventario-1", DestinationHealth.Unhealthy);
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Unhealthy);
        for (var i = 0; i < 4; i++)
        {
            using var respuesta = await prueba.Cliente.GetAsync("/api/v1/materiales/Acero");
            Assert.Equal(HttpStatusCode.ServiceUnavailable, respuesta.StatusCode);
        }
        await prueba.ReiniciarInventario2Async();
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Healthy);
        Assert.All(await LeerInstanciasAsync(prueba, 4), x => Assert.Equal("inventario-2", x));
    }

    [Fact]
    public async Task Readiness_no_satisfactoria_excluye_un_servidor_que_sigue_escuchando()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(segundaInstancia: true);
        prueba.Inventario2!.Listo = false;
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Unhealthy);
        Assert.All(await LeerInstanciasAsync(prueba, 8), x => Assert.Equal("inventario-1", x));
        using (var directo = new HttpClient())
        using (var respuesta = await directo.GetAsync(prueba.Inventario2.Direccion + "/api/v1/materiales/
Acero"))
            Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        prueba.Inventario2.Listo = true;
        await EsperarSaludAsync(prueba, "inventario-2", DestinationHealth.Healthy);
        Assert.Contains("inventario-2", await LeerInstanciasAsync(prueba, 4));
    }

    [Fact]
    public async Task Una_escritura_abortada_en_el_backend_no_se_reintenta()
    {
        await using var prueba = await GatewayPruebas.CrearAsync(segundaInstancia: true);
        using var respuesta = await prueba.Cliente.PostAsync("/api/v1/ordenes?abortar=true", new
StringContent("{\"cantidad\":1}"));
        Assert.Equal(HttpStatusCode.BadGateway, respuesta.StatusCode);
        Assert.Equal(1, prueba.Produccion.Escrituras);
        Assert.Equal(0, prueba.Inventario1.Escrituras + prueba.Inventario2!.Escrituras);
    }

    private static async Task<List<string>> LeerInstanciasAsync(GatewayPruebas prueba, int cantidad)
    {
        var instancias = new List<string>();
        for (var i = 0; i < cantidad; i++)
        {
            using var respuesta = await prueba.Cliente.GetAsync("/api/v1/materiales/Acero");
```

```
            instancias.Add(respuesta.IsSuccessStatusCode ? Assert.Single(respuesta.Headers.GetValues("X-
Instancia")) : $"HTTP-{(int)respuesta.StatusCode}");
        }
        return instancias;
    }

    private static Task EsperarSaludAsync(GatewayPruebas prueba, string destinoId, DestinationHealth salud)
=>
        GatewayPruebas.EsperarAsync(() =>
        {
            var estado = prueba.Proxy.Services.GetRequiredService<IProxyStateLookup>();
            var coincide = estado.TryGetCluster("inventario-cluster", out var cluster) &&
                cluster.Destinations.TryGetValue(destinoId, out var destino) && destino.Health.Active ==
salud;
            return Task.FromResult(coincide);
        }, $"El destino {destinoId} no alcanzó el estado activo {salud}.");
}
```

Documenta la intención de la instrucción próxima sin alterar su ejecución. Completa esta expresión o invocación dentro de GatewayBalanceoFuncionalTests.cs; se conserva íntegra para reproducir el comportamiento.

**Archivo:** `tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Gateway/AceriaApp.Gateway.csproj" />
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <Compile Include="../Compartido/GatewayPruebas.cs" Link="Compartido/GatewayPruebas.cs" />
    <Content Include="../../src/AceriaApp.Gateway/appsettings.json" Link="gateway.appsettings.json"
CopyToOutputDirectory="PreserveNewest" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Hace disponibles las API del framework ASP.NET Core para los servidores de prueba. Incluye el soporte de pruebas compartido en la compilación de este proyecto.

**Verificación concreta.** Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 39 proyectos; el compilador debe terminar con cero errores.

**Diagnóstico si falla.**

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

## PASO 8.4.17 — Ejecutar todos los tests del punto 8.4

Una prueba de catálogo acredita su declaración. Las pruebas funcionales utilizan el proxy y servidores HTTP reales, pero la persistencia y el broker se comprueban con la suite y el recorrido completo.

1. Ejecuta `dotnet test tests/AceriaApp.Balanceo.Tests/AceriaApp.Balanceo.Tests.csproj -c Release --no-build` en la terminal.
2. Comprueba que se ejecutan 15 pruebas: 10 de catálogo y 5 funcionales.
3. Si hay un fallo, identifica su nombre y revisa la regla o la respuesta esperada antes de modificar código.
4. Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Verificación concreta.** Confirma cero errores y cero pruebas omitidas; el reto todavía no forma parte de esta etapa.

**Diagnóstico si falla.**

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

## PASO 8.4.18 — Ejecutar todos los tests de la solución

La suite completa comprueba que el nuevo punto no rompe la persistencia, la mensajería, el dominio, la observabilidad ni la resiliencia desarrollados antes.

1. Configura `ACERIA_TEST_SQLSERVER` para tu SQL Server Express/LocalDB y `ACERIA_TEST_RABBITMQ` para el broker; sigue los requisitos del inicio del manual.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build -m:1`.
3. Comprueba que pasan las 491 pruebas acumuladas, incluidas las heredadas de M07, sin omisiones.
4. Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Verificación concreta.** Si falla la conexión, corrige el servicio, los permisos o la URI. Una prueba no ejecutada no se contabiliza como aprobada.

**Diagnóstico si falla.**

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

## PASO 8.4.19 — Crear el README del punto 8.4

El README del punto 8.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 8.3.

1. Abre `README.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 8.4 — Balanceo de carga básico

## Objetivo

Distribuir solicitudes entre dos instancias de Inventario, detectar caída y comprobar recuperación.

## Estado del proyecto

Etapa acumulativa con 39 proyectos, 26 proyectos de tests y 50 documentos. El cierre exige 491 pruebas
aprobadas sin omisiones; los resultados se registran en los informes del módulo.

## Cómo ejecutarlo

Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test
AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite
completa.

El recorrido funcional inicia Gateway en 5100, Producción en 5105 e Inventario en 5106. La etapa 8.4 añade
Inventario en 5107.

## Qué se ha añadido respecto al punto anterior

- Modelo ClusterBalanceo, validador ValidadorBalanceo y catálogo CatalogoClustersBalanceo.
- 10 pruebas de catálogo y 5 pruebas funcionales nuevas.
- Documento temático y ADR-034.
- Reto optativo en copia separada, con una prueba adicional.

## Documentación

- `docs/balanceo-carga.md`.
- `docs/ADR-034-balanceo-carga.md`.
- Las 48 fichas y decisiones anteriores permanecen idénticas.
```

`Desde esta carpeta: `dotnet restore AceriaApp.sln`, `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release`. Configura SQL Server/LocalDB y RabbitMQ según el manual antes de ejecutar la suite completa`. Conserva esta explicación o instrucción operativa dentro del documento de la etapa. `- `docs/balanceo-carga.md``. Registra una condición o una decisión que podrá contrastarse al revisar la etapa. `- `docs/ADR-034-balanceo-carga.md``. Registra una condición o una decisión que podrá contrastarse al revisar la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar el README del punto 8.3: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 8.3: rompe la separación.

## PASO 8.4.20 — Crear el ADR-034 del punto 8.4

El ADR-034 documenta la decisión sobre el balanceo y cierra el curso. Es un artefacto nuevo del punto 8.4 que no modifica los ADR existentes.

1. Crea `docs/ADR-034-balanceo-carga.md` dentro de la carpeta actual.
2. Copia el contenido completo de cada bloque siguiente; respeta rutas, espacios de nombres y versiones.
3. Guarda en UTF-8 con Ctrl+S y compara el archivo con su bloque para detectar líneas omitidas.
4. Comprueba que los archivos indicados existen; revisa que los archivos heredados de otras carpetas permanecen intactos.

**Archivo:** `docs/ADR-034-balanceo-carga.md`. **Operación:** Crear el contenido completo.

```
# ADR-034: Balanceo de carga y cierre del curso

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado YARP como gateway. Se quiere comprobar el reparto y la recuperación entre
dos instancias de Inventario, manteniendo una instancia de Producción. Este alcance demuestra el mecanismo de
balanceo del servicio elegido, sin declarar alta disponibilidad de todos los componentes.

## Decisión
Se fija `LoadBalancingPolicy: RoundRobin` en inventario-cluster, con destinos 5106 y 5107. produccion-cluster
conserva el destino 5105 y el gateway escucha en 5100. YARP también ofrece Random, LeastRequests y
PowerOfTwoChoices, pero no se dejan implícitos en el ejercicio. No se configura una propiedad nativa Weight
ni una política ponderada: los metadatos solo producirían ponderación si una extensión adicional los
interpretara. Inventario utiliza sondeos activos a `/health/ready` con intervalo y timeout de un segundo,
ConsecutiveFailures y umbral dos. `HealthyAndUnknown` excluye destinos Unhealthy y devuelve 503 si ninguno
permanece disponible; permite reincorporar un destino recuperado sin reiniciar el gateway. La validación
incluye reparto observable con X-Instancia, caída parcial, caída total y recuperación real. El cierre exige
39 proyectos, 26 proyectos de pruebas, 50 documentos y 491 pruebas esperadas en 8.4 sin omisiones, con
evidencia Linux, Windows y PDF de la misma revisión. El recorrido conserva ocho módulos y 34 puntos; esos
objetivos no constituyen un PASS antes de ejecutar CI y revisar las evidencias.

## Alternativas consideradas
- Balanceo en el orquestador: descartado por simplicidad del proyecto.
- Balanceo sin health checks: descartado por riesgo de enrutar a instancias no disponibles.
- Política ponderada personalizada: se pospone porque el objetivo es RoundRobin entre dos destinos
equivalentes; el indicador TienePesos del catálogo no instala esa política.

## Consecuencias
- Positivas: escalado horizontal, distribución de carga, tolerancia a fallos de instancia.
- Negativas: requiere mantener los dos destinos de Inventario y su política de salud; una petición asignada
durante la detección puede fallar. Producción y Gateway siguen siendo instancias únicas en el laboratorio.
- Mitigación: la validación automática y las buenas prácticas documentadas.

## Alcance comprobable de esta etapa

TienePesos expresa una intención de diseño, sin extensión ponderada implementada. Los catálogos no activan el
balanceador ni los checks. El estado Aceptado de este ADR expresa la decisión arquitectónica; el resultado de
ejecución se registra en los informes y sus hashes.

El directorio anterior se conserva intacto. Las copias del punto actual incluyen los cambios completos
indicados en el manual.
```

`Se fija `LoadBalancingPolicy: RoundRobin` en inventario-cluster, con destinos 5106 y 5107. produccion-cluster conserva el destino 5105 y el gateway escucha en 5100. YARP también ofrece Random, LeastRequests y PowerOfTwoChoices, pero no se dejan implícitos en el ejercicio. No se configura una propiedad nativa Weight ni una política ponderada: los metadatos solo producirían ponderación si una extensión adicional los interpretara. Inventario utiliza sondeos activos a `/health/ready` con intervalo y timeout de un segundo, ConsecutiveFailures y umbral dos. `HealthyAndUnknown` excluye destinos Unhealthy y devuelve 503 si ninguno permanece disponible; permite reincorporar un destino recuperado sin reiniciar el gateway. La validación incluye reparto observable con X-Instancia, caída parcial, caída total y recuperación real. El cierre exige 39 proyectos, 26 proyectos de pruebas, 50 documentos y 491 pruebas esperadas en 8.4 sin omisiones, con evidencia Linux, Windows y PDF de la misma revisión. El recorrido conserva ocho módulos y 34 puntos; esos objetivos no constituyen un PASS antes de ejecutar CI y revisar las evidencias.` Conserva esta explicación o instrucción operativa dentro del documento de la etapa.

**Verificación concreta.** El documento existe, su vista previa mantiene los encabezados y los documentos heredados conservan sus hashes.

**Diagnóstico si falla.**

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-034`: rompe la convención.

## PASO 8.4.21 — Verificar que no se ha modificado ningún archivo del punto 8.3

La progresión modifica copias independientes. Los nuevos archivos o configuraciones de una etapa no deben alterar un punto ya entregado.

1. Abre las carpetas `docs` de la etapa actual y de `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones`.
2. Compara con Get-FileHash los archivos que existían en la etapa anterior; cada hash SHA256 debe coincidir.
3. Comprueba también que los archivos de tests heredados conservan su contenido; solo se incorporan los tests del punto.
4. Verifica que `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.4-balanceo-carga`.

**Verificación concreta.** Verifica que `M08_API_GATEWAY/PROYECTO/8.3-enrutamiento-peticiones` no presenta modificaciones. Los cambios completos del punto pertenecen exclusivamente a `8.4-balanceo-carga`.

**Diagnóstico si falla.**

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

## PASO 8.4.22 — Verificar la estructura del punto 8.4 y la estructura completa del curso

El inventario convierte la comprobación de estructura en una condición observable y detecta omisiones que una compilación aislada podría pasar por alto.

1. Ejecuta `Get-ChildItem -Recurse -File` y revisa las carpetas src, tests y docs.
2. Verifica 39 proyectos, 26 proyectos de pruebas y 50 documentos.
3. Comprueba la presencia de las tres clases de `Balanceo`, sus tests, el documento temático y ADR-034.
4. Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable. El curso conserva ocho módulos y 34 puntos acumulativos.

**Verificación concreta.** Comprueba que no has introducido carpetas temporales, labs ni metaprompts dentro del entregable. El curso conserva ocho módulos y 34 puntos acumulativos.

**Diagnóstico si falla.**

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 8.3: viola la regla de no modificar.
- Omitir la carpeta `Balanceo`: los tests no encuentran las clases.

## PASO 8.4.23 — Verificar el `.gitignore` del punto 8.4

Las exclusiones conservan un repositorio reproducible sin binarios temporales ni datos específicos de Visual Studio.

1. Abre `.gitignore` de la etapa actual.
2. Comprueba que contiene bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
3. Compara su contenido con el archivo heredado y conserva sus exclusiones.
4. Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Verificación concreta.** Ejecuta `git status --short` desde la raíz del curso y verifica que los artefactos de compilación no aparecen como entregables.

**Diagnóstico si falla.**

- Modificar el `.gitignore` copiado del punto 8.3: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

### PASO 8.4.24 — Hacer el commit del punto 8.4 y verificar el estado final del curso

Un commit conserva el estado comprobado. El número de commits no tiene que coincidir con el de puntos: las correcciones y revisiones también forman parte del historial.

1. Desde la raíz del repositorio ejecuta `git status --short` y revisa todos los cambios de la etapa.
2. Añade exclusivamente los archivos docentes y del proyecto con `git add M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`.
3. Ejecuta `git commit -m "Punto 8.4: Balanceo de carga básico"` después de comprobar las pruebas.
4. Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Verificación concreta.** Ejecuta `git log --oneline -5` y `git status --short`; confirma el hito y revisa cualquier archivo pendiente.

**Diagnóstico si falla.**

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

## Reto de ampliación (optativo)

añadir un método `Resumen` a `ValidadorBalanceo` que devuelva el número total de clusters, el número total de destinos, el número de clusters con pesos y el número de clusters combinados con health checks, y un test que verifique el cálculo con el catálogo por defecto.

Crea una copia de `8.4-balanceo-carga` llamada `reto-8-4` fuera de las cuatro carpetas obligatorias. La copia anterior permanece intacta. Aplica las tres sustituciones completas siguientes en orden.

### Ampliación 1: añadir el contrato de resumen

**Archivo:** `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Balanceo;

public class ValidadorBalanceo
{
    private static readonly string[] PoliticasPermitidas = { "RoundRobin", "Random", "LeastRequests",
"PowerOfTwoChoices" };

    public ResultadoBalanceo Validar(IEnumerable<ClusterBalanceo> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();
```

```
        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (!PoliticasPermitidas.Contains(cluster.Politica))
            {
                errores.Add($"Política no permitida en '{cluster.Identificador}': '{cluster.Politica}'.");
            }

            if (cluster.NumeroDeDestinos < 2)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene al menos dos destinos.");
            }

            if (!cluster.CombinadoConHealthChecks)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no está combinado con health checks.");
            }
        }

        return new ResultadoBalanceo(errores.Count == 0, errores);
    }
}

public record ResultadoBalanceo(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenBalanceo(int Total, int TotalDestinos, int ConPesos, int ConHealthChecks);
```

## Ampliación 2: implementar el cálculo del resumen

**Archivo:** `src/AceriaApp.Api/Balanceo/ValidadorBalanceo.cs`. **Operación:** Reemplazar el contenido completo.

```
using System.Collections.Generic;
using System.Linq;
namespace AceriaApp.Api.Balanceo;

public class ValidadorBalanceo
{
    private static readonly string[] PoliticasPermitidas = { "RoundRobin", "Random", "LeastRequests",
"PowerOfTwoChoices" };

    public ResultadoBalanceo Validar(IEnumerable<ClusterBalanceo> clusters)
    {
        var errores = new List<string>();
        var lista = clusters.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay clusters declarados.");
        }

        foreach (var cluster in lista)
        {
            if (string.IsNullOrWhiteSpace(cluster.Identificador))
            {
                errores.Add("Un cluster no declara el identificador.");
            }

            if (!PoliticasPermitidas.Contains(cluster.Politica))
            {
                errores.Add($"Política no permitida en '{cluster.Identificador}': '{cluster.Politica}'.");
            }

            if (cluster.NumeroDeDestinos < 2)
```

```
            {
                errores.Add($"El cluster '{cluster.Identificador}' no tiene al menos dos destinos.");
            }

            if (!cluster.CombinadoConHealthChecks)
            {
                errores.Add($"El cluster '{cluster.Identificador}' no está combinado con health checks.");
            }
        }

        return new ResultadoBalanceo(errores.Count == 0, errores);
    }

    public static ResumenBalanceo Resumen(IEnumerable<ClusterBalanceo> clusters)
    {
        var lista = clusters.ToList();
        var totalDestinos = lista.Sum(c => c.NumeroDeDestinos);
        var conPesos = lista.Count(c => c.TienePesos);
        var conHealthChecks = lista.Count(c => c.CombinadoConHealthChecks);
        return new ResumenBalanceo(lista.Count, totalDestinos, conPesos, conHealthChecks);
    }
}

public record ResultadoBalanceo(bool EsValida, IReadOnlyList<string> Errores);

public record ResumenBalanceo(int Total, int TotalDestinos, int ConPesos, int ConHealthChecks);
```

## Ampliación 3: verificar el resultado con el catálogo

**Archivo:** `tests/AceriaApp.Balanceo.Tests/ValidadorBalanceoTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Balanceo;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Balanceo.Tests;

public class ValidadorBalanceoTests
{
    private readonly ValidadorBalanceo _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoClustersBalanceo.ClustersPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinClusters_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ClusterBalanceo>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay clusters"));
    }

    [Fact]
    public void Validar_PoliticaNoPermitida_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "PoliticaInventada", NumeroDeDestinos = 2,
CombinadoConHealthChecks = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Política no permitida"));
```

```
    }

    [Fact]
    public void Validar_MenosDeDosDestinos_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "RoundRobin", NumeroDeDestinos = 1,
CombinadoConHealthChecks = true }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene al menos dos destinos"));
    }

    [Fact]
    public void Validar_SinHealthChecks_DevuelveError()
    {
        var clusters = new List<ClusterBalanceo>
        {
            new() { Identificador = "X", Politica = "RoundRobin", NumeroDeDestinos = 2,
CombinadoConHealthChecks = false }
        };

        var resultado = _validador.Validar(clusters);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está combinado con health checks"));
    }

    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorBalanceo.Resumen(CatalogoClustersBalanceo.ClustersPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.TotalDestinos.Should().Be(7);
        resumen.ConPesos.Should().Be(1);
        resumen.ConHealthChecks.Should().Be(3);
    }
}
```

**Ejecución y resultado esperado.** En la copia optativa ejecuta `dotnet test AceriaApp.sln -c Release -m:1`. Deben pasar 492 pruebas; el proyecto `AceriaApp.Balanceo.Tests` debe ejecutar 16. Verifica que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que los cuatro puntos obligatorios no cambiaron. Registra el reto en un commit independiente cuando lo hayas comprobado.

## Resultado esperado del punto

- Solución autónoma con 39 proyectos y 26 proyectos de pruebas.
- 50 documentos, incluidos el tema del punto y ADR-034.
- 10 pruebas nuevas de catálogo y 5 funcionales; 491 acumuladas sin omisiones.
- Etapa anterior, documentación y tests heredados conservados; el reto sigue separado.

## Conclusión y continuidad

El punto completa el balanceo básico y cierra el recorrido del curso con un gateway verificable delante de los servicios. La arquitectura hexagonal, el dominio, los patrones, HTTP, eventos, observabilidad y resiliencia permanecen presentes en la solución acumulativa.

# Recorridos funcionales del gateway

Los recorridos se ejecutan dentro de la carpeta de la etapa indicada. El catálogo es una actividad de diseño; estos comandos ejercitan los procesos HTTP. No confundir una salida SIN_TESTS de reconstrucción con una prueba de ejecución.

## Preparar Producción e Inventario

Abre terminales PowerShell separadas en la raíz de la etapa y configura `$env:ASPNETCORE_ENVIRONMENT = "Development"`. Para una práctica local usa SQL Server Express/LocalDB con bases exclusivas del módulo; no reutilices una base de trabajo real. Define las cuatro conexiones en cada terminal:

```
$env:ConnectionStrings__AceriaApp = "Server=(localdb)\MSSQLLocalDB;Database=AceriaM08Api;Integrated
Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaHexagonal = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Hexagonal;Integrated Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaProduccion = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Produccion;Integrated Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaInventario = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Inventario;Integrated Security=True;TrustServerCertificate=True"
$env:Comunicacion__WorkersActivos = "false"
$env:Comunicacion__InventarioUrl = "http://localhost:5106/"
```

Configura una conexión SQL Server de esta terminal; el nombre después de __ coincide con el contexto que la aplicación lee. Establece la opción de entorno para el proceso que se iniciará desde esta terminal.

Estas conexiones usan cuatro bases para los modelos acumulados del curso. EnsureCreated es una ayuda de Development, no una política de migraciones para producción. Las dos instancias de Inventario deben compartir AceriaM08Inventario para observar el mismo estado persistente. Primero inicia una instancia y espera a que complete la inicialización de tablas; después inicia la segunda.

En la terminal de Inventario 1:

```
$env:ASPNETCORE_URLS = "http://localhost:5106"
$env:Inventario__Instancia = "inventario-1"
dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile
```

Compila e inicia el proyecto indicado sin sobrescribir el puerto mediante launchSettings.

En la terminal de Producción:

```
$env:ASPNETCORE_URLS = "http://localhost:5105"
dotnet run --project src/AceriaApp.Api --no-launch-profile
```

Comprueba `Invoke-RestMethod http://localhost:5105/health/ready` y `Invoke-RestMethod http://localhost:5106/health/ready`: ambos deben informar Healthy. Una respuesta Unhealthy requiere revisar conexiones y servicios antes de continuar.

## Recorrido 8.2: publicar recursos y conservar HTTP

En otra terminal de la etapa 8.2, inicia el proceso de gateway:

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
$env:ASPNETCORE_URLS = "http://localhost:5100"
dotnet run --project src/AceriaApp.Gateway --no-launch-profile
```

Consulta `Invoke-WebRequest http://localhost:5100/api/v1/materiales/Acero`: el cuerpo debe devolver material Acero y stock 300. Envía una cabecera `X-Correlation-ID` válida y comprueba la misma cabecera en la respuesta. Consulta un material inexistente: el gateway debe conservar el 404 del servicio. Una ruta no publicada debe devolver 404 y no convertirse en un acceso genérico al backend.

Para crear una orden del contrato REST acumulado usa:

```
$body = @{ cliente="Cliente M08"; material="Acero"; cantidad=5 } | ConvertTo-Json
$response = Invoke-WebRequest http://localhost:5100/api/v1/ordenes -Method Post -ContentType application/json
-Body $body
$response.StatusCode
$response.Headers.Location
```

Serializa el cuerpo con los nombres de propiedad del contrato REST de órdenes. Envía la petición HTTP y conserva estado y cabeceras para comprobar el contrato del gateway. Muestra el código de estado devuelto por el servicio a través del proxy.

Muestra la referencia al recurso creado; debe poder consultarse desde la entrada pública.

El resultado esperado es 201. Sigue Location a través del gateway y verifica que el recurso coincide con el cuerpo devuelto. No sustituyas una respuesta de validación 400/422 por un 200. Si el proceso de Producción no está disponible, la respuesta del proxy debe ser un error de gateway, nunca una orden creada ficticia.

## Recorrido 8.3: alias y transformaciones

Detén los procesos anteriores y repite su arranque desde 8.3 para ejecutar el código de esa etapa. Solicita `http://localhost:5100/inventario/api/v1/materiales/Acero`: el prefijo /inventario se elimina antes de llamar al servicio. YARP establece X-Canal=Gateway y el parámetro canal=gateway en este alias; el backend conserva el contrato de materiales. Las pruebas funcionales del punto utilizan un servidor que observa la petición para comprobar cabecera, query y ruta recibidas.

El alias solo publica los recursos de Inventario previstos. Una petición a `/inventario/diagnostico/fallo` o `/inventario/health/ready` no debe atravesar el gateway. Los métodos mutantes no se habilitan para rutas de consulta de Inventario. Una restricción de método puede producir 405; una ruta no registrada produce 404. El contrato y las pruebas distinguen ambos casos.

## Recorrido 8.4: reparto, caída y recuperación

Inicia los procesos desde 8.4. Arranca Inventario 1 antes de Inventario 2, que usa el mismo modelo y la misma base. En la terminal de la segunda instancia:

```
$env:ASPNETCORE_URLS = "http://localhost:5107"
$env:Inventario__Instancia = "inventario-2"
dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile
```

Solicita diez veces el material Acero mediante el gateway y registra X-Instancia:

```
1..10 | ForEach-Object {
    (Invoke-WebRequest http://localhost:5100/api/v1/materiales/Acero).Headers['X-Instancia']
}
```

Repite la solicitud para observar el reparto entre destinos disponibles.

Con ambas instancias sanas deben observarse inventario-1 e inventario-2. Detén Inventario 2 con Ctrl+C. Durante la detección todavía puede fallar una petición ya asignada; tras los sondeos fallidos, las nuevas solicitudes deben ir a Inventario 1. El intervalo es un segundo y el umbral es dos fallos consecutivos; no equivale a una garantía de tiempo real exacto. Repite la petición hasta observar diez respuestas correctas seguidas de la instancia superviviente.

Reinicia Inventario 2 sin reiniciar el gateway. Después de un sondeo satisfactorio debe reaparecer su identidad en las respuestas. Detén ambas instancias: cuando no queden destinos disponibles el gateway devuelve 503, sin inventar stock. Reinicia una y comprueba la recuperación. RoundRobin reparte peticiones entre destinos disponibles; no distribuye por pesos ni garantiza la misma duración de trabajo de cada instancia.

## Eventos y persistencia del sistema completo

Este recorrido se realiza íntegramente en `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`, con PowerShell 7, SQL Server LocalDB y el servicio RabbitMQ de `compose.yml`. No requiere consultar otro módulo. La orden DDD creada aquí es distinta de las órdenes básicas de `/api/v1/ordenes`: se crea en `/api/v1/produccion/ordenes`, se le añade una segunda línea y se planifica mediante `/api/v1/comunicacion/ordenes/{id}/planificar` para activar el flujo HTTP más outbox/inbox.

Abre cinco terminales PowerShell: A para Inventario 1, B para Inventario 2, C para Producción, D para Gateway y E para las solicitudes y verificaciones. Si siguen ejecutándose los procesos del recorrido anterior, detenlos con Ctrl+C en sus terminales antes de cambiar variables. En cada terminal, desde la raíz del repositorio, entra en la etapa:

```
Set-Location M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga
```

Sitúa todos los comandos y proyectos relativos en la etapa final del módulo.

### Iniciar LocalDB y RabbitMQ

En E ejecuta el bloque siguiente. El servicio Compose se llama `rabbitmq`; la imagen heredada es `rabbitmq:4.1.5-management` y publica AMQP en 5672 y la consola de administración en 15672. Si ya utilizas el broker didáctico de una etapa anterior en 5672, utiliza esa instancia con la misma URI y credenciales, o detén aquella instancia antes de iniciar esta. No elimines el volumen del broker ni las bases durante la comprobación de persistencia.

```
sqllocaldb start MSSQLLocalDB
if ($LASTEXITCODE -ne 0) { throw "No se pudo iniciar MSSQLLocalDB." }
docker compose up -d rabbitmq
if ($LASTEXITCODE -ne 0) { throw "No se pudo iniciar RabbitMQ." }
$brokerListo = $false
foreach ($intento in 1..30) {
    docker compose exec -T rabbitmq rabbitmq-diagnostics -q ping
    if ($LASTEXITCODE -eq 0) { $brokerListo = $true; break }
    Start-Sleep -Seconds 2
}
if (-not $brokerListo) { throw "RabbitMQ no respondió al diagnóstico." }
```

Inicia la instancia LocalDB con la que trabajarán los servicios Windows. Detiene el procedimiento si falla la herramienta de SQL Server. Inicia el servicio RabbitMQ declarado en compose.yml conservando su volumen.

No continúa si Docker informa de un fallo de creación o arranque. Registra si el broker ha respondido a su comprobación de salud. Permite hasta treinta comprobaciones durante el arranque del broker.

Ejecuta el diagnóstico dentro del broker, sin requerir terminal interactiva. Confirma disponibilidad solo cuando el diagnóstico devuelve código cero. Espera entre comprobaciones mientras RabbitMQ termina de arrancar.

Exige un broker disponible antes de activar productores y consumidores.

## Configurar y arrancar los cuatro procesos

Copia este bloque completo en A, B y C antes de iniciar sus aplicaciones. Las variables de una terminal no se transfieren a las demás. En PowerShell la barra inversa no es un carácter de escape: `(localdb)\MSSQLLocalDB` lleva una sola barra. Las dos instancias de Inventario usan exactamente `AceriaM08Inventario`; cada contexto de Producción conserva su propia base. `Development` permite la inicialización didáctica mediante EnsureCreated. El prefijo RabbitMQ debe ser idéntico en los tres procesos y conservarse al reiniciar.

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
$env:ConnectionStrings__AceriaApp = "Server=(localdb)\MSSQLLocalDB;Database=AceriaM08Api;Integrated
Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaHexagonal = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Hexagonal;Integrated Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaProduccion = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Produccion;Integrated Security=True;TrustServerCertificate=True"
$env:ConnectionStrings__AceriaInventario = "Server=(localdb)
\MSSQLLocalDB;Database=AceriaM08Inventario;Integrated Security=True;TrustServerCertificate=True"
$env:Comunicacion__InventarioUrl = "http://localhost:5106/"
$env:Comunicacion__RabbitMqUri = "amqp://curso:curso-local@localhost:5672/"
$env:Comunicacion__Prefijo = "aceria-m08-manual"
$env:Comunicacion__WorkersActivos = "true"
```

Selecciona la inicialización didáctica de bases y tablas prevista por los servicios. Configura la base del API REST acumulado, con autenticación de Windows. Configura la base independiente del contexto hexagonal.

Configura la base de las órdenes DDD y del outbox transaccional de Producción. Configura la base compartida de existencias, proyecciones e identificación de eventos recibidos. Fija el cliente interno de Producción hacia Inventario 1 para la consulta informativa de disponibilidad.

Configura el broker real de laboratorio para la publicación y recepción de eventos. Agrupa los recursos RabbitMQ de este recorrido; el mismo valor enlaza productor y consumidores. Activa el publicador outbox de Producción y los consumidores de Inventario.

En A inicia primero Inventario 1:

```
$env:ASPNETCORE_URLS = "http://localhost:5106"
$env:Inventario__Instancia = "inventario-1"
dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile
```

Reserva el puerto docente de Inventario 1. Identifica sus respuestas mediante la cabecera X-Instancia. Inicia el servicio real y conserva el puerto definido en esta terminal.

En E espera a que `Invoke-RestMethod http://localhost:5106/health/ready` responda con `estado` igual a `Healthy`. Así la primera instancia termina de crear y sembrar la base antes de arrancar la segunda. En B:

```
$env:ASPNETCORE_URLS = "http://localhost:5107"
$env:Inventario__Instancia = "inventario-2"
dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile
```

Asigna un puerto distinto a la segunda instancia que comparte el modelo y la base. Permite observar el segundo destino del balanceador. Inicia el segundo consumidor sobre el mismo broker y la misma base Inventario.

En C:

```
$env:ASPNETCORE_URLS = "http://localhost:5105"
dotnet run --project src/AceriaApp.Api --no-launch-profile
```

Publica Producción en el destino que conoce la configuración YARP. Inicia Producción con sus bases SQL y el publicador outbox activado.

En D:

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
dotnet run --project src/AceriaApp.Gateway --no-launch-profile -- --urls http://localhost:5100
```

Selecciona el entorno docente del proceso de entrada. Inicia YARP en el puerto público explícito; los destinos del appsettings de 8.4 son 5105, 5106 y 5107.

En E comprueba todos los procesos antes de escribir una orden:

```
$gateway = "http://localhost:5100"
$vivo = Invoke-RestMethod "$gateway/health/live"
if ($vivo.servicio -ne "Gateway" -or $vivo.estado -ne "vivo") { throw "Gateway no está vivo." }
foreach ($puerto in 5105,5106,5107) {
    $salud = Invoke-RestMethod "http://localhost:$puerto/health/ready"
    if ($salud.estado -ne "Healthy") { throw "Backend $puerto no está preparado." }
}
(Invoke-RestMethod "$gateway/api/v1/materiales/Acero").stock
```

Define la única entrada pública utilizada por las operaciones de negocio siguientes. Consulta la liveness propia del proceso gateway. Respeta el contrato de liveness de Gateway, diferente del contrato Healthy de los backends.

Recorre Producción y las dos instancias de Inventario. Comprueba directamente SQL y RabbitMQ en el backend que se va a utilizar. Detiene el recorrido si una dependencia no está disponible.

Comprueba el recorrido YARP hacia Inventario; la base nueva contiene stock 300 de Acero.

## Crear una orden de dos líneas y planificarla

Mantén E abierta hasta terminar el recorrido: conserva `$ordenId`, `$eventoId` y las URI necesarias para comparar antes y después del reinicio. La creación incorpora la primera línea de Acero 20; el POST a `/lineas` añade Inoxidable 10. La cantidad total esperada es 30.

```
$cuerpo = @{ cliente = "Axia M08 eventos"; material = "Acero"; cantidad = 20 } | ConvertTo-Json
$creada = Invoke-WebRequest -Uri "$gateway/api/v1/produccion/ordenes" -Method Post -ContentType "application/
json" -Body $cuerpo -SkipHttpErrorCheck
if ($creada.StatusCode -ne 201) { throw "Creación esperaba 201: $($creada.Content)" }
$ordenId = [string](($creada.Content | ConvertFrom-Json).id)
if ([guid]$ordenId -eq [guid]::Empty) { throw "La orden no tiene un identificador válido." }
$ordenUri = "$gateway/api/v1/produccion/ordenes/$ordenId"
$segundaLinea = @{ material = "Inoxidable"; cantidad = 10 } | ConvertTo-Json
$agregada = Invoke-WebRequest -Uri "$ordenUri/lineas" -Method Post -ContentType "application/json" -Body
$segundaLinea -SkipHttpErrorCheck
if ($agregada.StatusCode -ne 200) { throw "Añadir línea esperaba 200: $($agregada.Content)" }
$borrador = Invoke-RestMethod $ordenUri
if ($borrador.estado -ne "Borrador" -or $borrador.cantidadTotal -ne 30 -or @($borrador.lineas).Count -ne 2)
{ throw "La orden no tiene sus dos líneas y total 30." }
$planificarUri = "$gateway/api/v1/comunicacion/ordenes/$ordenId/planificar"
$aceptada = Invoke-WebRequest -Uri $planificarUri -Method Post -ContentType "application/json" -Body "{}"
-SkipHttpErrorCheck
if ($aceptada.StatusCode -ne 202) { throw "Planificación esperaba 202: $($aceptada.Content)" }
$confirmacion = $aceptada.Content | ConvertFrom-Json
if ($confirmacion.ordenId -ne $ordenId -or $confirmacion.estado -ne "Planificada") { throw "La aceptación
pertenece a otra orden o estado." }
$repetida = Invoke-WebRequest -Uri $planificarUri -Method Post -ContentType "application/json" -Body "{}"
-SkipHttpErrorCheck
```

```
if ($repetida.StatusCode -ne 409) { throw "La segunda planificación esperaba 409: $($repetida.Content)" }
$confirmacion
```

Serializa el contrato de creación de una orden DDD y su primera línea. Envía la creación a Producción a través del gateway y conserva la respuesta HTTP. Exige creación real y muestra el cuerpo si el contrato falla.

Extrae el identificador devuelto en el cuerpo id de la creación. Verifica que el identificador corresponde a un GUID no vacío. Construye la consulta pública de esta orden concreta.

Serializa la segunda línea del agregado, con material distinto y cantidad10. Solicita el cambio del agregado mediante su endpoint de líneas. Comprueba que la segunda línea fue aceptada antes de planificar.

Lee el agregado persistido mediante la entrada pública. Contrasta estado, número de líneas y suma antes de realizar la transición. Elige el flujo integrado que consulta disponibilidad y escribe la planificación con outbox.

Solicita la planificación una vez; el cuerpo vacío mantiene explícito el contrato utilizado por el smoke. Exige la aceptación del flujo; una respuesta distinta no se trata como éxito. Lee ordenId, estado y propagacion de la aceptación.

Relaciona el202 con la misma orden que se creó y planificó. Repite expresamente la operación para comprobar que una orden ya planificada no vuelve a transicionar. Comprueba el conflicto del segundo intento, conservando el código real del backend.

Muestra la aceptación original; propagacion Pendiente no implica que el consumidor ya haya recibido el evento.

El202 confirma la planificación y la escritura transaccional del evento pendiente. El publicador outbox envía al broker y los consumidores registran la proyección con su identificación de evento. El409 del segundo intento impide una segunda transición; no es el mecanismo de deduplicación del consumidor. La consulta de disponibilidad es informativa y no reserva ni descuenta stock.

## Esperar el evento recibido

Sigue en E. Mientras la entrega asíncrona está pendiente, la consulta de la proyección puede responder404. El bloque repite únicamente esa lectura y exige un200 con la orden, el total y el identificador del evento correctos.

```
$proyeccionUri = "$gateway/api/v1/planificaciones/$ordenId"
$limite = [DateTime]::UtcNow.AddSeconds(60)
$proyeccion = $null
while ([DateTime]::UtcNow -lt $limite) {
    $respuesta = Invoke-WebRequest -Uri $proyeccionUri -SkipHttpErrorCheck
    if ($respuesta.StatusCode -eq 200) { $proyeccion = $respuesta.Content | ConvertFrom-Json; break }
    if ($respuesta.StatusCode -ne 404) { throw "Consulta de proyección falló: $($respuesta.StatusCode) $
($respuesta.Content)" }
    Start-Sleep -Milliseconds 500
}
if ($null -eq $proyeccion) { throw "No llegó la proyección dentro del límite." }
if ($proyeccion.ordenId -ne $ordenId -or $proyeccion.cantidadTotal -ne 30) { throw "La proyección no coincide
con la orden de dos líneas." }
$eventoId = [string]$proyeccion.eventoId
if ([guid]$eventoId -eq [guid]::Empty) { throw "La proyección no identifica su evento." }
$proyeccion | Format-List eventoId,ordenId,cantidadTotal
```

Construye la consulta pública de la proyección que mantiene Inventario. Fija un límite de espera de sesenta segundos para la entrega asíncrona. Inicializa la evidencia observada, sin asumir que el evento ya se ha recibido.

Repite las lecturas hasta obtener la proyección o agotar el límite. Consulta a Inventario a través del gateway y conserva un404 como respuesta HTTP comprobable. Solo acepta y deserializa una proyección cuando el backend responde200.

Distingue la ausencia temporal de otros errores;503 o500 no se ocultan como entrega pendiente. Espacia las lecturas mientras publicador y consumidor completan la entrega. Hace visible que no se ha demostrado la entrega si nunca aparece un200.

Exige identidad y cantidad, evitando aceptar un evento de otra operación. Conserva el identificador de la entrega realmente observada. Exige un identificador de evento GUID no vacío.

Muestra la evidencia que debe conservarse igual después del reinicio.

Con ambos consumidores activos, la entrega puede procesarla cualquiera de las instancias. Las dos leen la misma base `AceriaM08Inventario`, por lo que la respuesta debe ser equivalente con cualquiera de los valores de X-Instancia. La tabla de planificaciones mantiene `EventoId` como clave y un índice único por `OrdenId`; el receptor compara la huella del contenido si se vuelve a entregar el mismo evento. El gateway no publica mensajes ni sustituye ese trabajo.

## Reiniciar los servicios y comprobar persistencia

Detén A, B y C con Ctrl+C. Conserva D y E abiertas. No elimines bases, tablas, volumen, prefijo ni identificadores. En A vuelve a ejecutar su mismo `dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile`; sus variables de entorno siguen presentes. En E comprueba nuevamente que `http://localhost:5106/health/ready` informa Healthy. Después repite el mismo comando de Inventario en B y el de Producción en C. Espera readiness Healthy en5105 y5107 antes de ejecutar el bloque siguiente en E. YARP recupera los destinos mediante sus sondeos, sin reiniciar el gateway.

```
$guardada = Invoke-RestMethod $ordenUri
if ($guardada.id -ne $ordenId -or $guardada.estado -ne "Planificada" -or $guardada.cantidadTotal -ne 30 -or
@($guardada.lineas).Count -ne 2) { throw "La orden persistida no conserva estado, líneas y total." }
$recibida = Invoke-RestMethod $proyeccionUri
if ($recibida.ordenId -ne $ordenId -or $recibida.cantidadTotal -ne 30 -or $recibida.eventoId -ne $eventoId)
{ throw "La proyección persistida cambió o desapareció." }
$guardada | Format-List id,estado,cantidadTotal,lineas
$recibida | Format-List eventoId,ordenId,cantidadTotal
```

Vuelve a obtener la misma orden después de reiniciar el proceso que la gestiona. Exige que SQL haya conservado el agregado completo y su transición, además del identificador. Consulta la proyección ya almacenada después de reiniciar ambos procesos de Inventario.

Comprueba que se conserva la entrega original, no una nueva planificación con un identificador diferente. Muestra la orden SQL recuperada para compararla con el estado previo. Muestra la proyección SQL recuperada con el mismo evento que se observó antes.

**Resultado esperado.** La orden conserva estado Planificada, dos líneas y cantidadTotal30; la proyección conserva el mismo ordenId, eventoId y total 30. Estos resultados se obtienen consultando procesos reiniciados y las mismas bases. Si el proceso todavía no ha completado su arranque o el gateway no ha restituido un destino, termina primero las comprobaciones de readiness y repite las lecturas. No vuelvas a crear la orden ni a planificarla para corregir una comprobación de persistencia.

**Diagnóstico si falla.** Un404 permanente en la proyección exige revisar que los tres procesos se reiniciaron con WorkersActivos=true, la misma RabbitMqUri y el mismo Prefijo. Comprueba también que Inventario1 y2 usan exactamente la misma base. Un409 en el primer intento de planificación indica que esa orden ya cambió de estado; crea una orden nueva para un recorrido nuevo, sin alterar la que se conserva como evidencia. Un503 al consultar disponibilidad exige recuperar Inventario1 en5106, que es el destino interno del flujo integrado. Un error de conexión SQL requiere comprobar LocalDB y sus permisos; cambiar el proveedor no acredita persistencia.

## Reconstrucción y comprobación del material

Desde la raíz del repositorio, `python .github/scripts/m08_e2e_desde_practicas.py --no-tests --out .qa/m08-reconstruccion.json` reconstruye las cuatro etapas a partir de los bloques de este manual. Su resultado es SIN_TESTS. Para validar con Docker, ejecuta `dotnet run --project tools/M08ValidationHost -c Release -- python .github/scripts/m08_validar.py`: se comprueban SQL Server, RabbitMQ, las etapas, los retos y el tráfico entre procesos. Cualquier fallo debe resolverse antes de declarar el módulo cerrado.

## Cierre del curso

El recorrido conserva ocho módulos y 34 puntos. El proyecto final reúne APIs REST, arquitectura hexagonal, patrones, DDD/CQRS, HTTP y eventos, observabilidad, resiliencia y publicación mediante un gateway. Los catálogos ayudan a razonar sobre decisiones; la ejecución y sus evidencias permiten distinguir qué comportamiento se ha implementado y comprobado. En la analogía de la acería, el gateway organiza la entrada a las líneas de trabajo, mientras cada línea conserva su responsabilidad y su estado; la recepción no fabrica el producto ni sustituye el registro de producción.